Table of Contents

Richa Singh — Behavioural Interview Prep

Analytics Manager | Flipkart | 10 Years Experience Coupang Leadership Principle Edition

TELL ME ABOUT YOURSELF

The opener. ~100 words, about 45 seconds spoken. Learn the shape, not the wording.

"I've been at Flipkart for ten years, and my work has largely been on the three things a marketplace competes on — speed, price and customers.

On speed, I found why 60% of demand wasn't being served locally and recovered 10 points of cluster fulfilment in two months. On price, I owned the measurement for our first automated pricing pilot across 10M+ listings, under CEO and CFO review. On customers, my uplift model halved monthly couponing spend while keeping 80% of the new customers.

Today I lead agentic analytics for Flipkart Minutes. I tend to build the measurement layer before the analysis."

Delivery notes

- **Three levers, one line each, then present role, then the thread.** Not chronological — ten years in order takes four minutes and lands nowhere.
- **The last line is the point.** It sets up Dive Deep, Think Systematically and Demand Excellence for whoever asks next.
- **Leave hooks open.** One sentence per lever. If they want the pricing story they'll ask, and you have twelve minutes of it ready.
- **Reorder for the interviewer.** Supply chain or ops → lead with fulfilment. Growth or category → lead with couponing. Platform or engineering → lead with the GenAI work.
- **Resist adding more.** The retention model, NC budget framework and BBD all want to be in here. Put them in and the numbers stop registering.

QUICK INTRO

Data and Analytics professional with **10+ years** of experience at Flipkart across customer growth, pricing automation, event analytics, and agentic analytics product development. Currently Analytics Manager leading GenAI/agentic flows for Flipkart Minutes and the central GenAI pod. Prior to this, led New Customer (NC) analytics, Event analytics, and Retail Placement analytics.

Project Arsenal (used throughout this document):

- **Multi-agent RCA system (GenAI)** — Google ADK + Claude agentic flow for Flipkart Minutes OPD root-cause diagnosis
- **Scalable Agentic Framework** — Looker semantic layer + BQ context store for enterprise-scale agentic analytics
- **Pricing Automation Analytics** — First CFO-level automated pricing Pilot across 10M+ listings
- **Cluster Fulfillment (CF) Improvement** — Deep-dive and governance framework recovering 10% CF in 2 months
- **Intelligent Couponing (Uplift Model)** — S-Learner XGBoost model reducing couponing spend 55% while preserving 80% NC
- **NC Budget Allocation Framework** — Cohort-level simulator unlocking 30 Cr incremental revenue on 11 Cr budget
- **Event Analytics** — Long-term impact of events, BBD control tower, Intrigue HVA strategy
- **Customer Retention Driver Model** — XGBoost model for Low-TPC customers, 100 bps M1R improvement
- **AOP Simulators & Customer Metric Governance** — Bottom-up organic projections, gap identification

HOW TO USE THIS DOCUMENT

Every question below now carries four things before the answer:

1. **Coupang Leadership Principle it maps to** — the primary principle the interviewer is almost certainly probing, plus the secondary principle that usually rides along with it.
2. **The principle in plain English** — what Coupang actually means by it, stripped of the careers-page phrasing.
3. **How interviewers may frame it** — the different question wordings that are all secretly testing the same principle. If you hear any of them, this is the story to reach for.
4. **The answer itself** — Story → Why this is the best fit → 2-Minute Executive Summary → Full STAR Version → What Interviewers Will Probe → Why this story screams the principle.

The 15 Coupang Leadership Principles, for orientation: Wow the Customer · Ruthless Prioritization · Company-wide Perspective · Dive Deep · Think Systematically · Simplify · Disagree and Commit · Hire and Develop the Best · Deliver Results with Grit · Influence without Authority · Aim High and Find a Way · Learn Voraciously · Demand Excellence · Move with Urgency · Hate Waste.

SECTION 1: LEADERSHIP & INITIATIVE

Q1. Tell me about a time you led a project without formal authority.

Coupang Leadership Principle: INFLUENCE WITHOUT AUTHORITY *(secondary: Demand Excellence)*

The principle in plain English: Coupang expects you to get a room full of senior people to move because your argument is better, not because your title says they have to. Their phrasing is that leaders drive alignment through data and insights, and create an environment where knowledge beats position. In practice the interviewer is testing one thing: when you had no lever to pull — no reporting line, no budget control, no veto — what did you actually do to change the outcome? Answers that lean on escalation, or on "my leadership backed me," score badly. Answers where the data itself became the authority score well.

How interviewers may frame this question:

- Tell me about a time you led a project without formal authority.
- Describe a situation where you had to drive alignment across teams that didn't report to you.
- Tell me about a time you had to convince a senior stakeholder who outranked you.
- Give me an example of influencing a decision where you had no decision rights.
- How do you get buy-in from a team whose incentives conflict with yours?
- Tell me about a time you were the most junior person in the room but changed the outcome.
- Describe a time you had to be the neutral party between two teams that both wanted a different answer.

Story: Pricing Automation Analytics — CFO-level Pilot across 10M+ listings

Why this is the best fit:

This is the cleanest Influence without Authority story in the arsenal, for three reasons. First, the authority gap was structural, not incidental — 30+ stakeholders across Product, Engineering, Program, BizFin, Data Science, Category and Operations, and I had no line authority over a single one of them. Second, every party in the room was *biased by design*: Category had owned manual pricing for years and stood to lose control, Data Science and Product were invested in their own product succeeding, BizFin was protecting P&L. There was no neutral party until analytics became it. Third, the CEO and CFO were personally in the room, which meant the only currency that worked was defensible measurement — anyone can be out-argued, but nobody can out-argue reaction-ID-level instrumentation. The role I was given was explicitly the unbiased governance body, and I held that stance for the full length of the Pilot programme.

2-Minute Executive Summary:

“Flipkart’s 1P catalog had 10M+ listings with 90–95% of pricing done manually by category teams on intuition. Competition was significantly ahead on systemic pricing, so the company had invested in an outsourced automated pricing product, and the CEO approved a Pilot at Super Category level during large sale events — with the CEO and CFO personally reviewing the results. The problem was that every one of the 30+ stakeholder teams had a stake in the answer: Category was biased against automation because it meant ceding pricing control, Product and Data Science were biased toward their own product, BizFin was watching P&L. There was no single owner and no neutral measurement body. I was brought in from Analytics to be exactly that — with no authority over any of these teams. I owned three workstreams: end-to-end instrumentation across both pricing products so the system could be measured as a whole rather than two isolated tools; a four-layer governance framework that defined what success actually meant for automated pricing; and the deep-dive into ASP deviation, which the instrumentation revealed as the single biggest driver of GMV and CM misses. The outcome was that ASP deviation fell from ~70% to ~30% of GMV, the CI-match portfolio was rationalised from 70–80% down to ~30–40% through a standardised KVI definition, ~₹500 Cr of bad burn was quantified and mitigated at BBD scale, and the framework became the standard template every subsequent Pilot was evaluated against. I was trusted to hold the neutral analytical line across all 30+ stakeholders without formal authority over any of them.”

Full STAR Version

I would like to talk about one of my projects in Pricing Automation charter which is one of the most career defining projects in my stint at FLipkart. This project involved both CEO and CFO reviews who were directly involved in understanding its nuance and progress in detail. At the same time this project gave me an experience of working with 30+stakeholders across product, Eng. Data Science, Biz Fin and Category. I would like to set up the context before we delve deeper into the detail.

SITUATION

Flipkart’s 1P catalog carried 10M+ listings, and 90–95% of pricing decisions were made manually by category teams using intuition and heuristics. That created three systemic P&L problems: bad burn investments driven by low-RPC pricing decisions, suboptimal pricing not grounded in supply-demand or elasticity signals, and frequent manual errors causing margin and revenue leakage. Meanwhile competition — Amazon in particular — was materially ahead on systemic automated pricing. Flipkart had made a substantial investment in outsourcing a pricing product, which is precisely why the CEO and CFO were involved in this Pilot project

The organisational problem was harder than the analytical one. Moving from manual to automated pricing touched every part of the org and each team arrived with a stake:

- **Category** had owned manual pricing forever and was biased against using the product at all — automation meant ceding control.
- **Data Science and Product** were naturally biased toward their product being clearly better than manual pricing.

- **BizFin** was involved because of the direct P&L impact.
- **Engineering** owned making the product work inside Flipkart's ecosystem.

With every party carrying significant bias on whether the product would work, the initiative needed one unbiased governance body that could work across all of them, surface the true picture of whether automated pricing was genuinely better than manual, and define the measurement framework to monitor it.

TASK

This is when my leadership brought me in to own the analytics workstream. Not because I had authority over any of these teams, but because the role demanded someone who could work across all parties, surface the true picture, and have the conviction to hold that stance while others pushed their own narratives. Three interconnected workstreams:

1. Design and drive end-to-end data instrumentation across both products so the system could be measured as a whole, not as two isolated tools.
2. Build the governance and measurement framework that defined what "success" meant for automated pricing, provided early warning signals, and guided product improvements across every Pilot.
3. Lead the analytical deep-dive into ASP deviation — which the instrumentation revealed as the single biggest driver of GMV and CM misses — including scoping, root-causing, and designing the feedback loop fix.

I additionally owned the measurement frameworks for two supporting tools: the Spend Management Tool and the Unit Capping Tool.

Following a business case built on 1P GMV portfolio sizing, the CEO approved a Pilot of automated pricing at the Super Category level during large sale events. The product objective was:

> **Primary Objective (used in 95%+ of Pilots)**
> Maximize Super Category GMV while keeping SC-level Contribution Margin (CM) as a guardrail.

Two products were built to work in tandem to enable end-to-end automated pricing:

- **Optimizer:** Used historical data + event cost inputs (NLC, BO, SMNM, logistics, CX costs) to recommend the best FSN × ASP × FSP combination that maximizes GMV within CM constraints. Also predicted unit, GMV, and CM outcomes per FSN.
- **Offer Intelligence (OI):** Bridged the FSP → ASP gap that the Optimizer could not solve for. Using BCD (Blended Conditional Discount) projection from mandatory and conditional offers, OI recommended which offers to map or unmap against each listing to achieve the Optimizer's target ASP.

ACTION

Step 1 — Won the argument that instrumentation was as important as the product. The initial instrumentation had been built independently by each product's engineering team. Each product worked end-to-end in isolation, but there was no shared data layer capturing the system-level story from price recommendation → offer mapping → realised ASP. This was my first influence-without-authority problem, because asking two engineering orgs to build shared instrumentation is asking them to spend sprint capacity on something that isn't their own product's feature list. I mapped the full end-to-end journey of both Optimizer and OI — reaction ID generation → FSP/ASP recommendation → BCD projection → offer go-live → actual BCD → realised ASP — and defined the exact metrics and granularity needed to stitch it: at **reaction ID × FSN level**, capturing FSP, ASP, predicted DRRs, reaction go-live time, offer go-live time, projected mandatory BCD, non-mandatory BCD, actual BCD, and actual GMV and CM. Then I ran multiple alignment rounds with Product, Engineering, Program and leadership across both product orgs, arguing that for a first-of-its-kind Pilot under CEO scrutiny, an unmeasurable product is indistinguishable from a failed one.

Step 2 — Built a four-layer governance framework that was harder to argue with than I was. With category teams resistant and the CEO watching, the framework had to be rigorous enough that no party could selectively reinterpret it.

Pricing_Automation_STAR

- **Layer 1 — Target sanity check.** Before each Pilot, I validated whether the GMV and CM targets set by Category were achievable, by benchmarking against similar-scale historical events with comparable cost inputs. This mattered because Category had an incentive to set inflated targets — a failed Pilot would discredit automation. Where targets looked inflated I escalated to BizFin and Category leadership to align on realistic ones. Crucially, this layer also protected Category's legitimate interest, which is what made it credible rather than adversarial.
- **Layer 2 — GMV/CM achievement plus portfolio decomposition.** I decomposed the portfolio by pricing mode — Optimizer-priced vs CI price-matched vs manual — and found that **70%+ of the portfolio was being held on CI match** by categories, severely limiting the Optimizer's scope. That single decomposition triggered a leadership-level discussion on standardising the definition of KVIs (Key Value Items). I led that analytical work using two signals — Search PPVs as an organic demand proxy and Price Elasticity from the Optimizer model — built a 3×3 grid across high/low Search PPV and price sensitivity, and aligned the org on a standard: very high sensitivity items and very high demand items go on CI match regardless of the other dimension. KVIs turned out to be only **3–4% of FSN count but 30–40% of SC GMV**, which created real room for the Optimizer on the rest of the portfolio.
- **Layer 3 — ASP adherence metrics.** I defined ASP Adherence as the GMV share where actual ASP landed within ±2% of the model-recommended ASP, and separately tracked positive deviation (CM over-achievement risk) and negative deviation (CM bleed risk). I then decomposed deviation into four buckets by go-live sync — both price and offer live, price live but offer not, offer live but price not, neither live — which was only possible because of the timestamps captured during

instrumentation. Finding: only **80% of GMV had price and offer live in sync**, and those cases showed materially lower deviation. That became an Engineering fix on the synchronisation pipeline.

- **Layer 4 — Customer-facing price fluctuation guardrail.** I introduced price fluctuation as a guardrail — frequency, magnitude and direction of price changes during events at FSN level, rolled up on unit weight. It surfaced that the sheer volume of Optimizer reactions was causing significantly higher price volatility than manual pricing, raising a price-perception risk on the platform. This became the input that led the Optimizer team to introduce reaction throttling. This was the first customer-facing metric anyone had put into the automated pricing framework.

Step 3 — Held the line on the real root cause when it was inconvenient. The instrumentation showed ~70% of GMV experiencing ASP deviation beyond ±2%, and the root cause sat at the OI end: BCD projection built on coarse historical event offer adoption with no feedback loop and no granularity optimisation. This was uncomfortable for the product teams pushing favourable readings of early Pilot results — some metrics genuinely did look positive, but ASP deviation was masking the real picture. I decomposed deviation into BCD projection error versus left-over gap (the unresolvable gap between FSP and ASP even after full offer mapping) and found left-over gap exceeded 10% for a significant share of GMV, meaning the system was structurally unable to hit target ASP in those cases. Extensive EDA across historical comparable events (May BSD, Jan RD, Aug ID Sale) computing offer adoption at multiple product-attribute × offer-attribute combinations showed the key error source: **Bank Tier × Payment Instrument** adoption differed significantly, and applying Bank-Tier-level adoption to Bank Tier × Instrument combinations was a major source of error. I recommended a PBO granularity of Event × Vertical × Price Tier × Bank Tier × Payment Instrument with a defined fallback chain, plus a live feedback loop: static adoption for the first 8–10 hours (live cumulative adoption is noisy early), then switch to live adoption with a trigger to update whenever deviation crosses 2%.

Step 4 — Quantified the money to make the case unignorable. For the Spend Management Tool I built a framework measuring bad burn savings per order — comparing the minimum blocked price that could have gone live against the minimum approved price that actually went live, with burn saved = (min approved − min blocked) × units sold — which sized **~₹500 Cr** of bad burn risk at BBD scale. For the Unit Capping Tool I measured price-switchover efficiency during event laddering on two dimensions: timing (the tool switched earlier than manual, accounting for orders in the payment funnel) and lag (reduced from minutes to ~10 seconds), quantified at FSN level — **~₹13 Cr on iPhone 15 alone in a single event**.

RESULT

- **ASP deviation reduced from ~70% to ~30%** of GMV beyond ±2% — directly improving the GMV and CM predictability of the entire Pilot programme.
- **CI-match portfolio rationalised from 70–80% to ~30–40%** via a defensible, data-backed KVI standard, meaningfully expanding the Optimizer's effective scope.

- **~₹500 Cr bad burn** quantified and mitigated at BBD scale; **~₹13 Cr** unit-capping saving on a single FSN in a single event.
- **First cross-product data layer** connecting Optimizer and OI, which became the foundation for every future Pilot review.
- **Engineering fix on go-live synchronisation**, after the instrumentation showed only 80% of GMV had price and offer live in sync.
- **Price fluctuation adopted as a guardrail** — shifting the product team's thinking beyond internal GMV/CM to platform price perception.
- The governance framework became **the standard evaluation template for every Pilot**, giving leadership a consistent lens to compare Pilots across Super Categories and to decide whether to expand or pause automated pricing on evidence rather than intuition.

KEY LEARNING

Leading without authority is not about persuasion technique — it is about becoming the only unbiased source of truth in a room where everyone else has a stake. When every party is biased, the team with the clearest instrumentation and the most defensible framework earns the leadership role regardless of the org chart. The second lesson is subtler: the framework has to protect the resisting party's legitimate interest too. My target sanity check layer existed partly to stop Category being handed unachievable targets. That is what turned the framework from something they were policed by into something they relied on.

What Interviewers Will Probe

- **How did you handle pushback from Category teams who resisted automation?** I didn't fight their bias head-on — I built a measurement framework rigorous enough that arguing with the data was harder than arguing with me. Then I showed them the framework protected them too: the target sanity check meant no Pilot would run against an inflated target, and if automated pricing genuinely underperformed, the framework would show that clearly. I also shared the framework with Category before taking it to leadership, so they were included rather than policed.

- **How did you manage 30+ stakeholders with conflicting incentives?** Structured alignment rounds with each party separately first, to understand their specific concern rather than their stated objection. Then one unified framework that visibly addressed each of those concerns. Category's real fear was accountability for a number they no longer controlled — that became Layer 1. Product's real fear was that the Pilot would be judged on a metric they couldn't influence — that became the four-bucket ASP deviation decomposition, which separated their product error from pipeline sync issues.

- **What was your hardest moment on this project?** When Data Science pushed a favourable interpretation of early Pilot results. Some metrics genuinely did look positive. Holding the stance required the instrumentation data showing ASP deviation was masking the real picture — and, importantly, framing it as "the

product is being under-served by the pipeline" rather than "the product is failing," which was both truer and easier to act on.

- **Why did you start with instrumentation rather than analysis? Didn't that delay everything?** It delayed the first analysis by weeks and I made that trade-off knowingly. In a multi-causal system with competing teams pointing at different root causes, an analysis without traceability just becomes another opinion. Once we could trace reaction ID → FSP → BCD projection → offer go-live → realised ASP, attribution stopped being debatable. Every week spent on instrumentation saved multiple rounds of argument later.
- **Wasn't the CEO/CFO involvement itself your authority?** It was the opposite — it raised the bar. Leadership attention meant every party had a stronger incentive to push their own narrative, not a weaker one. And leadership visibility cuts both ways: if I'd escalated every disagreement, I'd have burned the credibility I needed to be the neutral party. I escalated exactly one thing — inflated targets — and only with benchmarked evidence attached.
- **What would you have done if the framework had shown automated pricing was worse than manual?** Reported it exactly as clearly as I reported the wins. That was the whole point of being the neutral body, and it's why I put the price-fluctuation guardrail into the framework myself — it was the metric most likely to make the product look bad, and including it before anyone asked was what made the rest of the framework credible.

Why This Story Screams Influence without Authority

- ✔ Zero line authority over any of the 30+ stakeholders whose behaviour changed
- ✔ Alignment driven purely through instrumentation, decomposition and benchmarking — knowledge over position
- ✔ Built the framework to protect the resisting party's legitimate interest, not just to win the argument
- ✔ Held the analytical stance under CEO/CFO scrutiny when better-resourced teams pushed competing narratives
- ✔ Included the metric most damaging to the product (price fluctuation) voluntarily — credibility earned, not claimed
- ✔ The framework outlived the argument: it became the standard template for every subsequent Pilot

---

Q2. Describe a situation where you identified a problem and took initiative to solve it.

Coupang Leadership Principle: COMPANY-WIDE PERSPECTIVE *(secondary: Dive Deep)*

The principle in plain English: Coupang wants leaders who act like owners of the whole company rather than of their own scorecard — who trace upstream and downstream

consequences, and who raise problems that sit outside their own department instead of saying "that's not my job." The interviewer is checking whether you go looking for problems nobody assigned to you, and specifically whether you're willing to pick up a problem whose fix lives entirely in someone else's team. Answers where you solved something inside your own remit are fine but weaker; answers where you owned a problem you had no mandate to own are what this principle rewards.

How interviewers may frame this question:

- Describe a situation where you identified a problem and took initiative to solve it.
- Tell me about a time you solved a problem that wasn't your responsibility.
- Give me an example of when you acted like an owner of the whole business rather than of your own team's metrics.
- Tell me about a time you raised a problem in another team's area.
- Describe a time you considered the upstream or downstream impact of a decision others had missed.
- Tell me about a time you went beyond your job description.
- Have you ever taken on a problem knowing you couldn't fix it yourself?

Story: Cluster Fulfillment Improvement — diagnosing 8 weeks of 0% YoY, 0% WoW CF stagnation

Cluster_Fulfillment_STAR

Why this is the best fit:

My actual mandate in the Retail charter was narrow: report the health of L0 metrics. The existing reporting already did its job — it told the Retail Placement team *which* category, *which* state, *which* city was underperforming on Cluster Fulfillment. What it could not do was tell them *why*, and structurally fixing CF needed exactly that. So I picked up the deep dive as a problem statement of my own, despite the fact that not a single one of the fixes it produced sat inside my team — they landed with Category, IPC, Demand Planning and Operations. That is the definition of Company-wide Perspective: I owned a diagnosis whose entire remedy was in other people's kitchens, and then built the governance mechanism that held those other teams accountable to it. The upstream/downstream trace is also literal here — I followed a customer-facing speed metric backwards through fulfilment lanes, warehouse tiers, master selection lists and PO execution.

2-Minute Executive Summary:

"Cluster Fulfillment — the share of units fulfilled from a cluster's own warehouse — had been stuck at ~40% for eight weeks with 0% improvement both YoY and WoW. That meant 60% of customer demand was travelling longer fulfilment lanes, which directly degrades delivery speed and speed perception. My formal role was reporting L0 metric health, and our reporting was already slicing CF by zone, state, city and product hierarchy — so we knew where the problem was, but nobody had decomposed why. I took the deep dive on as my own problem statement. I built a CF Loss Waterfall that split the 60% loss by seller type

and warehouse tier into four sized, owner-mapped buckets — MP Forward 30%, Diamond Forward 15%, MP Deep 10%, Diamond Deep 5%. The largest bucket was structural: MP FBF unit share had grown from 35% to 65% while MP Forward CF stayed flat at ~1%, so the entire growth in marketplace volume had bypassed local fulfilment. The deepest investigation was Diamond Forward, where I found four distinct root causes, including two that were invisible until you compared lists against each other: the master selection list covered only 80% of cluster demand, and only 60% of that master was actually being stocked in Forward warehouses — a pure execution gap. I recommended a recency-weighted rolling-12-week priority 10K FSN list to take coverage to 95%, and built a Looker dashboard plus leadership-mailer cadence so each of the four teams owned their bucket weekly. Overall CF improved ~10% within two months, after months of zero movement."

Full STAR Version

SITUATION

This sat within Retail Analytics, partnering the Retail Placement team, whose mandate is placing the right selection in the right warehouse. Two structural facts frame the problem. First, selection comes in two types: **Diamond** (inventory owned and managed by Flipkart) and **MP/Marketplace** (third-party seller inventory, with MP-FBF physically held by Flipkart). Second, the network is two-tier — **Forward** and **Deep** warehouses, where a warehouse plus its mapped pincodes forms a cluster. If selection isn't available in the Forward, the order cascades to the parent Deep, then to other Deeps. Fulfilment is guaranteed, but at the cost of speed and customer experience.

Cluster Fulfillment (CF) = units fulfilled from the cluster's own warehouse ÷ total units fulfilled in that cluster's pincodes. Higher CF means faster delivery promises and better speed perception; low CF means orders travel long lanes.

For the preceding eight weeks CF stood at **~40%, with 0% improvement both YoY and WoW**. 60% of cluster demand was being fulfilled from outside the cluster. My charter responsibility was reporting the health of L0 metrics, and the reporting we had was genuinely useful — it deep-dived CF by zone, state and city in geography and then by product hierarchy, so the Placement team knew *this is the category you need to fix, this is the state you need to fix*. But the charter was new, and structurally fixing CF required understanding the factors behind the stagnation, not just its distribution. Nobody had decomposed it. The number just sat flat.

TASK

I picked this up as a problem statement of my own — no one assigned it. Specifically: decompose the 60% CF loss into root-cause drivers across seller type and warehouse tier so each bucket could be sized and attributed; identify, size and prioritise the improvement opportunity behind each bucket; and translate the findings into concrete, owner-mapped recommendations plus a governance mechanism that would drive sustained improvement rather than a one-time fix. The uncomfortable part of the task definition was that I knew in advance that every fix would be executed by a team other than mine.

ACTION

Step 1 — Built the CF Loss Waterfall. I decomposed the 60% loss by seller type and warehouse type, turning one stagnant number into four prioritised, addressable buckets:

Loss Bucket	Seller / WH Type	CF Loss
MP Forward CF Loss	MP | Forward WH	**30%**
Diamond Forward CF Loss	Diamond | Forward WH	**15%**
MP Deep CF Loss	MP | Deep WH	**10%**
Diamond Deep CF Loss	Diamond | Deep WH	**5%**
Total	**Overall**	**60%**

Step 2 — Sized the largest bucket and found it was structural. MP Forward loss was 30% — half the total. The finding that made it actionable: despite **MP FBF unit share growing from 35% to 65%** over the previous year, **MP Forward CF stayed flat at ~1%** for that entire period. The whole growth in marketplace volume had translated into zero local Forward fulfilment. That reframed the biggest lever from "improve CF" to a specific, ownable problem: solve MP selection placement in Forward warehouses.

Step 3 — Diagnosed MP Deep separately. The 10% MP Deep loss traced to four-zone seller onboarding gaps, which I scoped as a distinct seller-onboarding workstream rather than lumping it with placement.

Step 4 — Ran the deepest investigation on Diamond Forward. This 15% bucket needed the most structured work, and I split it into four sub-buckets each with its own root cause and fix:

Sub-Bucket	Root Cause	Recommendation
Non-Master Selection Loss	Master list covers only 80% of cluster demand	Add priority 10K FSNs (rolling 12-wk demand) → 95% coverage
Master Placement Gap	Only 60% of master selection actually stocked in Forwards	Drive 100% placement adherence
Forecasting / Spike Loss	Demand spikes exceeding forecast quantities	Improve demand forecasting for master FSNs
Operational Loss	IWIT and PO issues	Operational governance and issue resolution

The two findings that mattered most were the ones invisible from any dashboard. The **master coverage gap**: the existing master selection, finalised with Category, IPC and

Demand Planning inputs, covered only 80% of cluster demand — so I recommended a priority list of 10K FSNs built on rolling 12-week recency-weighted demand, expanding coverage to ~95%. And the **placement adherence gap**: comparing the master list against actual Forward inventory showed only **60% of master selection was physically placed in Forwards**. That is a pure execution gap, entirely independent of selection quality, and it only surfaced because I compared two lists nobody had thought to compare. Of the residual loss on master selection that *was* placed, ~20% was spike loss and a significant remaining share came from IWIT and PO operational issues.

Step 5 — Made every finding owner-mapped. Six recommendations, each with a named owner: MP Forward placement (largest bucket), four-zone seller onboarding, master list expansion to 95%, 100% placement adherence, demand forecasting spike capture, and operational governance on IWIT/PO. Every one of these sat with Category, IPC, Demand Planning or Operations — not with Analytics.

Step 6 — Built the governance mechanism, because a deck doesn't change a metric. I built a Looker dashboard tracking CF and each loss bucket weekly — non-master loss, forecasting error loss, placement adherence loss, operational loss — and established a structured governance cadence with the metrics surfaced in leadership mailers. That was the deliberate design choice: by surfacing each team's specific loss bucket in a leadership mailer, accountability became visible without me needing any authority to enforce it.

RESULT

- **Overall CF improved ~10% within two months** of intervention — after 8+ weeks flat at ~40% with 0% YoY.
- **Master FSN coverage expanded from 80% to 95%** of cluster demand via the priority 10K FSN methodology.
- **The 60% placement adherence gap was surfaced** — a pure execution problem invisible until the analysis, enabling corrective action independent of selection quality.
- **Standardised master definition:** a recency-weighted rolling-12-week priority FSN methodology replaced a more subjective selection process.
- **Sustained governance:** the Looker dashboard and leadership-mailer cadence turned a one-time analysis into an operating rhythm across four cross-functional teams.

KEY LEARNING

When a metric is flat for months, the instinct across an org is to treat it as a hard structural ceiling. Often it is a decomposition problem — nobody has broken it down far enough to see where the loss actually sits. Building the waterfall converted an invisible 60% loss into four sized, owner-mapped buckets, and two of the four root causes were only visible by comparing datasets that lived in different teams. The broader learning is that if you diagnose a problem whose fix belongs to other teams, the diagnosis is only half the deliverable; the other half is the governance mechanism that makes each team's slice of it visible every week.

What Interviewers Will Probe

- **How did you get four cross-functional teams to act on recommendations from an analyst with no authority over them?** The governance dashboard did the work. By surfacing each team's specific loss bucket in the leadership mailer, accountability became visible rather than requested. A team that owns a named, sized bucket in a document leadership reads weekly has a clear mandate. I didn't need to push; the cadence pushed.

- **What was the hardest part of the analysis?** Diamond Forward, because the 15% loss had four distinct root causes needing separate analysis each. The placement adherence gap was the most surprising — even after the right master list was selected, 40% of it wasn't being stocked in Forwards. That was invisible until we compared the master list against actual Forward inventory, which nobody had done because the two datasets belonged to different teams.

- **This wasn't your job. Why did you pick it up?** Because our reporting had hit its ceiling. We could tell Placement which state and category to fix, which is a useful input but not a cause. A metric flat for eight weeks with zero YoY doesn't move because someone works the same dashboards harder. And the cost of not picking it up was landing on the customer — 60% of demand travelling long lanes is a speed problem for real orders every day.

- **How did you prioritise which bucket to attack first?** Purely on sized loss with an execution-feasibility check. MP Forward was 30% — half the total — so it got the headline recommendation. But I deliberately didn't stop there, because the Diamond Forward sub-buckets were smaller yet far faster to move: expanding a master list and enforcing placement adherence are executable in weeks, while MP selection placement is a structural programme. Sequencing mattered as much as sizing.

- **How did you avoid the 10% improvement being a one-off?** That's exactly why the Looker dashboard and mailer cadence existed rather than just a deck. The four loss buckets became weekly tracked metrics with owners. The methodology change also helped — the rolling 12-week recency-weighted priority list is self-updating, so master coverage doesn't decay back to 80% the moment attention moves elsewhere.

- **How did you balance faster local delivery against inventory holding cost?** Through the rolling 12-week recency-weighted model. Only high-velocity FSNs with demonstrated regional demand qualified for Forward placement, so we weren't tying up capital in slow-moving stock to chase a speed metric. That's the trade-off the priority-10K methodology exists to manage.

Why This Story Screams Company-wide Perspective

- ✔ Picked up a problem statement nobody assigned, outside the boundary of my reporting mandate
- ✔ Every one of the six fixes sat with another team — Category, IPC, Demand Planning, Operations

- ✔ Traced a customer-facing speed metric upstream through lanes, warehouse tiers, master lists and PO execution
- ✔ Found two root causes that were only visible by joining datasets owned by different departments
- ✔ Built the governance mechanism that made other teams' accountability visible, rather than escalating
- ✔ Never said "our reporting already covers this" — the reporting did its job and was still insufficient

---

Q3. Give an example of a time you had to persuade someone to adopt your idea.

Coupang Leadership Principle: INFLUENCE WITHOUT AUTHORITY *(secondary: Hate Waste)*

The principle in plain English: Coupang's version of persuasion is unusually specific — leaders lead by communicating ideas clearly and drive alignment through data and insights. The word "data" is load-bearing. They are not looking for charisma, storytelling ability, or relationship capital; they are looking for whether you reduced the decision to evidence the other person could check themselves. The secondary principle here, Hate Waste, is Coupang's discipline of doing more with less and saving costs rather than cutting them — which is precisely what an uplift model does to an incentive budget.

How interviewers may frame this question:

- Give me an example of a time you had to persuade someone to adopt your idea.
- Tell me about a time you introduced a new method or technique to your team.
- Describe a time you had to sell an approach that people were unfamiliar with.
- How do you get buy-in for something that has never been tried before?
- Tell me about a time you changed how your organisation did something.
- Give me an example of when you had to overcome scepticism about your analysis.
- Tell me about a time you saved significant cost without cutting capability.

Story: Intelligent Couponing — persuading leadership that targeting the entire NAC base was inefficient

Why this is the best fit:

The persuasion problem here had three layers, which is what makes it a strong story rather than a simple one. First, I had to persuade leadership that a method they had used for years was *structurally* wrong, not just suboptimal. Second, I had to introduce a methodology — uplift modelling — that almost nobody in the room had encountered, which meant the persuasion couldn't rest on "trust the model." Third, the recommendation asked them to *shrink* the targeted base during a year when NC growth was the AOP metric everyone was measured on — so on the face of it I was asking them to accept fewer customers. The way

through all three was to make the logic checkable and then to de-risk it with a confirmatory experiment before asking for commitment.

2-Minute Executive Summary:

"Within the New Customer charter we had an AOP target of roughly 30% YoY NC growth, and until the product-led acquisition roadmap landed 4–5 months out, we were leaning on incentives to hit it. The existing couponing method was blunt: take the entire NAC base, split Test and Control, run a coupon, measure the average treatment effect, report incremental NC. The flaw was that targeting took no account of whether the offer actually changed anyone's behaviour — we were paying 'Sure Things' who would have bought anyway. I had to persuade leadership to move to uplift-based targeting, a methodology most had not used. I made the conceptual case first using the four behavioural segments — Persuadables, Sure Things, Sleeping Dogs, Lost Causes — which made the inefficiency visible without any statistics. I explicitly argued against a simple propensity model, because targeting high probability-to-convert users loads the group with Sure Things. Then I built it: a clean randomised A/B as the foundation, an S-Learner XGBoost uplift model scoring each user twice to get per-user CATE, and validation through decile analysis and a normalised cumulative gain chart at ~80% AUC. The finding that landed was that the top three deciles captured ~80% of incremental customers at ~50% of total cost. Before asking for full commitment I ran a confirmatory A/B — 90/10 on the top three deciles, 10/90 on the bottom seven — which proved it out. Monthly spend went from ₹9 Cr to ₹4 Cr, CAC from ~₹300 to ~₹200, with ~80% of incremental NC preserved. The model ran in production for five to six months."

Full STAR Version

SITUATION

The NC charter chased roughly 30% YoY NC growth. Two levers existed: product-led acquisition, which is the more sustainable path but needs a feature roadmap, A/B validation and 4–5 months to land; and incentive-led campaigns — couponing, free shipping, ₹9 stores — which buy incremental NC with budget. We made a deliberate call to lean on incentives until the product route landed.

The existing couponing strategy was blunt. It took the entire NAC (non-acquiring customer) base, split it Test/Control, ran a TD coupon on Test, measured ATE as Test minus Control conversion, and reported incremental NC at some RPC and CAC. The core inefficiency: targeting did not account for how much the offer actually changed a user's likelihood of converting. Users who would have purchased anyway were also being given the coupon — inefficient spend, lower RPC, avoidably high CAC. Leadership had always evaluated couponing this way, and the working assumption was that more targeting meant more reach meant more NC.

TASK

Persuade leadership that this approach was fundamentally inefficient, introduce uplift modelling as the alternative, and get the org to shift its targeting methodology. The guiding

question I framed for them was deliberately concrete: *what is the ~30% of the target cohort that brings ~80% of incremental NC at ~50% of overall cost?* Find that, and the budget collapses while incremental NC is largely preserved.

ACTION

Step 1 — Made the conceptual case before touching any model. I introduced the four behavioural segments and let the framework do the persuading:

Segment	Behaviour	Uplift
Persuadables (Compliers)	More likely to buy *because* they got the offer — the segment worth targeting	Positive
Sure Things (Always-takers)	Were always going to buy, offer or not — wasted spend	Zero
Sleeping Dogs (Defiers)	Became *less* likely to buy because of the offer	Negative
Lost Causes (Never-takers)	Were never going to buy regardless	Zero

This mattered because it let a non-technical audience see the inefficiency for themselves — no statistics required. I also pre-empted the obvious counter-proposal: I explicitly argued *against* a simple propensity model, because targeting high probability-to-convert users loads the target group with Sure Things and does nothing for targeting efficiency. Killing the intuitive-but-wrong alternative up front is what stopped the discussion looping back to it later.

Step 2 — Built the foundation properly so the method couldn't be attacked on rigour. An uplift model needs a representative sample of all customer types across treated and untreated groups, so we ran a clean randomised A/B: NAC base split Test/Control, Test receiving an x% off up to ₹z coupon. On conclusion we computed the ATE and confirmed statistical significance — a necessary gate, since a near-zero ATE would make uplift modelling pointless. While the experiment ran we captured each AID with its Test/Control flag plus a rich feature set: engagement signals (add-to-cart clicks, banner clicks, bounce rate, average session length, average scroll length, PPVs per visit, PPVs by BU) and dimensions (gender, city tier, zone, age). Post-experiment we appended the converted/not-converted flag.

Step 3 — Built the S-Learner and validated it three ways. A single XGBoost classifier taking all features plus the treated/not-treated indicator as one feature, with conversion as the dependent variable. To estimate uplift I scored the test hold-out twice — once with the treated flag at 1, once at 0 — and took the difference as per-user uplift (CATE/ITE). Validation:

- **Decile analysis:** ten bins ranked by predicted uplift, with true uplift (Test conv − Control conv) computed per decile. A calibrated model shows higher true uplift in the top deciles — it did.
- **Calibration scatter:** predicted average uplift against true uplift per bin sat very close to the 1:1 line.
- **Normalised cumulative gain:** AUC measures efficacy at finding Persuadables — ~50% for a random model, ~80% is the industry benchmark for a strong uplift model, and ours hit that.

Step 4 — Found the persuasion moment. The top three deciles captured **~80% of incremental customers at ~50% of total cost**, driving CAC to ~₹150 versus ~₹350 for targeting everyone, at an RPC of ~4 versus ~2. Feature analysis confirmed the intuition and made it explicable to business stakeholders: top-decile users had recently visited, done more ATCs and banner clicks, more search PPVs, added addresses, and had lower bounce rates. Engagement features, not demographics, were the strongest drivers of being a Persuadable — which is an intuitive story a business leader can hold onto.

Step 5 — De-risked before asking for commitment. This was the step that actually won it. Rather than asking leadership to move production targeting on the strength of a model, I scored the full NAC base, created deciles, and ran a confirmatory A/B — a 90/10 Test/Control split on the top three deciles and 10/90 on the bottom seven — to prove the top-3 ATE was significantly higher and that RPC and CAC were genuinely optimised. The February experiment confirmed the hypothesis, and the model went to production with the Data Science team's help, scoring the NAC base monthly for the next five to six months.

Step 6 — Killed the remaining objection. The lingering fear was that incentive-acquired customers would be low quality. I quantified coupon availers' long-term goodness — spend and M1/M2/M6 repeat — against non-availers, and showed their long-term behaviour was not materially different from organic customers. That removed the last reason to hold back.

RESULT

- **Monthly couponing spend ₹9 Cr → ₹4 Cr** (~55% reduction).
- **CAC ~₹300 → ~₹200** per incremental NC.
- **~80% of incremental NC preserved** versus full-base targeting — roughly 0.2 Mn incremental NC per month against a ~0.3 Mn target, at about half the spend.
- **Ran in production for five to six months**, scoring the NAC base monthly.
- **Uplift-based targeting became the standard methodology** for NC couponing, and the decile framework became the foundation for a second wave of offer-construct experimentation.
- **De-risked incentives as a lever** by establishing quantitatively that availers behave like organic customers on long-term repeat and spend.

KEY LEARNING

Persuasion works best when you reduce the perceived risk of adopting the new idea rather than maximising the perceived upside. The confirmatory A/B was the critical step — it gave

leadership a proof point *before* full commitment, which converted the decision from "trust this methodology" into "act on this result." The second lesson is that killing the intuitive-but-wrong alternative early matters as much as making your own case: if I hadn't explicitly ruled out propensity modelling at the start, every subsequent discussion would have drifted back to it.

What Interviewers Will Probe

- **Why an S-Learner rather than a T-Learner or X-Learner?** The dataset was well balanced across treatment and control, which is the condition where the S-Learner performs well — and a single model gave clean per-user CATE estimates. The practical reason mattered too: one model with a treatment flag is far easier to explain to a business audience than two separately trained models whose difference you then take, and explicability was part of the deliverable here, not a nice-to-have.

- **What if the confirmatory A/B had failed?** It would have meant the model wasn't generalising beyond the training window — which is exactly what I wanted to know *before* production, not after. I'd have gone back to feature engineering and re-examined whether the training experiment's offer construct was representative of what we'd actually deploy. That's the entire reason I ran the confirmatory test before asking for commitment rather than after.

- **How did you handle the objection that shrinking the targeted base would miss the NC AOP target?** Head-on with the arithmetic: we were preserving ~80% of incremental NC at ~50% of spend, which meant the freed budget could fund additional campaigns or other levers. And I showed the shortfall was in *incremental* NC, not total NC — the Sure Things we stopped paying for still converted, because by definition they were always going to.

- **How do you know the top three deciles were genuinely Persuadables and not just an artefact of the model?** Three independent checks pointed the same way: the decile analysis showed *true* uplift (actual Test minus Control conversion, not predicted) rising monotonically in the top deciles; the calibration scatter sat near the 1:1 line; and the confirmatory A/B on a fresh scored population reproduced it. Plus the feature story was behaviourally sensible — recent visitors with high ATC and search activity and low bounce rates are exactly who a coupon should tip over.

- **Why did leadership believe you rather than just asking Data Science to check it?** They did involve Data Science — the production deployment was done with them. I think what made it land was that I never asked for belief. The four-segment framework let them audit the logic themselves, and the confirmatory experiment let them audit the result. Nothing in the chain required trusting me.

- **What would you do differently now?** I'd instrument the Sleeping Dogs question properly. We treated negative uplift as a small tail and focused on isolating Persuadables, but with a monthly-scored production model there was a real opportunity to quantify whether any cohort was actively suppressed by the offer. That's genuine burn saved on top of the ₹5 Cr.

Why This Story Screams Influence without Authority

- ✔ Persuaded leadership to abandon a method they had used for years, purely on evidence
- ✔ Built the argument so the audience could audit it themselves — framework first, model second
- ✔ Pre-emptively killed the intuitive alternative (propensity) rather than letting it resurface
- ✔ De-risked with a confirmatory experiment *before* asking for commitment
- ✔ Removed the final objection with data (availer long-term quality), not reassurance
- ✔ ₹5 Cr/month freed with capability preserved — cost saved, not capability cut (Hate Waste)

Q4. Describe a time when you managed competing priorities.

Coupang Leadership Principle: RUTHLESS PRIORITIZATION *(secondary: Deliver Results with Grit)*

The principle in plain English: Coupang's framing is blunt — to focus on what you must win, you give up what you want to do, and that laser focus takes courage and confidence. The interviewer is listening for two specific things most candidates miss. One: what did you *drop*, and did dropping it cost you something? Two: how did you decide? A story where you heroically delivered everything is not a Ruthless Prioritization answer — it's a working-hours answer, which Coupang explicitly rejects. Where genuine sequencing let you deliver multiple things, the credible version is that you found the dependency chain, not that you worked harder.

How interviewers may frame this question:

- Describe a time when you managed competing priorities.
- Tell me about a time you had more work than you could possibly do. What did you drop?
- How do you decide what not to do?
- Give me an example of a time you said no to something valuable.
- Tell me about a time you had to sequence multiple high-stakes deliverables.
- Describe a period when everything was urgent. How did you handle it?
- What's the hardest prioritisation call you've made?

Story: BBD Analytics — control tower, Intrigue HVA strategy and customer shape projection in parallel

Why this is the best fit:

BBD is the single highest-stakes window in Flipkart's year, and I was carrying three workstreams inside it that each had a hard, immovable deadline and direct CEO-level visibility. What makes this a prioritisation story rather than a workload story is that the three weren't independent — one fed the next — and the real work was finding that dependency chain and then aggressively stripping everything out of the live window that could be pre-built. The prioritisation discipline was temporal: I front-loaded every decision that could possibly be made before the event so that the eight live days contained nothing but judgement calls.

2-Minute Executive Summary:

"During BBD I was simultaneously owning three high-stakes, time-bound workstreams: the baseline-shift decomposition and GMV-guardrailed customer shape projection that fed leadership target-setting; the Intrigue HVA strategy to maximise BBD traffic and customers; and the real-time control tower, the NRT reporting layer the CEO, CFO and SVP sat on for eight continuous days. All three had immovable deadlines and no room to slip. My approach was to find the dependency chain rather than to parallelise: the baseline-shift decomposition had to complete first because it defined which cohorts mattered, that fed the Intrigue cohort × HVA matrix, and both fed the metrics the control tower had to track. Then I applied a hard rule to the live window — anything that could be decided, validated or built before BBD started, was. Target validation with Demand Planning, hourly phasing, mailer dry runs, Omniture optimisation and a 24×7 roster across 20+ teams were all completed weeks ahead, so during the event the team spent its bandwidth on RCA rather than infrastructure. Outcome: the customer shape work drove a ~10% baseline shift in customer DRR, the Intrigue strategy delivered +5% incremental BBD UVs and ~+0.25 Mn customers, and the control tower ran all eight days with zero leadership escalations on data."

Full STAR Version

SITUATION

BBD is Flipkart's biggest event of the year and the event after which the platform sees a baseline shift across most metrics — post-festive customer DRR runs ~1.5x pre-festive DRR. Because of that, the entire leadership gets involved, which means live L0/L1 metric access, real-time achievement reads, and rapid deep dives whenever a number underachieves. I was carrying three things into that window simultaneously:

1. **Baseline shift decomposition and customer shape projection** — understanding what actually drives the post-festive shift and projecting the BBD customer shape that maximises it, with BBD GMV as a hard guardrail. This fed leadership target-setting, so it had to land pre-BBD.
2. **Intrigue HVA strategy** — Intrigue is the ~21-day, three-phase window before BBD that builds frenzy. The Events team had taken a goal to use it as a lever for BBD traffic and customers, and the cohort × HVA matrix had to be finalised before Intrigue started, which is itself three weeks before BBD.

3. **The live BBD control tower** — working with Engineering on the live tracker capturing all FK L0 metrics in NRT, and solely managing control-tower visibility where the Business and CEO sit, across eight continuous days.

TASK

Deliver all three at full quality with no slippage. The pre-BBD work had hard deadlines tied to target-setting and the Intrigue calendar; the control tower had to run flawlessly for eight days with zero escalations. The challenge was not effort — it was that the live window had no slack in it at all, so anything unresolved going in would have to be resolved in the worst possible conditions.

ACTION

Step 1 — Found the dependency chain instead of parallelising. The three workstreams looked independent and weren't. I sequenced deliberately: baseline shift decomposition first, because it defined *which cohorts mattered*. Breaking customer DRR down by TPC cohort across pre-festive (60 days prior) and post-festive (60 days after) windows attributed the ~1.5x shift: ~30% led by NCs acquired during festive, ~20% by the incremental post-festive NC DRR network effect, ~30% by Low and Med TPC customers who transacted during festive, and the remainder by High TPC. That output was the input to everything downstream — it told the Intrigue work which cohorts to build HVAs for, and it told the control tower which cohort cuts the deep-dive tracker needed.

Step 2 — Delivered the customer shape with the guardrail built in, not bolted on. I built a bottom-up organic BBD customer projection by TPC cohort and NC, based on customer incidences in previous BBDs by recency-of-purchase and TPC, computing the YoY BAU multiplier of incidences during a BAU period and applying it to historical BBD incidences. Critically, since GMV is driven by High TPC customers, the shape could not simply over-index on NC/Low/Med TPC to maximise baseline shift at the cost of missing BBD GMV. I projected the shape that maximised baseline shift *while still meeting BBD GMV* — which removed the trade-off leadership would otherwise have had to arbitrate, and therefore removed a review cycle from a calendar that had none to spare.

Step 3 — Compressed the Intrigue work by reusing the cohort frame. Because the cohorts were already established, the Intrigue analysis could go straight to the incremental question: for each cohort, controlled on recency, BUPC and SPC, the conditional probability of converting in BBD given an Intrigue visit versus not. That confirmed Intrigue visits mattered. Then phase importance (maximum incrementality from visiting all three phases, with Phases 2 and 3 driving the disproportionate lift) and construct ranking (Central Intrigue page the biggest conversion-lift driver, then Affordability, then BBD Special), and cohort-specific HVAs — Plus conversion during Intrigue for High TPC, transacting during Intrigue for Low TPC and new users. Each HVA got a target and a day-level tracker with UV phasing by cohort, which drove the daily Intrigue stand-up.

Step 4 — Applied the hard rule to the live window: front-load every decision. This was the actual prioritisation discipline. Everything that could be resolved before BBD started, was:

- Target validation for every FK L0 metric with Demand Planning, triangulated multiple ways — GMV and Units from category targets, and Units ÷ UPC as a cross-check since UPC is slow-moving with a known historical range.
- Dry runs with Engineering on target uploads, correct hourly and daily phasing, and dry-run hourly mailers so every metric section populated correctly with achievement and callouts.
- Omniture (Adobe) preparedness: expected traffic spike shared from historical BBD data for hardware readiness, plus an SOP to prevent report queue clogging — tagging sale-critical reports P0/P1, avoiding "contains" operators in segments, capping views per report at 10.
- A 24×7 roster secured across 20+ teams spanning Analytics, IT, Omniture and FDP for reporting, deep-dive and infra support across the full period.
- Deep-dive tracker structure pre-built: by TPC base, by recency-of-purchase month, by Business Unit, by zone/state/city tier/gender — and then the CVP path to isolate traffic versus conversion, and within conversion, pricing versus serviceability versus selection versus speed promise.

Step 5 — Kept the live window for judgement only. Sitting in the war room with a 3–4 member core team, the work was real-time deep-dive support across all L0 metrics — for example, when a payment gateway failed, quantifying GMV-loss impact within minutes using pre-built cheat sheets and roster support. That was only possible because none of the eight days was spent fixing a target upload or an unoptimised report.

RESULT

- **Customer shape work drove a ~10% baseline shift in customer DRR** post-BBD, and diversity plus event-to-event repeat were adopted as explicit BBD goals for the first time.
- **Intrigue strategy delivered +5% incremental BBD UVs**, Central Intrigue page interaction rising 12% → 14.5% (+0.25 Mn incremental BBD customers), with +0.1 Mn from Low-TPC Intrigue purchase focus and +0.1 Mn from game-zone engagement.
- **Control tower ran all eight days with zero leadership escalations** on NRT data visibility or L0 deep-dive, including from the CEO and CFO.
- **Zero Omniture queue clogging**, and the SOP was adopted as standard pre-event practice for subsequent sale events.

KEY LEARNING

Competing priorities become tractable when you stop treating them as parallel and find the dependency between them. Here the baseline-shift analysis fed the Intrigue target-setting, which fed the control tower's tracking structure — get that sequence right and downstream work is never blocked. The second and more transferable lesson is about *where* you spend scarce capacity: for a fixed live window with no slack, every hour of preparation buys back several hours of the most expensive time you have. The eight days of BBD were the least appropriate moment to be making any decision that could have been made in advance.

What Interviewers Will Probe

- **What did you actually give up? This sounds like you delivered everything.** Two things, deliberately. Exploratory analyses in the Intrigue window got deferred — I stopped at the construct and cohort ranking rather than going further into offer-level optimisation within each construct, which was genuinely valuable and would have taken time I needed for control-tower dry runs. And I deliberately did *not* rebuild the deep-dive tracker architecture, which I knew had rough edges, because a partially rebuilt tracker going into BBD is worse than a known-imperfect one.

- **What happens when something breaks during BBD anyway?** It did — a payment gateway failure. The answer is that the response was pre-built: cheat sheets for GMV-loss quantification, a roster with named owners per metric, and a control tower already populating correctly. We sized the impact within minutes. The prioritisation work is what created the capacity to absorb the unplanned thing.

- **How did you drive the Intrigue strategy with a team you didn't manage?** Through targets and a tracker rather than requests. Each HVA got a specific, named metric target — Central Intrigue page UV share against a historical ~15% baseline, game-zone interaction for new users and Low TPC, incremental Plus base for High TPC — and the day-level tracker drove the daily Intrigue stand-up with actionability on whatever was lagging. Once the metric is on the stand-up board with a target, it doesn't need me to chase it.

- **Why sequence rather than parallelise? Couldn't you have run all three at once with a team?** Partly I did — the roster and the junior analysts owning tracker sections were exactly that. But the analytical dependency was real: the Intrigue cohort work is not meaningful until you know which cohorts drive baseline shift. Running them in parallel would have meant building the Intrigue HVA matrix on assumed cohort priorities and then reworking it, which is slower than sequencing.

- **How did you handle the risk that the pre-BBD work slipped and squeezed the control tower prep?** By making the control tower prep non-negotiable and dated backwards from BBD. Dry runs, target uploads and Omniture optimisation had fixed dates weeks before the event, and the analytical work had to fit around those, not the other way round. The asymmetry was deliberate: a delayed analysis costs a review cycle; a control tower that fails on day one costs the CEO's confidence in every number for eight days.

- **What's the biggest lesson?** That prioritisation in a fixed-deadline environment is mostly about *when*, not *what*. Almost everything on my list was worth doing. The discipline was recognising that the live window was my scarcest resource and defending it aggressively — front-loading every decision that could possibly be front-loaded, and accepting that some good analysis would be deferred to protect it.

Why This Story Screams Ruthless Prioritization

- ✔ Three CEO-visible workstreams with immovable deadlines in the same window
- ✔ Prioritised by dependency chain, not by effort — sequencing was the actual insight
- ✔ Named explicit sacrifices: deferred offer-level Intrigue optimisation, deliberately did not rebuild a tracker mid-flight
- ✔ Identified the scarcest resource (the eight live days) and defended it by front-loading every decision
- ✔ Delivered without confusing hours with results — the live team spent its bandwidth on RCA, not infrastructure
- ✔ Zero escalations across eight days under CEO and CFO scrutiny

---

SECTION 2: TEAMWORK & COLLABORATION

Q5. Tell me about a time you worked with a difficult team member / stakeholder.

Coupang Leadership Principle: DISAGREE AND COMMIT *(secondary: Influence without Authority)*

The principle in plain English: Coupang treats constructive confrontation as a requirement, not a risk — leaders challenge openly when they disagree and explicitly do not compromise for social cohesion. The phrase that matters most is the last one: once a decision is made, proponents and opponents should be indistinguishable. So a "difficult stakeholder" question at Coupang is not testing whether you kept the peace. It is testing whether you were willing to have the hard conversation *and* whether you could work productively with the same person afterwards. Candidates fail this in two opposite ways: by avoiding the conflict, or by winning it so hard the relationship breaks.

How interviewers may frame this question:

- Tell me about a time you worked with a difficult stakeholder or team member.
- Describe a conflict you had at work and how you resolved it.
- Tell me about a time someone actively resisted your work.
- How do you handle a stakeholder who presents data selectively?
- Give me an example of when you had to challenge someone openly.
- Tell me about a time you disagreed with someone but had to keep working with them.
- Describe a situation where a team's incentives were directly opposed to yours.

Story: Pricing Automation — managing Category team resistance across multiple Pilots

Why this is the best fit:

Category teams were the hardest stakeholders on the highest-stakes project I have worked on, and their resistance was not passive. They pushed back on the measurement framework, questioned methodology, and at times presented selective data supporting their preferred narrative. This is a good Disagree and Commit story because the disagreement was genuine and I did not soften the framework to make it go away — but the resolution was not victory. Over multiple Pilots the framework they initially resisted became the review mechanism they relied on to understand their own pricing performance. That is the "proponents and opponents indistinguishable" outcome, arrived at without escalation.

2-Minute Executive Summary:

"On the Pricing Automation Pilot, Category teams were the most difficult stakeholders. They had owned manual pricing for years, and automation meant ceding control — so they weren't passively resistant, they actively challenged the measurement framework, the methodology, and sometimes presented selective reads of the data. My constraint was that the Pilot's success genuinely depended on their participation, so I couldn't win by escalating. Four things worked. First, I never made it personal — every disagreement was framed as 'data versus interpretation,' not 'my view versus yours.' Second, I addressed their actual fear rather than their stated objection: their real worry was being held accountable for GMV targets they no longer controlled, so I built the target sanity-check layer into the framework, which benchmarked category-set targets against comparable historical events and stopped any Pilot running against an inflated target. Third, I gave them the framework before leadership saw it, so they were included rather than policed. Fourth, when interpretations conflicted, I proposed additional analytical checks that both parties agreed to *in advance* — so the data settled it rather than me. Over multiple Pilots they moved from resistance to cautious engagement, and the framework they'd resisted became the mechanism they used to read their own performance."

Full STAR Version

SITUATION

The automated pricing Pilot asked Category teams to hand over a capability they had owned for years. 90–95% of pricing across 10M+ 1P listings was manual, done by categories on intuition and heuristics, and the Pilot's premise was that a product could do it better. Categories were structurally biased against the product — not irrationally, since automation meant ceding pricing control while remaining accountable for the GMV and CM outcomes. Their resistance showed up concretely: pushback on the measurement framework, challenges to methodology, and occasionally selective data presented in support of a preferred narrative. Meanwhile the CEO and CFO were reviewing Pilot results directly, which raised the temperature on every disagreement.

TASK

Work constructively with Category while protecting the integrity of the measurement framework — without escalating the conflict and without losing the collaborative relationship the Pilot depended on. The framework could not be diluted to buy peace,

because a diluted framework would have produced a Pilot verdict nobody could trust. But I also had no authority to impose it.

ACTION

Step 1 — Kept it structurally impersonal. I consistently positioned every disagreement as "data versus interpretation" rather than "my view versus their view," and reframed each pushback session as a question we needed to answer together. This sounds like a soft technique but it had a hard function: it meant every objection had to be converted into something testable, which is a much harder standard to sustain than an assertion.

Step 2 — Solved their real fear rather than their stated objection. Their stated objection was methodological. Their actual fear was accountability without control — being measured on GMV and CM outcomes produced by a product they didn't operate. So I built the **target sanity check** as Layer 1 of the governance framework: before each Pilot, benchmark the GMV and CM targets set by Category against similar-scale historical events with comparable cost inputs, and flag any target that looked inflated. Note who this protects. Category had an incentive to set high bars, because a failed Pilot would discredit automation — but the same layer also protected them from being handed an unachievable number by anyone else. Where targets looked inflated I escalated to BizFin and Category leadership to align on realistic ones. That single layer converted the framework from something done *to* them into something that also worked *for* them.

Step 3 — Gave them the framework first. I shared the measurement framework with Category teams before taking it to leadership. Practically this meant they had a chance to challenge definitions before those definitions were in front of the CEO, which is when a challenge becomes expensive for everyone. It also meant that when they did challenge later, the challenge was on substance rather than on process.

Step 4 — Pre-agreed the tie-breaker. When conflicting interpretations arose, I proposed additional analytical checks that both parties signed up to *in advance* of seeing the result. This is the mechanism I'd reuse anywhere. Once both sides have agreed what evidence would settle a question, the result settles it — and the person running the analysis stops being the counterparty in the argument.

Step 5 — Let the framework prove itself over multiple Pilots. I did not try to win the relationship in one review. The four-layer framework — target sanity check, GMV/CM achievement with portfolio decomposition, ASP adherence, and the price-fluctuation guardrail — produced things Category actually wanted to know. The portfolio decomposition, for instance, showed that 70%+ of the portfolio was being held on CI match, which led to the standardised KVI definition. That work reduced the CI-match portfolio from 70–80% to ~30–40% — a change to *their* operating model that came out of analysis they had initially resisted, and which they could see was defensible because it was built on Search PPVs and price elasticity rather than on assertion.

RESULT

- Over the course of multiple Pilots, Category teams moved from active resistance to cautious engagement.
- **The governance framework they initially resisted became the standard review mechanism** they relied on to understand their own pricing performance.
- No escalation was required to achieve it — the one thing I did escalate was inflated targets, with benchmarked evidence attached, which was as much in their interest as anyone's.
- The framework became the standard evaluation template for every subsequent Pilot across Super Categories.

KEY LEARNING

The most difficult stakeholders are usually the ones with the most at stake. Their stated objection and their actual fear are rarely the same thing, and addressing the stated objection well can still leave the resistance completely intact. Understanding the specific fear — here, accountability without control — and building the answer to it into the structure of the work is the fastest route to collaboration. The second lesson is about tie-breakers: agreeing what evidence would settle a disagreement *before* you generate the evidence removes you personally from the fight.

What Interviewers Will Probe

- **What if the Category team had simply refused to cooperate entirely?** I would have escalated — but not to force cooperation. I'd have surfaced the dependency to leadership: the Pilot's verdict requires Category participation, and here is specifically what cannot be measured without it. Making the dependency visible creates the right incentive without turning leadership into an enforcement mechanism, which would have poisoned the relationship for every Pilot afterwards.

- **Weren't you just giving in by building the target sanity check for them?** The opposite — it was the layer most likely to produce an uncomfortable conversation, because it meant challenging targets Category had set. It happened to protect them from inflated targets as well, which is why it built trust. Diluting the framework would have been giving in; adding a layer that held everyone to benchmarked reality was strengthening it.

- **How do you tell the difference between a stakeholder arguing in bad faith and one with a legitimate objection?** Test whether the objection survives being made specific. Legitimate concerns convert into a checkable question — "does this hold when you exclude event days?" — and the person is generally willing to pre-commit to what the answer would mean. Bad-faith objections tend to move when you close them, and the tell is that the person won't agree in advance on what evidence would settle it.

- **Did you ever have to concede a point to them?** Yes, and it improved the framework. The four-bucket decomposition of ASP deviation by go-live sync — both live, price only, offer only, neither — came out of pushback that it was unfair to

attribute deviation to pricing decisions when the pipeline itself wasn't synchronised. That was a fair challenge, and it turned out only 80% of GMV had price and offer live in sync, which became an Engineering fix. Conceding on a well-argued point is how you earn the right to hold the line elsewhere.

- **How did you avoid this becoming a personal conflict given the CEO-level visibility?** By never using the visibility. It would have been very easy to win an argument by letting a disagreement surface in a CEO review, and it would have cost me the neutral position that made the whole role work. I shared the framework with Category before leadership saw it precisely so that nothing arrived in that room as a surprise.
- **What's the biggest lesson about difficult stakeholders?** That resistance is information. Category's resistance was telling me something true — that the measurement framework, as originally scoped, would hold them accountable for outcomes they couldn't control. Treating that as an obstacle would have produced a framework that was technically correct and organisationally dead.

Why This Story Screams Disagree and Commit

- ✔ Genuine, sustained disagreement with the highest-stakes stakeholder group — challenged openly, not smoothed over
- ✔ Refused to dilute the framework for social cohesion, even under CEO-level pressure
- ✔ Conceded on a well-argued point (go-live sync decomposition) and it improved the outcome
- ✔ Pre-agreed tie-breaker analyses so evidence settled disputes, not seniority
- ✔ Deliberately never used leadership visibility as a weapon
- ✔ Ended with opponents and proponents indistinguishable — they adopted the framework as their own review mechanism

---

Q6. Give an example of a time you had to collaborate across teams to achieve a goal.

Coupang Leadership Principle: COMPANY-WIDE PERSPECTIVE *(secondary: Influence without Authority)*

The principle in plain English: Coupang's Company-wide Perspective is about thinking like an owner of the whole business — understanding upstream and downstream implications, raising problems inside or outside your department, and never saying "that's not my job." In a collaboration question, the interviewer is checking whether you treated other teams' constraints as your problem too. The weak version of a collaboration answer is a list of meetings attended. The strong version shows you understood what each team was optimising for, what it cost them to help you, and how you sequenced the work so nobody became a bottleneck.

How interviewers may frame this question:

- Give me an example of a time you had to collaborate across teams to achieve a goal.
- Tell me about the most complex cross-functional project you've run.
- Describe a time you had to work with a team whose priorities were different from yours.
- How do you manage dependencies on teams you don't control?
- Tell me about a time you had to bring together people who had never worked together.
- Give me an example of when you considered the downstream impact of your work on another team.
- Tell me about a time you unblocked a project that was stuck on another team.

Story: Multi-agent RCA System for Flipkart Minutes — first agentic flow, built across four orgs

Why this is the best fit:

This project required four teams that had never collaborated on anything agentic — LLM/infra, Engineering, Business, and Analytics — and none of them had a playbook, including me. That is what makes it the right collaboration story: nobody could rely on precedent to know what they owed anyone else. Every interface had to be negotiated, and the sequencing mattered enormously because each team's contribution gated the next. It also demonstrates the Company-wide Perspective test directly: securing a dedicated BigQuery processing slot and LLM key approval meant making the case for *why the whole company benefits*, using someone else's budget and someone else's infrastructure capacity.

2-Minute Executive Summary:

"Flipkart Minutes chases OPD — orders per day — and the way the business understood per-store OPD was hourly achievement mailers with dozens of input and output metrics against target. Those mailers generated long back-and-forth between Business and Analytics: analysts would work multiple GSheet trackers to answer 'what's happening' and then write deep-dive SQL to get to 'what do we do,' consuming the bandwidth that should have gone into acting on the actionables. The problem statement was to build a prompt system where a user could ask in natural language — how are orders trending in Bengaluru today — get a structured trend with growth against a benchmark and a Red/Amber/Green read, then follow up with 'why is this city dropping' and have the system run the RCA and return the actionable. This was my first agentic-flow project and there was no internal playbook. It needed four teams who had never worked together on anything like it: LLM/infra for model access and cost approval, Engineering for a dedicated BQ processing slot and Vertex AI deployment, Business for the RCA issue-tree design, and Analytics for schema and semantic-layer definitions. My role was both to build the system and to orchestrate the collaboration so no team became a bottleneck. It's now in daily use — Business and Analytics both check it at the start of the day. Data freshness went from over two hours to under one, RCA turnaround from hours or days to near-real-time, and actionability landed at City × Category × Vertical × Brand."

Full STAR Version

SITUATION

Flipkart Minutes (Quick Commerce) chases OPD as its key metric, with the ambition of becoming the second-largest quick-commerce player within a year. Growth comes from store expansion, owned by Supply Chain, and from improving per-store OPD, which is the track Analytics owns with Business. The existing mechanism for understanding per-store OPD was hourly achievement mailers capturing many input and output metrics against target. Those mailers triggered rounds of discussion between Business and Analytics about what was happening and why; Analytics would work through multiple GSheet trackers to answer the "what," then run deep-dive SQL to reach a real actionable. That consumed significant bandwidth that should have gone into acting on the actionables.

The ask was a prompt system taking natural-language queries, returning a structured trend with growth against a benchmark period and a clear RAG status, letting the user see which metrics by which city were Red, and then answering follow-ups like "why did orders drop in this city" by running the RCA and returning the right actionable — saving the manual effort of working through multiple reports and getting the actionable to the right owner in time to act. This was my first agentic-flow project, in an environment where nobody had an established playbook for the right way to do it.

TASK

Build the system, and orchestrate collaboration across four orgs so that each team's input arrived at the right stage without creating dependencies that stalled the build. Concretely: model access and cost approval from the LLM/infra team; a dedicated BQ processing slot and Vertex AI serverless deployment from Engineering; the RCA issue tree and use-case validation from Business; and schema plus semantic-layer definitions from Analytics.

ACTION

Step 1 — Chose the framework with leadership rather than in isolation. Since we had no existing playbook, the first decision was the framework itself. After multiple brainstorming rounds with our leaders we aligned on an agentic flow — a team of agents interacting to accomplish a task — built on **Google ADK**, chosen because it gives multi-agent orchestration (hierarchical, parallel, sequential), agent-as-a-tool and sub-agent delegation, Python functions and MCP connections as tools, shared context variables for passing data between agents, and serverless deployment on Vertex AI. It fit our Google infrastructure, where data already sat in GSheet and BigQuery. Making this a joint decision rather than a recommendation mattered later, because when we pivoted architecture mid-build nobody treated it as my mistake.

Step 2 — Made the LLM/infra ask in their currency, not mine. An agent needs a brain, instructions and tools; the brain is the expensive part. So the project started by justifying LLM access commercially: estimating input and output tokens per request, requests per minute, and hours of daily use, then costing it by model tier (using ~1,000 tokens ≈ 750 words, with input tokens cheaper than output). Relative per-request cost came out at

Gemini 2.5 Flash 1x, Gemini 2.5 Pro ~4x, Claude ~11x. That cost work is what secured LLM-key approval, and it directly drove the **hybrid-model architecture** — cheaper models where the task is simple, premium models where reasoning is complex:

Agent	Role	Model
Master (Router)	Traffic controller — routes the query to the right specialist	Gemini Flash
Discovery	Helps users who don't know what to ask; returns capability template and sample queries	Gemini Flash
Metric Pull	Agentic sub-flow: NLQ→SQL, runs query, returns structured table with growth vs baseline and RAG coding	Gemini Pro
Orders RCA	Agentic sub-flow: traverses the issue tree, runs attribution, synthesises RCA	Gemini 3 Pro / Sonnet

Metric Pull and RCA were themselves agentic sub-flows — teams of sequential and parallel agents — not single agents.

Step 3 — Co-designed the RCA issue tree with Business rather than for them. The agent had to traverse the same diagnostic logic a human analyst would. I ran multiple sessions with the Minutes business team to encode the full issue tree, and for each node the agent calls the NLQ→SQL agent, compares output to thresholds, and decides whether to traverse to a child or a sibling. For branches needing attribution I built Python tools — a contribution-analysis tool that, out of 80+ cities, surfaces only cities contributing ≥10% to the order drop for further category traversal, and a Shapley-attribution tool for AOV that isolates category-mix change from actual per-category AOV drop. The reason this had to be co-designed is that the tree encodes *their* judgement, and an issue tree the business doesn't recognise produces RCAs they won't act on.

Step 4 — Worked with Engineering on the constraints that only they could solve. V1 ran on the existing GSheet trackers as the backend, deliberately, to get a demo out fast and gather early feedback. Two hard limits then forced the move to BigQuery: data freshness (over two hours of latency via the hourly pipeline) and the 10M-cell file limit, which made a granular City × Cluster × Vertical × Brand view impossible. Minutes needed NRT, granular actionability, so Engineering provisioned a **BQ fact with a dedicated processing slot**, cutting freshness latency to within an hour, plus the Vertex AI serverless deployment. The V1 usage was what justified the ask — I wasn't requesting infrastructure for a hypothesis.

Step 5 — Built the Analytics-side semantic layer that made NLQ→SQL reliable. Rebuilt the Metric agent for BQ as an entity-extraction agent → an NLQ→SQL agent → a BQ execution tool → a growth/RAG Python function returning markdown. The NLQ→SQL agent carries a semantic layer of standard aggregation mappings — Orders = COUNT(DISTINCT order_external_id), GMV = SUM(net_amount) — plus schema discovery, dimension-filter resolution via fuzzy matching, and shared query patterns instead of 200+ hand-written queries. This is where cross-team definition alignment mattered most: if Analytics and Business don't agree what "orders" means, the agent will confidently produce a wrong number.

Step 6 — Hardened through documented failure. SQL generation broke repeatedly and each failure became a prompt rule: underscores not hyphens in aliases (BQ errors otherwise), spaces between keywords to avoid concatenation crashes, and strict rules never to invent CASE WHENs or use dimensions outside the schema. I also learned that instruction prompts need structure rather than plain-English paragraphs — each agent's prompt defines role, tools, positive triggers, negative triggers, output format and context-variable handling, and few-shot examples, using Markdown headers and bolding, which materially improved instruction-following. And since LLMs do next-token prediction and are poor at arithmetic, all computation was provided as Python tools rather than left to the model.

RESULT

- **Data freshness: over 2 hours → under 1 hour** (GSheet → BigQuery fact with dedicated slot).
- **RCA turnaround: hours or days → near-real-time**, automated and on demand.
- **Actionability granularity: City × Category × Vertical × Brand**, down to input-metric level — pricing, OOS, speed, SLA breach, surge visits.
- **In daily production use by both Analytics and Business** in their daily stand-ups. It is the first thing checked at the start of the day, giving a clear read on which cities need orders solved, which stores, and within each store which vertical × brand needs instock, pricing or serviceability fixed.
- A final synthesis agent produces an executive summary plus a detailed issue-tree breakdown, so the output is directly consumable rather than needing an analyst to interpret it.

KEY LEARNING

Cross-team collaboration on genuinely novel problems requires over-communicating on dependencies and being explicit about what each team owns, because when everyone is learning simultaneously, ambiguity about ownership creates more delay than technical difficulty does. The more specific lesson is that each team needs the ask framed in its own currency: infra needed a costed token model, Engineering needed evidence of real usage before allocating a processing slot, Business needed their own diagnostic logic reflected back at them, and Analytics needed definitional precision. The same request phrased four ways is not politics — it's the actual work.

What Interviewers Will Probe

- **Which team was the hardest to align and why?** Business, but not because of reluctance — because encoding the issue tree meant making explicit a diagnostic process that experienced people run intuitively. Getting from "I'd look at instock" to a node with a threshold, a traversal rule and a defined child node took multiple sessions. That was also the highest-value part, because an issue tree that doesn't match how the business reasons produces RCAs nobody trusts.
- **How did you avoid becoming the bottleneck yourself, since you were the only person who understood the whole system?** Imperfectly, at first — that was a real risk in V1. It's a large part of why the follow-on work moved context out of agent prompts into a governed context store of .md files, so business context, metric definitions and issue trees are managed separately from the agents that consume them. That's the difference between a system one person can maintain and one an org can.
- **Why didn't you just wait for BigQuery and build V2 directly?** Because I'd have been asking Engineering for a dedicated processing slot on the strength of a hypothesis. V1 on GSheet proved the capability, generated real usage in daily scrums, and surfaced the exact ceiling — the 10M-cell limit and the two-hour lag against a genuine need for granular NRT. That made the infrastructure request evidence-based rather than speculative, and it got approved faster because of it.
- **How did you handle the LLM cost conversation?** As a business case, not a technical request. Estimated tokens per request, requests per minute, daily usage hours, costed by tier. The output wasn't just approval — it shaped the architecture, because once you can see Claude is ~11x Flash per request, routing simple traffic through Flash and reserving premium models for genuine reasoning becomes obvious rather than a compromise.
- **What would you do differently on the collaboration?** Involve Engineering earlier on the data-layer question. I treated the backend as a V1 implementation detail and it turned out to be the architectural decision that mattered most. A single conversation about latency and cardinality requirements before building V1 wouldn't have changed the decision to ship on GSheet, but it would have made the V2 path a plan rather than a pivot.
- **How do you keep four teams aligned once the system is live?** Through the definitional layer, not through meetings. Once metrics live in a semantic layer defined once and consumed by agents, dashboards and humans alike, most cross-team drift stops happening. The proactive path matters too — a Pub/Sub event firing when a metric breaches threshold and triggering the RCA means the system pushes to stakeholders instead of waiting for someone to ask.

Why This Story Screams Company-wide Perspective

- ✔ Four orgs with no shared precedent — every interface negotiated rather than inherited
- ✔ Framed each ask in the other team's currency: costed token model, evidence of usage, their own diagnostic logic, definitional precision
- ✔ Used someone else's budget and infrastructure capacity by making the whole-company case
- ✔ Traced the problem downstream to who actually acts on the output, not just to the analyst who produces it
- ✔ Solved the definitional-drift problem structurally so the collaboration didn't depend on me
- ✔ Adopted into daily use by both Business and Analytics — the collaboration outlasted the build

---

Q7. Describe a situation where you had to give constructive feedback to a teammate.

Coupang Leadership Principle: DEMAND EXCELLENCE *(secondary: Hire and Develop the Best)*

The principle in plain English: Coupang says leaders have a passion for excellence, do not tolerate mediocrity, and hold themselves and others to a perpetually rising standard. That is a deliberately uncomfortable principle, and the feedback question is where it gets tested. The interviewer wants to know whether you actually hold the bar when holding it is awkward — mid-event, under pressure, with someone who is already struggling. The secondary principle matters too: Hire and Develop the Best is about raising the team's performance level and investing in people's potential, so the best answers hold the standard *and* leave the person more capable, rather than choosing between the two.

How interviewers may frame this question:

- Describe a situation where you had to give constructive feedback to a teammate.
- Tell me about a time you had to tell someone their work wasn't good enough.
- How do you handle underperformance on your team?
- Give me an example of when you held someone to a higher standard.
- Tell me about the most difficult feedback conversation you've had.
- Describe a time you had to correct someone's work under time pressure.
- How do you give feedback without demotivating people?

Story: BBD Control Tower — feedback on RCA quality under live event pressure

Why this is the best fit:

The circumstances are what make this story useful. It was mid-BBD, with leadership consuming our output hourly, and the person was already working under significant

pressure. Every easy option was available — do the RCA myself, let it slide for eight days, wait for a calmer moment — and each of those would have been tolerating mediocrity in the moment where the standard mattered most. Instead the feedback was given immediately, made about the standard rather than the person, and paired with a concrete support mechanism. The quality improved by the next mailer cycle, which is the only proof that matters for feedback given under time pressure.

2-Minute Executive Summary:

"During BBD the control tower team delivered rapid RCAs to leadership whenever a metric deviated from target. One teammate was consistently landing on root causes that were too high-level — 'orders dropped in Bangalore' rather than drilling to the input-metric level of pricing, OOS or speed — and on a couple of occasions had misread the question and produced deep dives that weren't relevant. The consequence was concrete: leadership came back with follow-up questions that consumed time we didn't have and eroded confidence in the team's analysis. I had no room for a long coaching conversation — we were mid-event. So I pulled him aside between mailer cycles and gave one very specific piece of feedback: the RCA template has input metrics — pricing, OOS, speed, SLA breach — and the expectation is to always land on one of those as the root cause, not on the city. The city is the *where*, not the *why*. I showed him a concrete example from an earlier mailer where we'd done it well, which made it about the standard rather than his performance. And I offered to review his next two RCAs before they went out, so he could course-correct in real time without leadership seeing an intermediate version. Quality improved noticeably by the next cycle. He told me later that the specific example was what made it land — abstract feedback like 'be more detailed' wouldn't have worked under that pressure."

Full STAR Version

SITUATION

BBD is eight continuous days of high-stakes NRT reporting consumed directly by the CEO, CFO and SVP. Part of the control tower's job is delivering rapid RCA whenever an L0 metric deviates from target — and the standard is specific: get from the metric movement to the input metric causing it, because that is what someone can act on. Traffic versus conversion first, and if conversion, then pricing, serviceability, selection or speed promise.

One teammate was consistently stopping short of that. His RCAs identified *where* — "orders dropped in Bangalore" — without reaching *why*. Separately, on a couple of questions he misinterpreted what was being asked and produced deep dives that weren't relevant to the question. Both problems had the same downstream cost: leadership asked follow-up questions, which consumed bandwidth in the window where bandwidth was scarcest, and repeated follow-ups start to undermine confidence in the whole team's analysis rather than one person's.

TASK

Give feedback fast enough to change the next mailer cycle. There was no time for a structured coaching conversation and no possibility of deferring it — eight days of BBD

would have meant eight days of the same pattern in front of the CEO. The feedback had to be specific, immediately actionable, and delivered in a way that didn't demoralise someone already under significant pressure. The failure modes were symmetrical: too soft and nothing changes, too blunt and his performance degrades further at the worst possible moment.

ACTION

Step 1 — Chose the moment deliberately. I pulled him aside *between* mailer cycles rather than during one, and in private. That timing was not incidental — feedback delivered while someone is mid-task lands as criticism of the task; feedback in the gap lands as a standard for the next one.

Step 2 — Made the feedback about the artefact, not the person. The exact framing was: "The RCA template has input metrics — pricing, OOS, speed, SLA breach — and the expectation is to always land on one of these as the root cause, not on the city. The city is just the *where*, not the *why*." That sentence does three things at once. It names the standard, it explains the reasoning behind the standard, and it locates the gap in the output rather than in his ability.

Step 3 — Showed what good looks like. I pulled up a concrete example from an earlier mailer where the team had done exactly this well — moved from the metric to the geography to the specific input metric at fault. Showing the example rather than describing it converted an abstract expectation into a pattern he could copy immediately, and it made the conversation about a team standard he was being brought up to rather than about his individual shortfall.

Step 4 — Attached a support mechanism, not just an expectation. I offered to review his next two RCAs before they went out. This mattered more than the feedback itself. Under BBD conditions, the risk of trying to raise your own standard is that your next attempt is visible to the CEO — so the fear of getting it wrong pushes people back toward safe, vague answers. Reviewing the next two removed that risk and gave him real-time correction with no exposure.

Step 5 — Did not take the work off him. The easiest path would have been to write those RCAs myself for the remaining days. I deliberately didn't. That would have solved the eight-day problem and taught him nothing, and it would have signalled to the rest of the team that the bar is enforced by senior people absorbing the work rather than by everyone meeting it.

RESULT

- **RCA quality improved noticeably by the next mailer cycle** — landing on input metrics rather than geographies.
- The follow-up-question load from leadership dropped, protecting the team's bandwidth for the rest of the event.
- **Zero leadership escalations across all eight days** of BBD on data visibility or L0 deep-dive quality.

- He later told me that the specific example was what made the feedback land — abstract feedback like "be more detailed" would not have worked under that kind of pressure.

KEY LEARNING

Constructive feedback under time pressure has to be hyper-specific and solution-oriented, because the goal is not to evaluate performance but to change behaviour before the next cycle. Show the person what good looks like rather than describing it, and offer to support the correction so the cost of attempting the higher standard isn't paid in public. The harder lesson is about not rescuing: absorbing someone's work is the fastest way to fix an output and the surest way to guarantee you'll be absorbing it again next event.

What Interviewers Will Probe

- **Why not just do the RCAs yourself given the stakes?** Because the stakes cut both ways. Yes, eight days of CEO visibility argues for taking no risk — but I'd also have been the single point of failure for every RCA in a window with a payment-gateway failure and a live traffic spike in it. And the next event would have started from the same place. Reviewing his next two gave me the quality control I needed without removing the ownership.

- **How did you know he'd take it well? What if he'd become defensive?** I structured it to make defensiveness unlikely rather than betting on his reaction — private timing, the standard rather than the person, a worked example from the team's own output, and an offer of help attached. If he had become defensive, the fallback was the template itself: the RCA format's input-metric fields are an objective standard, so the conversation could stay on whether the output met the format.

- **Was the feedback fair given he was under extreme pressure?** The pressure is exactly why it was fair. If the standard relaxes during the event, it doesn't exist — BBD *is* the event the standard was written for. What wouldn't have been fair is holding the bar without giving him the means to reach it, which is what the worked example and the two-RCA review were for.

- **What about the second problem — misreading the questions?** That one had a different root cause: under time pressure he was starting the analysis before fully parsing the ask. The two-RCA review caught it structurally, because reviewing before send meant we checked the question-to-answer match, not just the depth. It's a good illustration that two symptoms in the same person can need two different fixes.

- **How do you scale this? You can't personally review everyone's work.** You can't, which is why the durable fix was upstream. The reason I could point at a template with named input metrics is that the deep-dive structure had been pre-built before the event — traffic versus conversion, then pricing/serviceability/selection/speed. A well-specified template is how you hold a standard without standing over people, and after that BBD the checklist he built to prevent recurrence became part of the onboarding for the next cycle.

- **Would you give the same feedback the same way outside a high-pressure event?** The content, yes. The compression, no. Outside the event I'd have spent time on *why* his instinct stopped at geography — probably a reasonable habit from reporting work, where the geography cut is often the deliverable. Understanding the origin of the habit is how you stop it recurring, and that conversation needs time BBD didn't have.

Why This Story Screams Demand Excellence

- ✔ Held the bar at the moment it was hardest to hold — mid-event, under CEO visibility
- ✔ Refused the two easy outs: letting it slide for eight days, or absorbing the work
- ✔ Named a specific, objective standard rather than a vague quality expectation
- ✔ Showed what good looks like instead of describing it
- ✔ Paired the standard with a support mechanism so the person could actually reach it
- ✔ Behaviour changed by the next cycle — the only real test of feedback given under pressure

---

Q8. Tell me about a time when your team failed to meet a goal. What did you learn?

Coupang Leadership Principle: LEARN VORACIOUSLY *(secondary: Think Systematically)*

The principle in plain English: This is the principle where Coupang is most explicit about what it wants to hear: *ego is the enemy — we avoid rationalising mistakes and are vocally self-critical.* A failure question at Coupang is a test of whether you will own a miss cleanly, without hedging it into a story where you were actually right all along. The secondary principle is the other half of a good answer: Think Systematically means you don't just learn the lesson, you change the process so the defect can't recur. A failure story with a genuine own and a structural fix hits both.

How interviewers may frame this question:

- Tell me about a time your team failed to meet a goal. What did you learn?
- Describe your biggest professional mistake.
- Tell me about a time you were wrong.
- Give me an example of a project that didn't go the way you planned.
- What's something you'd do completely differently if you could redo it?
- Tell me about a time your analysis was right but had no impact.
- Describe a time you received difficult feedback and what you did with it.

Story: AOP Simulators — surfacing a ₹2,600 Cr feasibility gap too late in the planning cycle

Why this is the best fit:

This is a genuine miss, and it's the right kind of miss to talk about because the analysis was correct and it still failed. That distinction is uncomfortable and therefore credible: we were not wrong about the number, we were wrong about *when* we produced it, and being right too late is indistinguishable from being wrong in its consequences — a year of chasing an unachievable plan. It also has a clean structural fix rather than a resolution, which is what makes it a Learn Voraciously answer rather than an excuse.

2-Minute Executive Summary:

"During an annual AOP planning cycle I owned the bottom-up projections for customer L0/L1 metrics. As part of that, leadership wanted a top-down ₹4,000 Cr-per-month Grocery GMV target vetted, with Grocery launching in 62 further cities. I built a simulator projecting monthly Grocery GMV at TPC × city-tier level — Grocery GMV as serviceable-cities AAC × penetration × Grocery TPC × Grocery AOV. Even under an extreme scenario, with penetration expanded to all-ever grocery users with add-to-basket intent and Grocery TPC at the 90th percentile of every TPC × city bucket, the simulator reached only about ₹1,400 Cr per month. That's a ~₹2,600 Cr gap: with a similar Grocery selection plan, the target simply wasn't feasible. The analysis was right and it was defensible. The failure was timing — we presented it at a stage where leadership had already partially committed to the top-down number, and by then the cost of resetting it was political as well as analytical. The gap was acknowledged but the target wasn't fully revised, which meant a year of chasing a plan we had already demonstrated was unachievable. What I changed: I rebuilt the AOP work as a plug-and-play simulator that could answer leadership's questions live in the room rather than in a follow-up deck, and I pushed the bottom-up analysis four weeks earlier so it informed target-setting instead of challenging a commitment. The following year the simulator was used inside leadership reviews and the gap was surfaced early enough to shape the conversation."

Full STAR Version

SITUATION

Each year leadership sets top-down targets — typically a stretch — on customer L0/L1 metrics, and the business needs a rigorous bottom-up projection to know how big the gap is and where it sits. That gap then gets divided between Product and Business as owned initiatives, so the entire year's goals hang off the exercise. Get the projection wrong, or produce it at the wrong moment, and the whole year is mis-anchored.

Within that cycle, Grocery was launching in 62 further cities across T1 and T2+ over JAS/OND/JFM, and leadership wanted the top-down ₹4,000 Cr/month Grocery GMV target vetted. I built a simulator projecting monthly Grocery GMV at TPC × city-tier level, modelling **Grocery GMV = Grocery serviceable-cities AAC × Grocery penetration × Grocery TPC × Grocery AOV**. Even under a deliberately extreme scenario — penetration expanded to all-ever grocery users with add-to-basket intent, and Grocery TPC set at the

90th percentile of each TPC × city bucket — the simulator reached only ~₹1,400 Cr/month. A ~₹2,600 Cr gap against the target. With a similar Grocery selection plan, the number was not feasible.

TASK

The analysis itself was the easy part. The task I failed at was getting it in front of the decision at the point where it could still change the decision.

ACTION — and where it went wrong

What we did right. The methodology was sound and stress-tested: a multiplicative decomposition into inputs that could each be independently benchmarked, and a deliberately generous scenario so the conclusion couldn't be dismissed as pessimistic assumptions. When even the most optimistic feasible scenario lands 65% short of target, that's a hard result.

What we got wrong. We presented it after leadership had already partially committed to the top-down number. By that stage the ask had changed shape: it was no longer "help us set the right target," it was "you are telling us the target we have already socialised is wrong." Those are very different conversations and the second one is much harder to win, regardless of the quality of the analysis. The gap was acknowledged; the target was not fully revised downward. The consequence was a year spent chasing a plan we had already demonstrated was unachievable — which has a real cost beyond the number, because teams calibrating against an impossible target learn to discount targets generally.

The honest self-assessment. I could frame this as a stakeholder-management failure external to me, and it would be partly true. But the accurate version is that I treated the analysis as the deliverable and the timing as a scheduling detail. Nobody asked me to start four weeks earlier — and nobody was going to, because the calendar was inherited. Owning the calendar was part of owning the projection, and I hadn't understood that.

What I changed — structurally, not attitudinally.

- **Rebuilt the AOP work as a plug-and-play simulator.** Rather than a deck with a conclusion, a tool taking the input metrics that drive each L0/L1 output metric and running three comparable scenarios — Baseline (organic trend continuing last year's trajectory), Stretch (the middle path), and the Top-Down target — so you can see exactly where the output metrics land and what input values each scenario implies. That means when leadership asks "what MAC do I need for X units," the answer happens in the room rather than in a follow-up cycle. Removing the lag between question and answer was the single biggest fix, because the reason the finding arrived late was partly that every iteration cost a round trip.
- **Moved the bottom-up analysis four weeks earlier**, so findings informed target-setting rather than challenging commitments already made.
- **Built the projections to be individually defensible and cross-validated.** Each metric got a bottom-up build and an independent second method: NC from a monthly simulator with BAU DRR, DRR growth/decay, days in month, Tier-1 event

spike, payday and festive spikes; MAC decomposed into New plus Repeat, with repeat from prior-month buyers via M1R…M6R plus two reactivation bases, and M1R/M2R themselves projected via customer TPC-distribution movement; AAC built two ways — a cohort-additive approach and an incidences-based approach — which landed very close; 12M TPC built as rolling-12M transactions ÷ AAC and cross-checked as Units ÷ U2O using Demand Planning's bottom-up category units and a projected U2O trend from four years of seasonality. Cross-validation existed for one reason: an unachievable-target conversation is only winnable if the projection cannot be attacked on method.

RESULT

- The following year's AOP process was materially better. **The simulator was used inside leadership reviews**, and the bottom-up versus top-down gap was surfaced early enough to influence the target-setting conversation rather than contest it afterwards.
- **Five L0/L1 metrics modelled bottom-up** (NC, MAC, M1R, 12M TPC, 12M AAC), with two independently cross-validated methods for AAC and 12M TPC.
- The identified gap drove the division of goals between Product and Business — effectively shaping the year's customer strategy.
- Manager feedback that year: *"Very detailed AOP made, framework has been tuned and stress tested well through multiple triangulations."*

KEY LEARNING

Analytical correctness is necessary and not sufficient. A finding that arrives after the decision has been made has almost no value, and the fact that it was right makes it worse rather than better — it means the org had the information and still committed to the wrong number. The lesson I actually took is that owning a projection includes owning when it lands, and that if the existing calendar puts your analysis after the decision, changing the calendar is part of the job rather than someone else's problem. The secondary lesson is about lag: every round trip between question and answer is a chance for the decision to move on without you, which is why the plug-and-play simulator mattered more than any single improvement to the projection itself.

What Interviewers Will Probe

- **Wasn't this really a leadership failure rather than yours?** Partly, and I don't think that's the useful read. Leadership committed to a number before the feasibility work landed, which is a real process flaw. But the calendar that produced that sequence was one I had accepted without challenge, and I had the information needed to know the sequence was wrong. Blaming the process I'd gone along with would be exactly the rationalising I want to avoid.

- **How do you know your ₹1,400 Cr projection was right?** Because I built it to fail in the direction of the target. Every input was set generously — penetration extended to all-ever grocery users with basket intent, TPC at the 90th percentile of every

bucket. That's not a central estimate, it's a ceiling. If the honest central case had been ₹1,400 Cr I'd have been much less confident asserting a gap.

- **What if leadership had reset the target and the business had then over-delivered?** That's the risk of a feasibility read and it's why the simulator's three-scenario structure matters. The output isn't "the target is wrong," it's "here is what each scenario requires of the input metrics." If the business believed it could deliver 90th-percentile TPC across every bucket, the simulator makes that an explicit, checkable commitment rather than an implicit one. Being wrong about a ceiling is much less costly than being silent about it.
- **You surfaced this late once. How do you know the fix actually worked?** Because the following year the simulator was in the leadership review itself rather than feeding it, and the gap entered the target-setting conversation instead of following it. The manager feedback on triangulation is the softer evidence; the harder evidence is that the bottom-up versus top-down gap became the basis for splitting goals between Product and Business, which only works if it's available before goals are split.
- **What's the general principle you'd apply now?** Work backwards from the decision date, not forwards from when the request arrives. If the decision is in week six and the analysis takes four weeks, starting in week three is a failure I've already committed to — even though nothing has gone wrong yet.
- **Do you still consider this a failure given how well the next cycle went?** Yes. The next cycle went well because of what this one cost, and a year of the org chasing an unachievable Grocery plan is a real price that the improvement doesn't refund.

Why This Story Screams Learn Voraciously

- ✔ A clean own: the analysis was right and it still failed, and the failure was mine to fix
- ✔ Explicitly refuses the available excuse (leadership committed early) rather than leaning on it
- ✔ Vocally self-critical about the actual root cause — treating timing as a scheduling detail rather than part of ownership
- ✔ Fixed the system, not the attitude: plug-and-play simulator, four-week earlier start, cross-validated methods
- ✔ Names the cost that the later success doesn't erase
- ✔ Converted a one-off miss into a durable capability the org still uses for target-setting

---

SECTION 3: PROBLEM-SOLVING & CRITICAL THINKING

Q9. Describe a situation where you solved a problem in a creative or unconventional way.

Coupang Leadership Principle: HATE WASTE *(secondary: Learn Voraciously)*

The principle in plain English: Coupang's Hate Waste is sharper than generic frugality. Their phrasing has two halves: find ways to do more with less, and *save* costs rather than simply cut them — plus don't skimp on a penny today if it costs a dollar tomorrow. So the principle is specifically about eliminating waste while preserving the outcome. Cutting the coupon budget in half is easy; cutting it in half while keeping 80% of the customers is the thing they're looking for. The creative-solution question is a natural home for this because genuine creativity in analytics usually means finding the waste nobody had noticed was waste.

How interviewers may frame this question:

- Describe a situation where you solved a problem in a creative or unconventional way.
- Tell me about a time you did more with less.
- Give me an example of when you eliminated significant waste.
- Tell me about a time you challenged how a metric was being measured.
- Describe a solution you came up with that nobody else had considered.
- Tell me about a time you found a way to cut cost without cutting output.
- Have you ever reframed a problem and got a completely different answer?

Story: Intelligent Couponing — uplift modelling instead of standard propensity scoring

Why this is the best fit:

The creativity here was not the model — S-Learners are well documented. The creativity was recognising that the entire org, and most of the industry at the time, was answering the wrong question. Propensity modelling asks *who is most likely to buy*. That sounds obviously right for a targeting problem and is precisely backwards for an *incentive* targeting problem, because the highest-probability converters are the people who need the incentive least. Reframing from "who will buy" to "who will buy *because of* the coupon" is a one-sentence change that redirects half a budget. And it maps to Hate Waste exactly: ₹5 Cr a month of spend was going to customers who were always going to convert, which is waste in the purest sense — money spent producing an outcome that would have happened anyway.

2-Minute Executive Summary:

"The standard approach to coupon targeting on my team, and across most of the industry at the time, was propensity modelling — target the users with the highest probability of converting. It seems obviously correct, and for an incentive problem it's fundamentally

flawed, because high-probability converters are 'Sure Things' who buy without any coupon. Every rupee spent on them produces an outcome that was going to happen anyway. The creative move was reframing the question: not 'who is most likely to buy' but 'who is most likely to buy *because of* the coupon.' Those are different questions with different answers. I built it as an S-Learner uplift model — a single XGBoost classifier taking the treated/untreated flag as a feature, scored twice per user to get the difference in conversion probability, which gives per-user uplift. I validated from first principles with decile analysis of predicted versus true uplift, a calibration scatter against the 1:1 line, and a normalised cumulative gain chart hitting the ~80% AUC industry benchmark. The finding: the top three deciles held ~80% of incremental customers at ~50% of cost. Monthly spend fell from ₹9 Cr to ₹4 Cr with ~80% of incremental NC preserved and CAC down from ~₹300 to ~₹200. Then I extended the same decile framework to offer-construct testing, which disproved the standing assumption that deeper discounts always lower RPC."

Full STAR Version

SITUATION

The team's coupon targeting method — and the industry default — was propensity modelling: score users on probability to convert, target the top. The logic is intuitive and it is wrong for incentives. Conceptually, any target base splits into four behavioural segments by response to an offer: Persuadables who buy *because* of it, Sure Things who were always going to buy, Sleeping Dogs who become *less* likely because of it, and Lost Causes who were never going to buy. Propensity targeting loads the target group with Sure Things, because that is exactly who scores highest on probability to convert. The result was structurally inefficient spend, depressed RPC and avoidably high CAC — and none of it was visible in the reporting, because the campaign *did* produce customers. It just produced customers who would have arrived anyway.

TASK

Reframe the problem and build the unconventional solution: a methodology the team had not used, which I had to learn from scratch, to isolate Persuadables and explicitly avoid spending on Sure Things. The guiding question I set was concrete: what is the ~30% of the target cohort that delivers ~80% of incremental NC at ~50% of overall cost?

ACTION

Step 1 — Reframed the question, then killed the intuitive alternative. The four-segment framework was the creative core, because it makes the inefficiency of propensity targeting visible without any statistics — once someone can see "Sure Things" as a named category, the argument is over. I made the design choice explicit: we deliberately did *not* use a propensity model, because targeting high probability-to-convert users loads the group with Sure Things and does nothing for targeting efficiency. Naming the rejected alternative up front is what stopped the discussion cycling back to it.

Step 2 — Built the experimental foundation, because uplift can't be inferred from observational data. An uplift model needs a representative sample of all customer types

across treated and untreated groups. We ran a clean randomised A/B: NAC base split Test and Control, Test receiving an x% off up to ₹z coupon. On conclusion we computed ATE — Test conversion minus Control conversion — and confirmed statistical significance, which was a necessary gate, because a near-zero ATE would make the entire exercise pointless. While it ran, we captured each AID with its Test/Control flag plus engagement features (add-to-cart clicks, banner clicks, bounce rate, average session length, average scroll length, PPVs per visit, PPVs by BU) and dimensions (gender, city tier, zone, age), then appended the converted flag afterwards.

Step 3 — Chose the S-Learner and got per-user uplift out of it. A single XGBoost classifier over all features plus the treated indicator, with conversion as the dependent variable. The trick is in the scoring: score the test hold-out twice, once with the treated flag at 1 and once at 0, and take the difference. P(conversion | treated) − P(conversion | not treated) is the per-user uplift, the CATE/ITE. One model, individual-level estimates, and — importantly for the adoption problem — an architecture explicable to a business audience in two sentences.

Step 4 — Built validation from first principles rather than trusting a fit statistic. Uplift models can't be validated like classifiers, because the counterfactual is never observed for any individual. So three checks at the group level:

- **Decile analysis:** ten bins ranked on predicted uplift, with *true* uplift (actual Test minus Control conversion) computed per bin. A calibrated model shows true uplift rising in the top deciles — it did.
- **Calibration scatter:** predicted average uplift plotted against true uplift per bin, sitting very close to the 1:1 line.
- **Normalised cumulative gain:** percentage of incremental customers gained against percentage of users targeted. AUC measures efficacy at finding Persuadables — ~50% for random, ~80% is the industry benchmark for a strong uplift model, which ours reached.

Step 5 — Read the feature story, which made the finding intuitive as well as true. Top-three-decile users had recently visited, done more ATCs and banner clicks, more search PPVs, added addresses, and had lower bounce rates. Engagement features, not demographics, were the strongest predictors of being a Persuadable. That's a behaviourally sensible story — the person already circling the purchase is the one a coupon tips over — and having it made the result far easier to defend than a black-box output.

Step 6 — Stress-tested and productionised. Scored the full NAC base, created deciles, and ran a confirmatory A/B with a 90/10 Test/Control split on the top three deciles and 10/90 on the bottom seven, to prove the top-3 ATE was significantly higher and RPC/CAC genuinely optimised. The February experiment confirmed it, and the model went to production with Data Science, scoring the NAC base monthly for five to six months.

Step 7 — Extended the framework beyond its original purpose. Once the right target group was identified, the next question was the right *offer* for that group. Using the score deciles I designed multiple A/B experiments on the top deciles — T1/T2/T3 and C1/C2/C3

cells — testing 30% off up to ₹50 versus 10% off up to ₹100 versus a ₹9 store with limited selection. Decomposing RPC as incremental revenue over incremental burn and CAC as incremental cost over incremental burn showed the relationship was non-linear: the 30%-off construct had *higher* RPC and *lower* CAC, because the offer perception drove a spike in incremental NC large enough to more than compensate for the higher burn. That disproved a standing leadership assumption that deeper discounts always reduce RPC.

RESULT

- **Monthly spend ₹9 Cr → ₹4 Cr** (~55% reduction), **CAC ~₹300 → ₹200, 80% of incremental NC preserved** — roughly 0.2 Mn incremental NC monthly against a ~0.3 Mn target at about half the spend.
- **Ran in production for five to six months**, scoring the NAC base monthly.
- **Uplift-based targeting became the standard methodology** for NC couponing.
- **The decile framework became reusable infrastructure** — it was the foundation for the second wave of offer-construct experimentation.
- **De-risked incentives as a lever**: quantified that coupon availers' long-term repeat and spend were not materially different from organic customers, removing a key objection to scaling incentive-led acquisition.

KEY LEARNING

The most powerful creative solutions usually come from questioning what you are measuring rather than optimising how you measure it. "Are we answering the right question?" produced a ₹5 Cr-a-month answer; "how do we score propensity better?" would have produced a marginally better version of the wrong thing. The second lesson is that the reframe is the cheap part — the expensive part is building validation rigorous enough that an unfamiliar methodology survives scrutiny, and that is where most of the actual work went.

What Interviewers Will Probe

- **Why uplift modelling rather than propensity — explain it as if to a non-technical stakeholder.** Propensity finds the people most likely to buy. Uplift finds the people whose decision actually *changes* because of the coupon. They sound similar and they're almost opposites: the person most likely to buy is usually the person who needs the coupon least. We were paying for customers who were already on their way to checkout.

- **How did you validate the model was measuring incrementality and not just conversion?** Because the validation used *true* uplift, not predicted. In the decile analysis, each bin's true uplift is actual Test conversion minus actual Control conversion — an observed quantity from the randomised experiment, not a model output. If the model were secretly ranking on propensity, the top deciles would show high conversion but flat *difference* between Test and Control. They didn't.

- **What was the risk of the reframe being wrong?** If ATE had been near zero, the whole approach collapses — you can't isolate Persuadables in a population where

the offer changes nobody's behaviour. That's why confirming ATE significance was an explicit gate before modelling rather than a step in the write-up.

- **Isn't a 20% loss of incremental NC a real cost?** Yes, and it was the crux of the decision. The trade was ~20% of incremental NC for ~50% of the budget, which frees ₹5 Cr a month for other levers. Whether that's right depends on the marginal return of the freed budget — which is precisely why the NC budget allocation framework became the natural follow-on project.
- **How did you handle the Sleeping Dogs — the negatively-affected segment?** Honestly, not as well as I should have. We treated negative uplift as a small tail and concentrated on isolating Persuadables. With a monthly-scored production model there was a real opportunity to quantify whether any cohort was actively suppressed by the offer, and that's genuine burn saved on top of the ₹5 Cr. It's the main thing I'd do differently.
- **What made this creative rather than just correct?** The model was standard. What wasn't standard was noticing that a metric everybody trusted — incremental NC from a campaign — was structurally incapable of revealing the inefficiency, because the campaign genuinely produced the customers it reported. The waste was invisible inside the measurement framework being used to evaluate it.

Why This Story Screams Hate Waste

- ✔ Found waste that was invisible inside the existing measurement framework — the campaign "worked" while half the spend produced nothing
- ✔ Saved cost rather than cutting it: ₹5 Cr/month removed with ~80% of the outcome preserved
- ✔ Did more with less at every level — one model instead of two, deciles reused for offer testing
- ✔ Didn't skimp where it mattered: ran a full randomised experiment and a confirmatory A/B before touching production spend
- ✔ Left reusable infrastructure behind, so the saving compounded into offer-construct optimisation
- ✔ Disproved a standing assumption (deeper discount = lower RPC) that had been quietly mis-allocating budget

---

Q10. Tell me about a time when you had to make a decision with incomplete information.

Coupang Leadership Principle: MOVE WITH URGENCY *(secondary: Learn Voraciously)*

The principle in plain English: Coupang's language here is unusually strong — urgency is a sense of crisis, inaction is treated as a threat to survival, and explicitly: *we "learn by doing" and do not delay decisions to seek a "perfect" solution.* So the incomplete-information question isn't testing your risk analysis. It's testing whether you moved. Candidates who

describe gathering more data until they were confident are answering a different question than the one being asked. The strong answer makes a defensible decision on partial information, states what would have changed their mind, and then shows the learning loop that closed the gap.

How interviewers may frame this question:

- Tell me about a time you had to make a decision with incomplete information.
- Describe a decision you made without having all the data you wanted.
- How do you act when there's no established playbook?
- Tell me about a time you had to move fast with real uncertainty.
- Give me an example of when waiting for more information would have been the wrong call.
- Tell me about a decision you made that you knew might be wrong.
- How do you balance speed against being sure?

Story: Multi-agent RCA System — architectural decisions with no internal playbook

Why this is the best fit:

Three consequential architectural decisions had to be made before there was any evidence available to make them with, because the evidence only exists once you've built something. That is the purest form of the incomplete-information problem: not missing data, but data that cannot exist yet. And the response pattern maps exactly onto Coupang's "learn by doing" — decide on a defensible first-principles framework, make the assumption explicit so it's testable, build, and let the failure modes tell you what to change. The V1-to-V2 pivot is the proof the loop closed rather than the decision simply being lucky.

2-Minute Executive Summary:

"When I started building the multi-agent RCA system for Flipkart Minutes, there was no internal playbook for agentic flows at Flipkart. Nobody on the team had done it, Google ADK had limited documentation, and the right architecture for our use case — multi-agent, hybrid-model, NLQ→SQL, issue-tree traversal — could only be discovered by building it. Three decisions had to be made with very incomplete information: which model tier for each agent, GSheet or BigQuery as the backend, and how to structure the RCA issue tree for reliable traversal. On models, I built a first-principles cost-versus-complexity framework: estimated tokens per request, requests per minute and daily usage hours, costed by tier — Flash at 1x, Pro at ~4x, Claude at ~11x — and routed simple work to cheap models and genuine reasoning to premium ones. On the backend I chose GSheet for V1 with the limitations stated up front, because we already had GSheet trackers holding deep-dive data and I wanted a working capability in front of users fast. On the issue tree I accepted the first version would be wrong and co-designed it with the business, then observed where the agent made bad traversal calls and refined. V1 hit its ceiling exactly where predicted — the 10M-cell limit and two-hour latency against a real need for granular NRT — and that gave me the evidence to secure a dedicated BQ slot. Data freshness went from over two hours to

under one. The business got value while the architecture improved, rather than waiting for a perfect V1."

Full STAR Version

SITUATION

Flipkart Minutes needed a natural-language system for metric pulls and order-drop RCA, to replace a workflow where analysts worked through multiple GSheet trackers and then wrote deep-dive SQL for every question — consuming the bandwidth that should have gone into acting on the answers. This was the first agentic-flow project I had built, in an environment where nobody had an established playbook for the right way to do it. Google ADK's documentation was limited. There was no internal precedent for cost structures, prompt architecture, or how reliably an agent could traverse a diagnostic tree.

TASK

Three architectural decisions had to be made before any evidence existed:

1. **Which LLM tier for each agent** — with a ~10x per-token cost spread across tiers, getting this wrong meant either an unaffordable system or an unreliable one.
2. **GSheet or BigQuery as the backend data layer** — a decision with weeks of engineering consequence either way.
3. **How to structure the RCA issue tree** so an agent could traverse it reliably — where "reliably" had no benchmark, because nobody had tried.

ACTION

Step 1 — Replaced missing evidence with a defensible framework. For model selection I couldn't know empirically which agent needed which tier, so I built the decision on cost-versus-complexity from first principles: estimate input and output tokens per request, requests per minute, and hours of daily use, then cost by tier (~1,000 tokens ≈ 750 words, input tokens cheaper than output). That produced relative per-request costs — Gemini 2.5 Flash 1x, Gemini 2.5 Pro ~4x, Claude ~11x — and a hybrid architecture: Flash for the Master router and Discovery agent where the task is routing and templating, Pro for Metric Pull where NLQ→SQL needs real language understanding, and Gemini 3 Pro / Sonnet for Orders RCA where the reasoning is genuinely hard. This was a judgement call with limited real-world data, but the *framework* was defensible, which mattered twice: it secured LLM-key approval, and it gave me something to revise against rather than a guess to defend.

Step 2 — Chose the fast path deliberately and stated the limits out loud. For the backend, I started with GSheet. The reasoning was concrete rather than lazy: the team already used GSheet trackers to answer some deep dives, so a level of deep-dive data was readily available there, and I wanted V1 in front of users quickly to showcase the capability and gather early feedback. I explicitly documented the two known limitations from day one — **2+ hour data freshness latency** via the hourly pipeline, and the **10M-cell file limit** — and told the business team that V1 was a capability proof, not the final architecture, with V2 committed as a fast follow. Stating the ceiling in advance is what turned hitting it into a validated requirement rather than a failure.

Step 3 — Accepted that the issue tree would be wrong first. Rather than trying to design a complete tree up front, I co-designed it iteratively with the Minutes business team: build, test against real queries, observe where the agent made wrong traversal decisions, refine. For each node the agent calls the NLQ→SQL agent, compares output to thresholds, and decides whether to traverse to a child or sibling — so the failure mode is specific and observable. Where branches needed attribution I supplied Python tools rather than trusting the model: a contribution-analysis tool surfacing only the cities contributing ≥10% of an order drop out of 80+, and a Shapley-attribution tool for AOV separating category-mix change from actual per-category AOV drop.

Step 4 — Let the failures write the rules. SQL generation broke repeatedly and each break became a documented prompt rule: underscores rather than hyphens in aliases, spaces between keywords to avoid concatenation crashes, strict prohibitions on inventing CASE WHENs or using dimensions outside the schema. Similarly on prompts: plain-English paragraphs produced unpredictable agent behaviour, so each prompt was restructured with explicit role, tools, positive triggers, negative triggers, output format and context-variable handling, plus few-shot examples, using Markdown structure — which materially improved instruction-following.

Step 5 — Made the V2 decision on evidence rather than on principle. V1 hit its ceiling exactly where I'd said it would: Minutes needed near-real-time actionability at City × Cluster × Vertical × Brand, which the 10M-cell limit made impossible, and the two-hour latency was too slow for intraday action. That real usage evidence is what justified a **dedicated BQ processing slot** from Engineering, cutting freshness to within an hour. I then rebuilt the Metric agent for BQ — entity extraction → NLQ→SQL with a semantic layer of standard aggregation mappings, schema discovery and fuzzy dimension-filter resolution → BQ execution → growth/RAG Python function returning markdown.

RESULT

- **Data freshness: over 2 hours → under 1 hour.**
- **RCA turnaround: hours or days → near-real-time**, at City × Category × Vertical × Brand granularity down to input metrics.
- **In daily production use** by Analytics and Business in their stand-ups — the first thing checked at the start of the day.
- **The incremental V1→V2 path meant the business got value while the architecture improved**, rather than waiting for a perfect V1 — and V2 was built faster because V1 had already validated the prompt logic and surfaced the real requirements.
- The hybrid-model cost architecture held up, and the same cost-framework logic is what secured LLM access in the first place.

KEY LEARNING

In novel problem spaces the best decision framework is: build something small and correct, measure the failure modes, iterate. The most important discipline is making the current assumptions explicit so they can be tested quickly — I said in advance that GSheet would

fail on granularity and freshness, which meant hitting that wall was a validated requirement rather than a mistake to defend. The corollary is that a decision made on a defensible framework with no data is far more revisable than a decision made on intuition, because you can point at which input changed.

What Interviewers Will Probe

- **How did you know GSheet was the right V1 call and not just the easy one?** Because the alternative wasn't "a better V1," it was "no V1 for weeks." The deep-dive data already sat in GSheet trackers, so V1 could be genuinely useful immediately. And the test I applied was narrow: does it solve the core use case well enough to be used in daily stand-ups? It could pull metric trends and run directional RCA in natural language. Everything beyond that was optimisation.
- **What was the risk of shipping V1 with known limitations?** The real risk was anchoring — that users would internalise the 2-hour lag as acceptable and make V2 a harder sell for engineering resources. I mitigated it by being completely transparent about the limitations up front and setting the V2 transition plan before V1 launched, so the ceiling was a shared expectation rather than a discovery.
- **What if the hybrid-model decision had been wrong?** It would have shown up as either cost overrun or quality failure on specific agents, both observable within days. That's the point of building the decision on a cost-versus-complexity framework: if Metric Pull had failed on Pro, the fix is promoting one agent a tier, not rearchitecting. Framework-based decisions fail locally; intuition-based decisions fail everywhere at once.
- **Weren't you gambling with the business's time by building on an architecture you knew you'd replace?** I'd argue the opposite — the alternative gamble was larger. Building V2 first would have meant requesting a dedicated processing slot and weeks of engineering time for a capability nobody had used, and if the use case had turned out weaker than expected, that's a much bigger write-off than a GSheet prototype.
- **How do you decide when you have enough information to move?** I ask what the cost of being wrong is versus the cost of the delay, and whether the wrong decision is recoverable. Model tiering is trivially reversible. Backend choice is expensive but the V1 version was scoped so the sunk cost was the prompt logic — which transferred to V2 anyway. The issue tree was the one I couldn't get right in advance at all, so I designed for iteration rather than accuracy.
- **What's the biggest lesson?** That in a genuinely novel space, the question isn't "what's the right decision" but "what's the fastest defensible decision that generates the evidence for the next one." Waiting for certainty in a space where no precedent exists is just a slower way of guessing.

Why This Story Screams Move with Urgency

- ✔ Three consequential decisions made before any evidence could exist — and made, not deferred
- ✔ Explicitly refused to wait for a perfect architecture; shipped a constrained V1 into daily use
- ✔ Learned by doing: every SQL failure and prompt failure became a documented rule
- ✔ Stated the assumptions and limitations up front so they were testable rather than defensible
- ✔ V1 usage became the evidence that unlocked V2's infrastructure — speed compounded rather than costing
- ✔ Treated the alternative — weeks of business bandwidth wasted on manual trackers — as the real risk

---

Q11. Give an example of a complex problem you solved and how you approached it.

Coupang Leadership Principle: DIVE DEEP *(secondary: Think Systematically)*

The principle in plain English: Coupang's Dive Deep is about operational excellence through hands-on detail — digging down to the smallest details to gain full understanding, which is what equips you to empower the right people and deliver results. They add a pointed line: *no task is beneath us.* So the interviewer is looking for evidence you went several layers below where most people stop, and specifically that you did unglamorous work — building instrumentation, reconciling timestamps, chasing definitions — rather than staying at the level of analysis and inference.

How interviewers may frame this question:

- Give me an example of a complex problem you solved and how you approached it.
- Walk me through the most technically difficult analysis you've done.
- Tell me about a time you had to get to the bottom of something nobody could explain.
- Describe a problem with multiple suspected root causes. How did you attribute it?
- Tell me about a time you had to go deeper than the dashboards.
- Give me an example of when the obvious explanation turned out to be wrong.
- What's the most detailed piece of work you've personally done?

Story: Pricing Automation — ASP deviation diagnosis and the BCD projection feedback loop

Why this is the best fit:

This is the Dive Deep story in the arsenal where the depth was physical rather than analytical. The problem — ~70% of GMV deviating more than ±2% from model-recommended ASP — was multi-causal, and multiple teams were pointing at each

other's root cause. The only way to attribute it was to build a data layer that traced a single reaction ID from price recommendation through offer mapping to realised ASP, which meant defining metrics at reaction ID × FSN level and reconciling go-live timestamps across two independently instrumented products. That is unglamorous, detail-obsessed work that nobody wants to own, and it was the prerequisite for every subsequent finding. And the eventual root cause sat four layers down: not "the model is wrong," but that bank-tier-level offer adoption was being applied to bank-tier × payment-instrument combinations.

2-Minute Executive Summary:

"The single biggest driver of GMV and CM misses in the automated pricing Pilot was ASP deviation — the gap between the Optimizer's recommended price and the ASP actually realised on the platform. At the start, ~70% of GMV was deviating more than ±2%, which made the Pilot's GMV and CM predictions close to meaningless, because the model's recommendations weren't reaching customers. It was multi-causal and teams were pointing at different root causes. My first move was to build the instrumentation, because attribution was impossible without it: I mapped the full journey across both products — reaction ID generation → FSP/ASP recommendation → BCD projection → offer go-live → actual BCD → realised ASP — and defined metrics at reaction ID × FSN level including every timestamp. That let me decompose deviation two ways. First by go-live synchronisation into four buckets — both price and offer live, price live but offer not, offer live but price not, neither — which showed only 80% of GMV had both live in sync, and those cases had materially lower deviation. That became an Engineering pipeline fix. Second, into BCD projection error versus left-over gap, the structurally unresolvable gap between FSP and ASP even after full offer mapping, which exceeded 10% for a significant share of GMV. Then EDA across comparable historical events found the actual error source: bank tier × payment instrument adoption differed significantly, and OI was applying bank-tier-level adoption to bank-tier × instrument combinations. I recommended a finer PBO granularity with a fallback chain, plus a live mid-event feedback loop. ASP deviation went from ~70% to ~30% of GMV."

Full STAR Version

SITUATION

The Pilot ran two products in tandem. The **Optimizer** used historical data plus event cost inputs (NLC, BO, SMNM, logistics, CX costs) to recommend the FSN × ASP × FSP combination maximising GMV within CM constraints, and predicted unit, GMV and CM outcomes per FSN. **Offer Intelligence (OI)** bridged the FSP → ASP gap the Optimizer couldn't solve, using BCD (Blended Conditional Discount) projection from mandatory and conditional offers to recommend which offers to map or unmap against each listing to hit the Optimizer's target ASP.

The problem: **~70% of GMV was experiencing ASP deviation beyond ±2%** of model-recommended ASP. When deviation is that high, the Pilot's GMV and CM predictions are meaningless, because the model's recommendations aren't being executed on the platform. And at project start there was no scientific model for offer adoption projection at all — OI relied on coarse historical event heuristics, with no feedback loop, no shared

instrumentation between the two products, and no governance framework to measure system-level success. Meanwhile the CEO and CFO were reviewing results directly and competing teams were attributing the deviation to different causes.

TASK

Diagnose the exact root causes across the full pricing stack — Optimizer → OI → offer go-live → realised ASP — quantify each cause's contribution, and design a mechanism that would reduce deviation systematically rather than case by case.

ACTION

Step 1 — Built the instrumentation first, knowing it would delay the analysis. The initial instrumentation had been built independently by each product's engineering team. Each product worked end-to-end in isolation, but there was no shared data layer capturing the system-level story. So before any analysis I mapped the full end-to-end journey of both products — reaction ID generation → FSP/ASP recommendation → BCD projection → offer go-live → actual BCD → realised ASP — and defined the exact metrics and granularity required to stitch it together: at **reaction ID × FSN level**, capturing FSP, ASP, predicted DRRs, reaction go-live time, offer go-live time, projected mandatory BCD, non-mandatory BCD, actual BCD, and actual GMV and CM. This is the least interesting work in the project and it was the precondition for everything else — you cannot attribute a deviation you cannot trace. Once live, I partnered with data analysts to build the pipeline and Pilot dashboard, and got it embedded into the standard Pilot review cadence.

Step 2 — Decomposed by go-live synchronisation, which the timestamps made possible. With reaction start/end times and offer go-live timestamps captured, I split ASP deviation into four buckets: (a) both price and offer live, (b) price live but offer not, (c) offer live but price not, (d) neither live. The finding: **only 80% of GMV had both price and offer live in sync**, and those cases showed significantly lower deviation. That isolated a chunk of the problem as a pipeline synchronisation defect rather than a modelling error, and it went straight to Engineering as a fix. It also mattered politically — it separated the product teams' modelling error from an infrastructure problem, which is what made the rest of the diagnosis credible to them.

Step 3 — Separated projection error from structurally unfixable gap. I decomposed deviation into BCD projection error versus **left-over gap** — the unresolvable gap between FSP and ASP even after full offer mapping. Left-over gap exceeded 10% for a significant portion of GMV, which meant the system was structurally not set up to hit target ASP in those cases at all. This distinction is the reason the fix worked: without it, effort would have gone into improving a projection for listings where no projection could have succeeded.

Step 4 — Found the actual root cause four layers down. Extensive EDA on historical similar-scale events — May BSD, Jan RD, Aug ID Sale — computing offer adoption at multiple combinations of product attributes × offer attributes and measuring projected versus actual BCD deviation at each granularity. The key finding: **bank tier × payment instrument (debit/credit/UPI) adoption differed significantly**, and applying bank-tier-level adoption to bank-tier × instrument combinations was a major source of

error. That is a granularity bug, not a model quality problem, and no amount of tuning at the original granularity would have fixed it.

Step 5 — Recommended a specific granularity with a fallback chain. For static offer adoption (PBO): **Event × Vertical × Price Tier × Bank Tier × Payment Instrument**, with a defined fallback chain for sparse cells — Event × Vertical × Clubbed Price Tier × Bank Tier × Payment Instrument, then Event × Vertical × Price Tier × Bank Tier, then Event × Vertical × Clubbed Price Tier × Bank Tier. The fallback chain is the detail that makes it deployable: a granularity recommendation without one fails silently on thin data.

Step 6 — Designed a live feedback loop for mid-event correction. Static adoption for the first 8–10 hours of an event, because live cumulative adoption is noisy early on; after that maturity threshold, switch to live adoption, with a continuous check on deviation between actual and cumulative live adoption and an update trigger whenever deviation crosses 2%. This converted a once-per-event projection into a self-correcting system.

Step 7 — Attacked the structural constraint on scope in parallel. The portfolio decomposition had shown 70%+ of the portfolio was being held on CI match by categories, which both limited the Optimizer's scope and contributed unnecessary deviation. I led the analytical work to standardise the KVI definition using Search PPVs as an organic demand proxy and price elasticity from the Optimizer model, built a 3×3 grid across high/low search PPV and price sensitivity, and aligned the org on a rule: very high sensitivity items and very high demand items go on CI match regardless of the other dimension. KVIs proved to be only 3–4% of FSN count but 30–40% of SC GMV — which rationalised CI match from 70–80% down to ~30–40%.

RESULT

- **ASP deviation reduced from ~70% to ~30%** of GMV beyond ±2% — directly improving the GMV and CM predictability of the Pilot.
- **CI-match portfolio reduced from 70–80% to ~30–40%** via a defensible KVI standard, meaningfully expanding the Optimizer's effective scope.
- **Engineering fix on go-live synchronisation**, after the four-bucket decomposition isolated it as a pipeline defect.
- **The feedback loop was incorporated into the Pilot review cadence**, and ASP deviation became a tracked governance metric across all subsequent Pilots.
- The instrumentation became **the first cross-product data layer** connecting Optimizer and OI, and the foundation for every future Pilot review.

KEY LEARNING

Complex problems with multiple suspected root causes require instrumentation before analysis. You cannot attribute a deviation correctly without the data to trace the full journey, and building that layer first — even when it visibly delays the analysis — always pays off, because every week of instrumentation removes multiple rounds of argument later. The second lesson is about separating the fixable from the unfixable early: identifying that left-over gap exceeded 10% for a meaningful share of GMV stopped us from optimising a projection for listings where no projection could ever have worked.

What Interviewers Will Probe

- **Why did you start with instrumentation rather than analysis, and how did you justify the delay?** Because in a two-product system with competing teams, an analysis without traceability is just another opinion in the room. I argued it as risk: for a first-of-its-kind Pilot under CEO scrutiny, an unmeasurable product is indistinguishable from a failed one. It cost weeks and it converted attribution from a debate into a query.

- **How did you convince two engineering orgs to build shared instrumentation that wasn't on either roadmap?** By mapping the journey myself first and bringing a specific metric spec at reaction ID × FSN level rather than a request for "better data." Half the resistance to instrumentation requests is that they're vague. And I ran multiple alignment rounds framing it as the thing that would protect their product from being blamed for pipeline errors — which the four-bucket decomposition then actually did.

- **How did you know bank tier × payment instrument was the error source rather than one of a dozen other candidate granularities?** By testing them systematically rather than hypothesising. The EDA computed offer adoption at multiple product-attribute × offer-attribute combinations across three comparable historical events and measured projected-versus-actual BCD deviation at *each* granularity. Bank tier × instrument stood out because the adoption rates themselves differed significantly across instruments within the same bank tier, which is exactly the signature of a granularity that's being collapsed incorrectly.

- **Why the 8–10 hour threshold in the feedback loop? Isn't that arbitrary?** It's empirical rather than arbitrary — live cumulative adoption early in an event is dominated by early-purchaser mix and is a poor predictor of full-event adoption. The threshold is where cumulative live adoption becomes more accurate than the static prior. The 2% update trigger is calibrated to the same ±2% band we used to define ASP adherence, so the system corrects at the granularity we actually measure.

- **What was the hardest part?** Holding the diagnosis when it was inconvenient. Some early Pilot metrics genuinely looked positive and Data Science had a favourable read, so establishing that ASP deviation was masking the real picture meant contradicting a narrative several teams preferred. The four-bucket decomposition helped, because it let me be precise about which part was the product's problem and which part wasn't.

- **What would you do differently?** Push for the BCD adoption model to be a proper predictive model rather than a granular static prior with a live correction. The granularity recommendation plus feedback loop was the right fix for the Pilot timeline, but the underlying gap — no scientific model for offer adoption projection — was still there when I handed it over.

Why This Story Screams Dive Deep

- ✔ Did the unglamorous foundational work first — reaction ID × FSN instrumentation, timestamp reconciliation across two products
- ✔ Root cause found four layers down: a collapsed granularity, not a model quality problem
- ✔ Two independent decompositions (go-live sync; projection error vs left-over gap) before drawing any conclusion
- ✔ Separated the structurally unfixable from the fixable, preventing wasted optimisation
- ✔ Deployment detail carried through to a fallback chain and an empirically-set feedback threshold
- ✔ No task treated as beneath the role — including building the data layer nobody else wanted to own

---

Q12. Tell me about a time when you anticipated a potential issue and took preventive action.

Coupang Leadership Principle: THINK SYSTEMATICALLY *(secondary: Demand Excellence)*

The principle in plain English: Coupang's Think Systematically is short and unusually specific: build scalable processes with prompt feedback mechanisms, and take measures not only to fix defects but to *prevent* them in future. So this question is not really about foresight — it's about whether your response to a known risk was a process that removes it, or a plan to handle it faster. Answers where you were personally vigilant score much lower than answers where you built something that made vigilance unnecessary, and the strongest answers show the fix outliving the situation.

How interviewers may frame this question:

- Tell me about a time you anticipated a potential issue and took preventive action.
- Describe a time you prevented a problem rather than fixed one.
- Tell me about a process you built to stop a recurring failure.
- Give me an example of when you built something to prevent defects rather than catch them.
- How do you prepare for a high-stakes, time-bound event?
- Tell me about a risk you identified that nobody else was worried about.
- What's something you built that's still being used?

Story: BBD Control Tower — Omniture preparedness, the org-wide SOP, and dry runs

Why this is the best fit:

This is the cleanest prevention story available, because the failure mode was known, the consequence was severe and public, and the fix was structural rather than personal.

Omniture queue clogging during BBD would delay hourly mailers, and delayed mailers during an eight-day event with the CEO and CFO watching produce escalations that damage confidence in every number the team produces. The response wasn't a monitoring rota — it was an organisation-wide SOP that changed how every Analytics team configured reports, a hardware readiness signal to Adobe based on projected traffic, and repeated dry runs weeks in advance. And the proof it was systematic rather than heroic is that the SOP became standard pre-event practice for subsequent sale events.

2-Minute Executive Summary:

"BBD is eight days of continuous NRT reporting consumed directly by the CEO, CFO and SVP, and the traffic layer of the control tower runs on Omniture. From prior BBDs there was a known failure mode: when too many unoptimised reports run simultaneously during peak traffic, the Omniture report queue clogs. That delays hourly mailers, and delayed mailers during BBD produce leadership escalations — which cost far more than the delay itself, because they erode confidence in every number the team produces. My call was that this could not be managed reactively during the event, so I eliminated it beforehand. Three things. First, I worked with the Adobe team to create an organisation-wide SOP for report optimisation — P0/P1 tagging for sale-critical versus non-critical reports, guidelines on segment construction avoiding 'contains' operators, limits on simultaneous scheduled reports, and a cap of ten views per report — and then evangelised it across every Analytics team, ran optimisation sessions with Report Builder users, and tracked removal of non-compliant scheduled reports. Second, I shared expected traffic spike estimates from historical BBD data with Adobe so hardware readiness was in place before the event. Third, multiple dry runs of hourly mailers and live dashboards weeks ahead, to catch data gaps, target upload errors and metric calculation issues before they could surface live. Result: zero queue clogging, zero escalations across eight days, and the SOP became standard pre-event practice."

Full STAR Version

SITUATION

BBD is Flipkart's largest sale event — eight days of continuous, high-stakes NRT reporting where the entire leadership needs live L0/L1 metric access, real-time achievement reads and rapid deep dives whenever a number underachieves. I was responsible for working with Engineering to enable the live BBD tracker capturing all FK L0 metrics in NRT, and I solely managed control-tower visibility — the room where Business and the CEO sit — for BBD metrics deep-dive across Analytics.

The traffic layer runs on Omniture (Adobe Analytics), and prior BBDs had established a specific known risk: **queue clogging when too many unoptimised reports run simultaneously during peak traffic**. The mechanism is straightforward — heavy segments and many scheduled reports competing for the same processing capacity — and the consequence is not just a slow report. Clogging delays the hourly mailers, and a delayed mailer in the control tower means leadership doesn't have its numbers on time, which produces escalation. The second-order cost is worse than the first: once leadership has escalated on data availability, every subsequent number carries a question mark.

TASK

Anticipate and eliminate the risk before BBD started, rather than managing it reactively during the event. The important framing here is that "manage it faster during the event" was a genuinely available option — we could have staffed a monitoring rota and killed offending reports as clogging appeared. I rejected that, because a reactive fix guarantees at least one delayed mailer, and one is enough.

ACTION

Step 1 — Wrote an organisation-wide SOP rather than fixing our own reports. This was the key decision. The clogging risk did not come from the control tower's own reports — it came from *every* Analytics team's scheduled reports competing for the same capacity during peak. Optimising only my own reports would have been necessary and useless. So I worked with the Adobe team to create an org-wide SOP for Omniture report optimisation:

- **P0/P1/P2 tagging** to separate sale-critical from non-critical reports, so priority was explicit rather than assumed.
- **Segment construction guidelines** — specifically avoiding "contains" and "contains any" operators, which are expensive to evaluate at BBD traffic volumes.
- **Limits on simultaneous scheduled reports**, so the peak-hour load was bounded.
- **A cap of ten views per report**, which addressed a common pattern of one report carrying dozens of breakdowns.

Step 2 — Drove adoption, because an SOP nobody follows is a document. I evangelised the SOP across all Analytics teams before BBD, ran optimisation sessions directly with Report Builder users — which mattered because most non-compliance was habit rather than resistance, and people needed to see how to rebuild a segment without a "contains" clause — and kept a tab on removal of non-compliant scheduled reports. That last part is the unglamorous half: adoption without verification isn't adoption.

Step 3 — Gave the infrastructure owner a number they could act on. I shared expected traffic spike estimates by sales channel, derived from historical BBD data, with the Adobe team so hardware preparedness could be arranged before the event. This is a small piece of work with high leverage — infra teams can provision for a projected spike, but they can't provision for "BBD will be busy."

Step 4 — Dry-ran everything that could fail, weeks early. Multiple dry runs of the hourly mailers and live tracking dashboards, well before BBD, specifically to catch data gaps, target upload errors and metric calculation issues while there was still time to fix them properly. This ran alongside target validation for every FK L0 metric with Demand Planning, dry runs with Engineering on target uploads, and verification of correct hourly and daily phasing — so that every mailer section populated correctly with achievement and callouts. The dry run is also where definitional mismatches surface, which is how a discrepancy between the DP team's NC definition and our reporting pipeline's definition got caught before it could appear live.

Step 5 — Backed it with a roster so prevention had a fallback. Prevention shouldn't mean no contingency. I coordinated 20+ teams across Analytics, IT, Omniture and FDP, securing 24×7 rosters for reporting, deep-dive and infra support across the full BBD period, with a 3–4 member core team in the war room. Because the preventable failures had been prevented, that capacity was available for genuinely unpredictable things — for instance, when a payment gateway failed, quantifying the GMV-loss impact within minutes using pre-built cheat sheets.

RESULT

- **Zero Omniture queue clogging during BBD.**
- **Zero leadership escalations across all eight days**, including from the CEO and CFO, on NRT data visibility or L0 deep-dive.
- **The SOP was adopted as standard pre-event practice** for subsequent Flipkart sale events — the fix outlived the event it was built for.
- The dry runs caught target upload and definitional issues pre-event, including an NC definition mismatch between Demand Planning and the reporting pipeline that would otherwise have shown up as a live discrepancy between the target mailer and the tracker.

KEY LEARNING

The best risk management is prevention, not faster mitigation — and the distinction is whether your fix requires anyone to be paying attention. A monitoring rota would have made us quicker at responding to clogging; the SOP made clogging not happen. The second lesson is about scope: the risk originated outside my team, so a fix scoped to my team would have been thorough and pointless. High-stakes, time-bound events reward obsessive preparation precisely because the live window is the most expensive place to solve anything.

What Interviewers Will Probe

- **Why an org-wide SOP rather than just optimising your own reports?** Because the clogging wasn't caused by our reports. Omniture capacity is shared, so a control tower with perfectly optimised reports still clogs if forty other scheduled reports with heavy segments run at peak. Scoping the fix to my own team would have felt responsible and changed nothing.

- **How did you get other teams to comply when you had no authority over their reports?** Mostly by making compliance easy rather than mandatory. The optimisation sessions with Report Builder users were the highest-leverage part, because most non-compliance was habit — people didn't know a "contains any" segment was expensive. Then P0/P1 tagging gave teams a way to protect what genuinely mattered to them, so the SOP wasn't purely a restriction. And I verified removal of non-compliant scheduled reports rather than assuming.

- **How did you know queue clogging would actually happen? Isn't this over-preparation?** It was a known failure mode from prior BBDs, not a hypothesis.

And the asymmetry justifies it: the cost of the SOP was a few weeks of sessions and follow-up, while the cost of one delayed mailer is a leadership escalation that puts a question mark over every number for the rest of the event. I'd have done the same work at a much lower probability.

- **What did the dry runs catch that you wouldn't have predicted?** The definitional mismatch was the valuable one — Demand Planning had set the BBD NC target on a slightly different definition (event window versus calendar day) than our reporting pipeline captured. That's not a technical bug and no amount of code review would have surfaced it. It only appears when you run the target and the tracker side by side against real data, which is exactly what a dry run is for.
- **Did the prevention work ever come at the cost of something else?** Yes — it consumed time I could have spent on deeper analysis in the Intrigue window, and I made that trade deliberately. The judgement was that a control tower failing on day one is unrecoverable within the event, whereas a deferred analysis is recoverable afterwards.
- **How do you decide what to dry-run when you can't dry-run everything?** I work backwards from irreversibility. Anything that would be visible to leadership before we could fix it gets dry-run — mailers, target uploads, phasing. Anything we could quietly correct mid-event doesn't. That's also how the roster was scoped: prevention for the visible-and-irreversible, contingency capacity for everything else.

Why This Story Screams Think Systematically

- ✔ Built a process that removed the defect rather than a rota that responded to it faster
- ✔ Scoped the fix to where the risk actually originated — org-wide, not team-wide
- ✔ Paired the process with a prompt feedback mechanism: dry runs surfaced defects while they were still cheap
- ✔ Verified adoption rather than assuming it — tracked removal of non-compliant reports
- ✔ Gave the infrastructure owner an actionable projection so they could prevent their half too
- ✔ The SOP became standard practice for subsequent events — the process outlived the problem

---

SECTION 4: ADAPTABILITY & RESILIENCE

Q13. Tell me about a time when your project priorities changed suddenly. How did you adapt?

Coupang Leadership Principle: AIM HIGH AND FIND A WAY *(secondary: Move with Urgency)*

The principle in plain English: Coupang's phrasing here is the most ambitious of the fifteen — aim for jaw-dropping results and work backwards, because goals usually get set to what seems possible within existing conditions, and leaders should have the courage to aim high and then drive whatever change is necessary to deliver the "wow." Applied to a sudden change in priorities, the interviewer is watching for whether you treated the harder new requirement as a constraint to negotiate down or as the target to rebuild toward. The weak answer defends the original plan; the strong answer rebuilds the architecture because the higher bar was the right bar.

How interviewers may frame this question:

- Tell me about a time your project priorities changed suddenly. How did you adapt?
- Describe a time you had to change direction mid-project.
- Tell me about a time a requirement changed after you'd already built something.
- How do you respond when the goalposts move?
- Give me an example of when you had to rebuild something you'd already delivered.
- Tell me about a time you were asked for something that seemed impossible with your current approach.
- Describe a setback that turned into an improvement.

Story: Multi-agent RCA System — the V1 GSheet → V2 BigQuery pivot mid-build

Why this is the best fit:

The requirement that arrived mid-build was genuinely harder than the one I had designed for: near-real-time actionability at City × Category × Vertical × Brand, against an architecture with a 10M-cell ceiling and 2+ hours of latency. The available options were to negotiate the requirement down — coarser cuts, accept the lag — or to rebuild the data layer. I rebuilt it, because the harder requirement was the one that made the system actually useful for intraday action. And that is the Aim High test: the architecture changed to meet the ambition rather than the ambition being trimmed to fit the architecture. What was framed as a setback became the system's strongest differentiator.

2-Minute Executive Summary:

"Midway through building the multi-agent RCA system for Flipkart Minutes, after V1 was deployed on GSheet as the backend, the business requirement changed materially: the Minutes team needed near-real-time actionability at City × Category × Vertical × Brand

level. GSheet fundamentally couldn't support that — the 10M-cell file limit made that cardinality impossible, and the hourly pipeline meant over two hours of data latency. So the entire backend had to be rebuilt. I could have negotiated the requirement down to coarser cuts, and I didn't, because granular NRT was what made the system useful for intraday action rather than for retrospective reporting. The pivot meant rebuilding the Metric Pull agent from scratch as a sequence — entity extraction, then an NLQ→SQL agent carrying a semantic layer of standard aggregation mappings with schema discovery and fuzzy dimension-filter resolution, then BQ execution, then a growth/RAG Python function returning markdown — and securing a dedicated BigQuery processing slot from Engineering. The migration exposed a set of SQL generation failures, and I documented each as a prompt engineering rule, so the system got more robust through the iteration. Data freshness went from over two hours to under one, and the system now delivers actionability at exactly the granularity that triggered the pivot. The change that initially looked like a setback became the system's strongest differentiator."

Full STAR Version

SITUATION

V1 of the multi-agent system was live, running on the existing GSheet trackers as its backend. That had been a deliberate V1 choice: the team already used GSheet trackers for some deep dives, so usable data was readily available there, and it got a working capability in front of users quickly to demonstrate the concept and gather early feedback. The Metric Pull agent at that point was two sequential agents — an entity-extraction agent pulling metric list, target/benchmark date and hour into machine-readable JSON, and a metric-puller agent with a GSheet connection tool plus a growth/RAG tool.

Then the requirement changed. The Minutes team escalated that they needed **near-real-time actionability at City × Category × Vertical × Brand level**. Two hard constraints made that impossible on GSheet: the **10M-cell file limit**, which made a granular City × Cluster × Vertical × Brand view unachievable, and **data freshness of over two hours** via the hourly pipeline. Minutes is a quick-commerce business chasing OPD where the whole point of the diagnosis is to act during the day — a two-hour-old answer at a coarse cut is a report, not an intervention.

TASK

Pivot the entire backend architecture from GSheet to BigQuery without losing the progress made, while also improving the system's capability. Concretely that meant rebuilding the Metric Pull agent from scratch, designing a semantic layer for NLQ→SQL, and securing a dedicated BQ processing slot from Engineering — all while the business continued using V1 daily.

ACTION

Step 1 — Accepted the higher bar instead of defending the architecture. The instinctive move when a requirement invalidates your design is to argue the requirement down: do we really need brand-level, could we live with a two-hour lag. I didn't, for a

specific reason — the granularity *was* the value. The output the business needed was "these two cities need orders solved, these ten stores, and within each store this vertical × brand needs instock or pricing or serviceability fixed." Strip the granularity and the system stops producing actionables and starts producing observations. So I framed the pivot internally as an upgrade rather than a correction: V1 had proven the concept; V2 would prove the scale.

Step 2 — Separated what changed from what didn't. This was what made the pivot cheap rather than catastrophic. The product goal hadn't changed at all — natural-language metric pull and RCA with growth against benchmark and RAG status. Only the data layer had. So the Master router, the Discovery agent, the RCA issue tree, the attribution tools and the synthesis agent all survived intact, along with the prompt architecture and every prompt-engineering rule learned in V1. What needed rebuilding was one sub-flow.

Step 3 — Rebuilt Metric Pull as a proper NLQ→SQL chain. Entity-extraction agent → NLQ→SQL agent → BQ execution tool → growth/RAG Python function returning markdown. The NLQ→SQL agent is where the real design work went, and it carries four things: a **semantic layer of standard aggregation mappings** (Orders = COUNT(DISTINCT order_external_id), GMV = SUM(net_amount)) so metric definitions are fixed rather than inferred; **schema discovery**; **dimension-filter resolution via fuzzy matching**, because users type city and brand names inexactly; and **shared query patterns** instead of 200+ hand-written queries, which is what makes it maintainable.

Step 4 — Used the pivot's failures as hardening. The BQ migration exposed a series of SQL generation bugs, and I treated each as a permanent rule rather than a one-off fix: underscores not hyphens in aliases (BQ errors otherwise), spaces between keywords to avoid concatenation crashes, and strict prohibitions on inventing CASE WHENs or using dimensions outside the schema. The system that emerged was more robust than V1 had been, not just faster — the iteration was doing real work.

Step 5 — Made the infrastructure ask on the strength of real usage. Engineering provisioned a **BQ fact with a dedicated processing slot**, cutting freshness latency to within an hour. That request landed because V1 was in daily use and had hit a demonstrable ceiling against a stated business need — I wasn't asking for infrastructure for a hypothesis, which is a much weaker ask.

Step 6 — Managed expectations rather than absorbing the pressure silently. I communicated clearly to the business team that V2 would take additional time but would deliver the NRT granularity they needed, and delivered incremental updates through the build to keep confidence high. V1 stayed live throughout, so the pivot never meant a gap in capability.

RESULT

- **Data freshness: over 2 hours → under 1 hour** with the dedicated BQ processing slot.

- **Actionability at City × Category × Vertical × Brand**, down to input-metric level — pricing, OOS, speed, SLA breach, surge visits — which is precisely the requirement that triggered the pivot.
- **RCA turnaround from hours or days to near-real-time.**
- **V2 was built faster than V1 would suggest**, because V1 had already validated the prompt logic, the issue tree and the real user requirements.
- The system is in daily production use by both Analytics and Business, checked at the start of the day, and the architectural change that initially felt like a setback became the system's strongest differentiator.

KEY LEARNING

Mid-project pivots are far less disruptive when you separate *what you're building* from *how you're building it*. The product goal never changed; only the architecture did. Holding that distinction explicitly is what let most of the system survive and made the pivot feel like an upgrade rather than a failure. The deeper lesson is about which side of the negotiation to take when a requirement gets harder: the granularity was the whole value of the system, so trimming it to protect the architecture would have preserved my design and destroyed the product.

What Interviewers Will Probe

- **Why not negotiate the requirement down? Coarser cuts would have shipped immediately.** Because the granularity was the product. The business need was to know which store and which vertical × brand needs instock or pricing fixed *today*. At a coarse cut with a two-hour lag, the system tells you Bengaluru is down — which the mailers already did. I'd have preserved my architecture and delivered something that didn't change anyone's day.

- **Wasn't the GSheet V1 therefore a wasted effort?** No, and this is the part I'd defend hardest. V1 validated the prompt architecture, the issue tree and the RCA logic — all of which transferred directly. It also generated the real usage evidence that justified a dedicated BQ processing slot, which I would not have got for an unproven concept. The sunk cost was one sub-flow's data connector.

- **How did you keep the business confident during a rebuild?** V1 stayed live, so there was never a capability gap. And I was explicit that V2 would take longer but deliver what they'd asked for, with incremental updates through the build. The thing that erodes confidence is silence during a rebuild, not the rebuild itself.

- **How did you decide what to keep versus rebuild?** By asking what depended on the data layer. The Master router, Discovery agent, RCA issue tree, attribution tools and synthesis agent are all data-layer agnostic — they operate on outputs and context variables. Only the metric-fetch path was coupled to GSheet. That mapping took an afternoon and saved weeks.

- **What if BigQuery had also turned out to be insufficient?** Then the next constraint would have been query latency at agent scale rather than storage or

freshness — which is exactly the problem the follow-on Metric Store work addresses, precomputing L0/L1 metric values so a proliferation of agents doesn't clog the fetch queue. So in a sense that's what happened: the architecture kept being pushed by the ambition rather than the other way round.

- **What's the biggest lesson?** That a moved goalpost is information about what the product is actually for. My instinct in the moment was that the requirement had changed unreasonably late. In hindsight it hadn't changed at all — it had become clear, and V1 was what made it clear.

Why This Story Screams Aim High and Find a Way

- ✔ Took the harder requirement as the target rather than negotiating it down to fit the build
- ✔ Rebuilt the data layer rather than trimming the ambition — architecture served the goal, not the reverse
- ✔ Worked backwards from the business outcome (act today, at store × brand level) to the infrastructure needed
- ✔ Drove the change necessary: secured dedicated BQ infrastructure that didn't exist for this use case
- ✔ Turned the failure surface of the migration into permanent hardening rules
- ✔ The "setback" became the system's strongest differentiator and its reason for daily adoption

---

Q14. Describe a situation where you faced a major setback and how you recovered.

Coupang Leadership Principle: DELIVER RESULTS WITH GRIT *(secondary: Disagree and Commit)*

The principle in plain English: Coupang is explicit that grit is not effort — *we do not confuse long hours or effort with results*. The rest of the definition is the useful part: leaders finish what they start, work around dependencies, and never let excuses stop them. So a setback question is testing whether you finished. The pattern they're looking for is repeated resistance, a recovery that changed something concrete rather than just persisting harder, and a delivered outcome at the end. Persistence without a delivered result is the answer that fails this principle.

How interviewers may frame this question:

- Describe a situation where you faced a major setback and how you recovered.
- Tell me about a time your recommendation was rejected.
- Give me an example of when you had to push something through significant resistance.
- Tell me about the longest you've had to fight for an idea.
- Describe a time your finding contradicted what leadership believed.

- How do you keep going when a project stalls repeatedly?
- Tell me about a time you finished something that would have been easy to abandon.

Story: BBD Customer Shape Projection — challenging the High-TPC-first playbook

Why this is the best fit:

The setback here was not a project failure — it was a *dismissal*. I produced a finding that contradicted the dominant BBD playbook, and the first response was not disagreement but "this can't be right." That is the hardest kind of setback to recover from, because there's no argument to answer. And the recovery was analytical rather than interpersonal: I didn't persist harder, I rebuilt the framing until the perceived trade-off disappeared. The grit shows up as multiple review iterations over a genuinely long period, holding a position that every other team in the room had an incentive to reject, and landing a delivered outcome — a ~10% baseline shift and a permanent change to how BBD strategy is framed.

2-Minute Executive Summary:

"Post-festive baseline shift — where platform customer DRR runs about 1.5x pre-festive DRR after BBD — is one of Flipkart's most important long-term growth outcomes. For years the dominant belief driving BBD planning was to maximise GMV by pivoting toward High-TPC customers, who drive the highest GMV per customer during the event. BBD GMV was the metric everyone was measured on, so the entire resource allocation and targeting machinery was built around that instinct. I was asked to decompose the baseline shift and project the customer shape that would maximise it. The finding went directly against the playbook: about 30% of the shift was led by NCs acquired during festive, about 20% by the incremental post-festive NC DRR network effect, about 30% by Low and Med TPC customers who transacted during festive — and High TPC, the cohort everyone prioritised, contributed only the remainder, with its contribution structurally capped because those users were already entrenched. The first response was dismissal, not disagreement. The recovery was not persistence — it was rebuilding the framing. I built a GMV-guardrailed customer shape simulator that projected multiple shapes with BBD GMV as a hard constraint on every scenario, which showed you could shift the shape toward NC and Low/Med TPC *without* missing GMV, because GMV depends on the absolute size of the High-TPC cohort being served, not its share of the mix. Once the trade-off was shown not to exist, the resistance dissolved. Leadership adopted the shape as the BBD targeting framework, diversity and event-to-event repeat became explicit BBD goals for the first time, and it drove a ~10% baseline shift in customer DRR."

Full STAR Version

SITUATION

BBD is the event after which the platform sees a baseline shift across most metrics, with post-festive customer DRR running ~1.5x pre-festive DRR across the ~1-month festive window covering BBD and Diwali. Understanding and maximising that shift is one of the

highest-leverage things the Events charter can do, because it converts a one-time GMV spike into lasting platform growth.

For years the dominant belief was straightforward: maximise GMV by pivoting toward High-TPC customers (8+ transactions in 12 months), who drive the highest GMV per customer during the event. BBD GMV was the primary metric business teams, category and leadership were measured and rewarded on, so the entire BBD resource allocation machinery, targeting strategy and success narrative was built around this instinct. I was asked to decompose the post-festive baseline shift and project the BBD customer shape that would maximise it — and the answer the data produced was the opposite of what the room expected.

TASK

Decompose what actually drove the post-festive baseline shift, by breaking down customer DRR by TPC cohort (Low = 1–3 transactions/12M, Med = 4–7, High = 8+) across the pre-festive window (60 days prior) and post-festive window (60 days after) — and then project the BBD customer shape that would maximise baseline shift while keeping BBD GMV as a non-negotiable guardrail. The challenge was never analytical. It was that the finding asked the organisation to deprioritise what it had always done, in favour of cohorts that looked worse on the metric everyone was rewarded on.

ACTION

Step 1 — Built the decomposition. Breaking the ~1.5x post-festive DRR multiplier into cohort contributions gave an unambiguous answer: ~30% of the shift led by NCs acquired during festive, ~20% by the incremental post-festive NC DRR network effect (festive brings more NCs daily than pre-festive, and the effect persists), ~30% by Low and Med TPC customers who transacted during festive, and the remainder by High TPC. High TPC's contribution was structurally capped because those customers were already entrenched platform users — BBD did not meaningfully change their long-term trajectory.

Step 2 — Established the mechanism, not just the number. This is what made the finding robust rather than a statistical artefact. Low and Med TPC customers are early in their platform journey, and their exposure to BBD deals, selection range and affordability constructs plays a pivotal role in establishing the platform's value proposition for them. For High TPC that value is already established — they come to BBD regardless of targeting. I had separately run an extensive EDA comparing similar customer cohorts, controlled on TPC, SPC, SCPC, BUPC and pre-purchase BU, on their long-term behaviour when they purchased during an event versus a comparable BAU period. Low and Med TPC were exactly where events brought maximum incrementality: **~30% lift in TPC and ~4% improvement in churn** versus the BAU-period equivalent. That was the mechanism-level proof.

Step 3 — Hit the setback. Leadership's first response was that shifting focus away from High TPC felt like a risk to the GMV number they were accountable for. This was not disagreement with the analysis; it was dismissal of the conclusion, which is a different problem — there was no analytical objection to answer. Multiple rounds of review followed.

My manager's year-end feedback captured the shape of it accurately: *"problem definition, approach and presentation went through several iterations to finally reach some conclusion."*

Step 4 — Recovered by removing the trade-off rather than arguing about it. This was the actual recovery, and it was analytical work, not persuasion. Instead of presenting "target Low TPC and NC more," I built a **GMV-guardrailed customer shape simulator**: a bottom-up organic BBD customer projection by TPC cohort and NC, based on customer incidences in previous BBDs by recency-of-purchase and TPC, computing the YoY BAU multiplier of incidences during a BAU period and applying it to historical BBD incidences. Then I projected multiple shapes — NC-heavy, Low/Med TPC-heavy, High-TPC-heavy — with BBD GMV as a **hard constraint on every scenario**. The result: you could shift the shape toward NC and Low/Med TPC without missing BBD GMV, because GMV depends on the absolute size of the High-TPC cohort still being served, not on its share of the customer mix. The objection wasn't rebutted; it was dissolved.

Step 5 — Connected the two analyses into one story. The long-term event impact analysis was the mechanism-level proof that made the baseline shift decomposition credible. Alone, the decomposition was a correlation. Together they told a coherent story: Low/Med TPC customers benefit disproportionately from events because the event experience shapes their long-term platform behaviour, and that shows up as baseline shift; High TPC customers are already shaped. Each review iteration I held the stance, tightened the GMV guardrail logic, and connected the finding more explicitly to the long-term impact work.

Step 6 — Held the position across a room with aligned incentives against it. Category, business and marketing teams all had reasons to prefer High TPC focus, because their in-event GMV metrics looked better that way. I maintained the analytical position consistently across every iteration — not aggressively, but without softening the finding to make it more comfortable.

Step 7 — Triangulated the targets so the recommendation was operationally executable. Validated BBD customer targets bottom-up via historical BBD incidences by recency and TPC × the BAU YoY multiplier, and triangulated Demand Planning's target multiple ways — GMV and Units from category targets, and Units ÷ UPC as a cross-check since UPC is slow-moving with a known historical range. Then hourly phasing of BBD customers based on previous BBD phasing, so the shape could actually be steered during the event rather than just planned before it.

RESULT

- **Leadership adopted the GMV-guardrailed NC and Low/Med TPC customer shape** as the BBD targeting framework.
- **Diversity and event-to-event repeat were taken as explicit BBD goals for the first time** — a fundamental shift in how BBD strategy was framed.
- **~10% baseline shift in customer DRR** post-BBD.
- **+0.7 Mn incremental customers diversified over plan** during BBD against the diversity goal.

- **Low-TPC event-to-event repeat improved +500 bps YoY.**
- The analysis shaped the following year's event strategy — **Low and Med TPC acquisition and retention became a year-round charter goal**, not just a BBD consideration.
- Manager's year-end feedback: *"An analysis that led to forming the core of event strategy this year — to focus on NCs and Low TPCs throughout the year. Given we have not done something similar before, this was one of a kind analysis… A very detailed analysis that exhibited APS, EE and BP altogether."*

KEY LEARNING

The hardest recommendations to recover from are the ones asking people to deprioritise what they have always done — especially when what they have always done correlates with the metric they're measured on. The recovery here was not about changing the finding; it was about building the scaffolding that made it impossible to dismiss. The GMV guardrail simulator was that scaffolding: it removed the perceived trade-off, and once people could see the recommendation didn't cost them their GMV number, the resistance dissolved. **The setback was not a data problem. It was a framing problem — and solving the framing problem was the real analytical work.**

What Interviewers Will Probe

- **How did you handle leadership's concern that this would hurt BBD GMV?** I never asked them to trade GMV for baseline shift. The simulator had GMV as a non-negotiable guardrail from the start, so what I presented were the specific customer shapes that met the GMV target *while* maximising baseline shift. Once the trade-off was shown not to exist, the objection had nothing to attach to.

- **How many iterations did it take, and how did you avoid it becoming a war of attrition?** Several — the manager feedback captures it. What kept it from being attrition is that each iteration changed something concrete: tighter guardrail logic, more explicit connection to the long-term impact analysis, side-by-side scenarios with GMV held constant. Repeating the same argument louder would have been effort without results, which is the opposite of what grit means here.

- **What would you have done if leadership had not adopted it?** Documented the finding and methodology clearly and proposed a controlled BBD experiment — one cohort on the NC/Low TPC shape and one on the traditional High TPC shape — to generate empirical evidence that would have been undeniable the following year. Sometimes the right answer needs one more cycle of evidence before it lands, and the job is to make sure the evidence exists rather than to win the current meeting.

- **Why was this a setback rather than a normal stakeholder alignment challenge?** Because the initial reaction was dismissal, not disagreement. When you present a counterintuitive result and the first response is "this can't be right," there is no objection to answer — you're in setback territory. The recovery required rebuilding the framing repeatedly until the logic was airtight enough to survive the dismissal instinct.

- **How did you know your decomposition was right and not just a cohort-definition artefact?** Two independent supports. The mechanism — the long-term impact EDA on cohorts controlled on TPC, SPC, SCPC, BUPC and pre-purchase BU, showing ~30% TPC lift and ~4% churn improvement specifically for Low/Med TPC event transactors. And the target triangulation: the bottom-up customer projection was cross-checked against Demand Planning's number via Units ÷ UPC. A cohort artefact wouldn't survive both.
- **How did you sustain the position when every other team's incentives ran against you?** By making it about their constraint rather than my conclusion. Once the guardrail was GMV — their metric, held hard — I wasn't asking anyone to accept a worse outcome on the thing they were accountable for. That's what made holding the position sustainable rather than adversarial.
- **Was there a point you considered softening the finding?** Yes, after the first few dismissals, the tempting move was to present it as "consider some additional NC focus" — directionally true, easy to accept, and worth nothing. That version wouldn't have changed the targeting framework or produced a 10% baseline shift.

Why This Story Screams Deliver Results with Grit

- ✔ Repeated dismissal over multiple review cycles, held without softening the finding
- ✔ Recovery was concrete work, not persistence — the guardrail simulator dissolved the objection
- ✔ Worked around the real dependency (every team's incentive was GMV) by making GMV a hard constraint
- ✔ Finished it: adopted as the BBD targeting framework, not left as an interesting analysis
- ✔ Delivered measurable outcomes — ~10% baseline shift, +0.7 Mn diversified, +500 bps event repeat
- ✔ Changed the org permanently: Low/Med TPC became a year-round charter goal

---

Q15. Give an example of when you had to learn a new skill quickly to complete a project.

Coupang Leadership Principle: LEARN VORACIOUSLY *(secondary: Move with Urgency)*

The principle in plain English: Coupang frames this as being hungry for the best ideas and seeking them from all sources, embracing growing pains, and — pointedly — *ego is the enemy: we avoid rationalising mistakes and are vocally self-critical.* The distinction that matters for this question is motive. Coupang isn't rewarding curiosity as a personality trait; it's rewarding learning that happened because the business needed it. So the strong answer emphasises that the learning was instrumental, that you were willing to be a beginner in public, and that you were candid about what went wrong on the way.

How interviewers may frame this question:

- Give me an example of when you had to learn a new skill quickly to complete a project.
- Tell me about a time you worked outside your area of expertise.
- How do you get up to speed on something you know nothing about?
- Tell me about the most recent significant thing you learned and why.
- Describe a time you were the least knowledgeable person in the room.
- Give me an example of learning something because the business needed it, not because you were interested.
- Tell me about a mistake you made while learning something new.

Story: Multi-agent RCA System — learning agentic AI, prompt engineering and LLM cost architecture from scratch

Why this is the best fit:

The gap between what I knew and what the project required was genuine and wide: a strong analytics and ML background, and zero prior experience with agentic frameworks, LLM prompt engineering, or cost architecture for multi-model systems. Google ADK was new to the team, no internal playbook existed, and the business needed a production-ready system in weeks. What makes it a good Learn Voraciously story rather than a good learning story is the motive and the candour — the learning was instrumental, driven by a business problem rather than by interest, and the honest version includes the fact that my first instincts on prompts were wrong and had to be discarded through repeated failure.

2-Minute Executive Summary:

"When I was asked to build Flipkart's first multi-agent RCA system for the Minutes business, I had a strong analytics and ML background and zero prior experience with agentic AI frameworks, LLM prompt engineering, or cost architecture for multi-model systems. Google ADK was new to the team, there was no internal playbook, and the business needed something production-ready within weeks. I learned by building rather than by preparing. I started with the Master router — the simplest component — while learning ADK, and expanded outward as understanding grew. On cost architecture I learned the fundamentals from first principles because I needed them for an approval: roughly 1,000 tokens to 750 words, input tokens cheaper than output, and per-request costs of Flash at 1x, Pro at ~4x, Claude at ~11x — which is what secured the LLM key and drove the hybrid-model design. On prompt engineering my initial instinct was wrong: I wrote clear plain-English paragraph instructions, and they produced unpredictable agent behaviour. Through repeated failures I learned that instruction prompts need explicit structure — role, tools, positive triggers, negative triggers, output format, context-variable handling, few-shot examples, with Markdown headers and bolding — which materially improved instruction-following. I also learned to stop trusting the model with arithmetic and provide all computation as Python tools. Later I learned LookML to build the semantic layer that became the foundation for enterprise-scale agentic analytics. The system went to

production and my manager's feedback was that I'd led the approach end-to-end in an environment where nobody knew the right way to build."

Full STAR Version

SITUATION

The Minutes business needed a natural-language system for metric pulls and order-drop RCA to replace a workflow consuming significant analyst bandwidth on GSheet trackers and ad-hoc deep-dive SQL. I was part of the GenAI POD in Analytics, responsible for enabling GenAI solutions across Analytics and Business, so this landed with me. The honest starting position: I had depth in analytics, experimentation and ML, and no experience with agentic frameworks, LLM prompting, or the cost architecture of multi-model systems. Google ADK's documentation was limited. Nobody on the team had built anything like this, so there was no one to learn from internally — we were going to be the team that wrote the playbook, which is a good learning environment and a bad safety net.

TASK

Learn enough about agentic architecture, Google ADK, prompt engineering, LLM cost modelling and SQL semantic-layer design to build a working production system — while building it, within weeks, for a business that needed it working rather than needing me educated.

ACTION

Step 1 — Started building before understanding everything. Rather than trying to learn ADK comprehensively first, I built the Master router agent — the simplest component, a traffic controller routing queries to the right specialist — while learning the framework, then expanded outward as understanding grew. The framework choice itself came out of multiple brainstorming rounds with leadership rather than from my own judgement, which was the right call given how little I knew: we picked ADK because it provides multi-agent orchestration (hierarchical, parallel, sequential), agent-as-a-tool and sub-agent delegation, Python functions and MCP connections as tools, shared context variables for passing data between agents, and serverless deployment on Vertex AI — and because it fit our Google infrastructure where data already sat in GSheet and BigQuery.

Step 2 — Learned the commercial layer first, because it gated everything else. An agent needs a brain, and the brain has a price. So the very first thing I had to learn properly was LLM cost architecture: estimating input and output tokens per request, requests per minute, hours of daily use, then costing by model tier — using the ~1,000 tokens ≈ 750 words heuristic and the fact that input tokens are cheaper than output. That produced relative per-request costs of Gemini 2.5 Flash at 1x, Gemini 2.5 Pro at ~4x and Claude at ~11x, which secured LLM-key approval and directly drove the hybrid-model design: Flash for the Master router and Discovery agent, Pro for Metric Pull, Gemini 3 Pro / Sonnet for Orders RCA. Learning the economics before the engineering was accidental at the time and correct in hindsight — the cost structure is what determined the architecture.

Step 3 — Was wrong about prompts, and found out by failing. My instinct was that a well-written plain-English paragraph describing what an agent should do would work, because that's how you brief a person. It produced unpredictable behaviour — agents doing adjacent things, skipping steps, inventing outputs. Through repeated failed iterations I learned that instruction prompts need *structure*, not prose: each agent's prompt defines its role, its tools, positive triggers (what to do), negative triggers (what explicitly not to do), output format and context-variable handling, and few-shot examples — using Markdown headers and bolding, which materially improved the model's ability to follow complex instructions. The negative triggers were the least intuitive and most important part; specifying what an agent must *not* do turned out to matter as much as what it should.

Step 4 — Learned the model's actual weaknesses rather than assuming. Since LLMs do next-token prediction and are poor at arithmetic, I moved all computation into Python tools — a current-date function, growth-versus-baseline calculation, RAG colour-coding, a contribution-analysis tool surfacing only cities contributing ≥10% of an order drop out of 80+, and a Shapley-attribution tool for AOV separating category-mix change from per-category AOV drop. That's a lesson I would not have learned from documentation; it came from watching a confident model produce a wrong number.

Step 5 — Documented failures as rules so the learning compounded. SQL generation on the BQ migration broke repeatedly, and each failure became a permanent prompt rule: underscores not hyphens in aliases, spaces between keywords to avoid concatenation crashes, strict prohibitions on inventing CASE WHENs or using dimensions outside the schema. Treating each failure as a durable rule rather than a fix is what turned a chaotic learning period into a hardened system.

Step 6 — Kept learning outward as the problem got bigger. Once one agentic flow existed, the real problem surfaced: how do you build agentic flows *at scale* when each team has its own definitions, dashboards and SQL repositories, and change management of definitions, RCA trees and synthesis templates would become unmanageable? That required learning **LookML** to build a semantic layer — views mapping to facts and explores where every measure, dimension and business rule is defined exactly once, consumed by agents, dashboards and humans alike — plus designing a governed context store in .md files so business context, metric definitions, issue trees and synthesis templates are versioned separately from the agents that consume them. And I learned enough about event-driven architecture to make RCA proactive: a Pub/Sub event in BQ firing when a metric breaches a threshold, sending a payload to the ADK agent, which runs the RCA and emails the synthesis with actionables to stakeholders.

RESULT

- **A working multi-agent system deployed and adopted in production**, used daily by Analytics and Business in stand-ups.
- **RCA turnaround from hours or days to under an hour**; data freshness from over two hours to under one.
- **The semantic layer plus context store approach became the foundation** for Flipkart's enterprise agentic analytics strategy — one semantic layer as the single

source of truth for every measure and dimension, governed context, and a proactive alert-triggered RCA path.
- Manager's feedback: *"Led the approach, took it end-to-end, got it done in an environment where nobody knew the right way to build."*

KEY LEARNING

The fastest way to learn a technical skill is to have a real production problem that requires it — abstract learning without application has poor retention, and in a novel space there's no documentation to be thorough with anyway. Build first, fill gaps as they appear, and document failures as rules so the learning compounds instead of repeating. The more uncomfortable lesson is that my strongest prior instinct — that clear prose makes clear instructions — was actively wrong, and I only found that out by shipping agents that misbehaved. Being willing to discard a confident assumption quickly mattered more than learning anything specific.

What Interviewers Will Probe

- **How did you learn so quickly?** By working from first principles on the parts that gated progress rather than trying to learn the whole domain. I needed the cost model to get an LLM key, so that came first. I needed the Master router to have anything to test, so that came second. Anything that wasn't blocking, I didn't learn yet.

- **What was the hardest thing to learn?** Prompt structure, because it required unlearning rather than learning. Writing clear instructions for a person and writing them for a model are different disciplines, and my competence at the first actively slowed down the second. Negative triggers — explicitly specifying what an agent must not do — were the least intuitive thing and made the biggest difference.

- **What mistakes did you make while learning?** Two worth naming. Prose prompts, which I've covered. And initially trusting the model with computation — growth calculations and date handling — which produced confident wrong numbers. That's a dangerous failure mode in an analytics tool, and it's why every calculation ended up in a Python tool.

- **Why does this demonstrate learning voraciously rather than just doing your job?** The distinction is where the initiative came from. Nobody asked me to learn LookML or event-driven architecture — those came from noticing that a single custom agentic flow doesn't generalise, and that the change-management problem would eventually eat the whole approach. The Minutes system was my job. The semantic layer and context store were me chasing the bigger problem the first project revealed.

- **How do you avoid learning superficially under that kind of time pressure?** By making the depth requirement concrete: can I explain why this works to the person approving it, and can I predict where it will break? The cost model had to survive an approval conversation. The hybrid-model design had to have a defensible reason per

agent. Where I couldn't meet that bar — the issue tree's traversal reliability — I designed for iteration rather than pretending I knew.

- **What would you tell someone starting the same journey?** Get something end-to-end working badly before you make any part of it good. My earliest instinct was to design the full architecture before building, and in a space with no playbook that's just an elaborate guess. And write down every failure as a rule the same day, because the second time you hit a BQ alias error you won't remember the first.

Why This Story Screams Learn Voraciously

- ✔ Genuine zero-to-production gap: no agentic, prompt-engineering or LLM cost experience at the start
- ✔ Learning was instrumental — driven by a business need and an approval gate, not by interest
- ✔ Vocally self-critical: names two wrong instincts (prose prompts, trusting model arithmetic) rather than presenting a smooth ramp
- ✔ Embraced growing pains — hardened the system through documented failure rather than avoiding failure
- ✔ Sought ideas from all sources: leadership brainstorming on framework choice, Business on issue trees, Engineering on infrastructure
- ✔ Kept learning outward unprompted — LookML, context-store governance, event-driven proactive RCA

SECTION 5: COMMUNICATION & INFLUENCE

Q16. Tell me about a time when you had to communicate a complex idea to a non-technical audience.

Coupang Leadership Principle: SIMPLIFY *(secondary: Influence without Authority)*

The principle in plain English: Coupang's Simplify is short and sharp: complexity is the enemy of scale, speed and customer experience, and leaders are fanatical about simplifying what they touch. Note that it says *what they touch* — not "how they explain things." So the strongest answers to a communication question show that you simplified the thing itself, or at least built a durable mental model the organisation kept using, rather than delivering one well-pitched presentation. The test is whether the simplification outlived the meeting.

How interviewers may frame this question:

- Tell me about a time you had to communicate a complex idea to a non-technical audience.
- Explain a technical concept you've had to make accessible to business stakeholders.
- How do you present analysis to leadership?
- Tell me about a time your audience didn't understand you and what you changed.

- Give me an example of simplifying something complicated.
- Describe how you'd explain a model you built to someone with no statistics background.
- Tell me about a framework you introduced that people kept using.

Story: Intelligent Couponing — explaining uplift modelling to business leadership

Why this is the best fit:

The concepts involved were genuinely inaccessible to the audience — CATE estimation, S-Learner architecture, AUC of a normalised cumulative gain chart — and the audience were the people who controlled the budget, so an unclear explanation didn't just fail to inform, it blocked the decision. What makes this a Simplify story rather than a presentation story is that the simplification was structural and durable: the four-segment framework became shared vocabulary the team kept using in targeting discussions long after the project. The communication architecture outlasted the model.

2-Minute Executive Summary:

"After building the uplift model I had to present the concept and the results to business and leadership stakeholders who made the budgeting decisions. They had no statistical modelling background, and the vocabulary of the work — uplift, CATE, S-Learner, AUC of a normalised cumulative gain chart — was completely foreign. Presenting it technically would have produced confusion and then scepticism, and scepticism in a budget conversation means no. Four things worked. I led with the business problem rather than the model: we have been paying for customers who would have bought anyway, and this tells us which customers only buy because of the coupon. I used the four behavioural segments — Persuadables, Sure Things, Sleeping Dogs, Lost Causes — as the conceptual anchor, which gave them a map they could hold onto through everything that followed. I translated every model output into a business outcome: not 'the top three deciles have the highest CATE' but 'if we target the top 30% of users we get 80% of the new customers at 50% of the cost.' And I made the proof point the confirmatory A/B rather than the model, because for a business audience a real experiment is more credible than any model diagnostic. Leadership approved production deployment, the model ran five to six months without needing re-explanation, and the four-segment framework became shared vocabulary the team used in later targeting discussions."

Full STAR Version

SITUATION

The uplift model was built and validated. The remaining obstacle was entirely communicative: the people who had to approve moving production coupon spend onto a new targeting methodology were business-first stakeholders with no ML background. The concepts they would need to evaluate the decision included conditional average treatment effect, the S-Learner architecture and its double-scoring trick, decile-level calibration, and the AUC of a normalised cumulative gain chart benchmarked against an industry standard.

None of that vocabulary was available to them, and presenting it technically would have produced two bad outcomes — either polite confusion, or the specific kind of scepticism that attaches to a method you can't evaluate.

The stakes were concrete: this was a recommendation to *reduce* the size of the targeted base during a year when ~30% YoY NC growth was the AOP target. On the surface, that looks like accepting fewer customers.

TASK

Communicate why the new approach was better than the existing one, what the model was actually doing, and what the results meant — in a way a business leader with no ML background could understand, evaluate independently, and make a decision from. The word that matters there is *evaluate*: the goal was not comprehension, it was giving them enough to disagree with me if the logic was wrong.

ACTION

Step 1 — Led with the inefficiency, not the method. The opening was the business problem in one sentence: *we've been paying for customers who would have bought anyway; this model tells us which customers only buy because of the coupon.* That framing does the persuasive work before any methodology appears, and it means everything technical that follows is answering a question they already care about rather than introducing one.

Step 2 — Gave them a conceptual map to hold. The four behavioural segments were the anchor: Persuadables who buy *because* of the offer, Sure Things who were always going to buy, Sleeping Dogs who become less likely because of it, and Lost Causes who were never going to buy. This is the single most important choice in the whole communication, for two reasons. It makes the inefficiency of the existing approach self-evident without any statistics — once "Sure Things" exists as a named category, the audience reaches the conclusion themselves. And it gives them a structure that every subsequent technical detail can be attached to, so nothing arrives context-free.

Step 3 — Pre-empted the obvious wrong alternative in their language. The natural business intuition is "target the people most likely to buy." I addressed that explicitly and simply: the people most likely to buy are the people who need the coupon least. Killing the intuitive alternative in plain language, early, is what stopped the discussion looping back to it in later rounds.

Step 4 — Translated every output into a business outcome. No model vocabulary survived into the results. "Top three deciles have the highest CATE" became *if we target only the top 30% of users, we get 80% of the new customers at 50% of the cost.* Validation became a plain statement that the model's ranking was checked against what actually happened in a real experiment. CAC and RPC — metrics they already used daily — carried the numbers: CAC of ~₹200 versus ~₹300, RPC of ~4 versus ~2.

Step 5 — Made the proof point an experiment, not a diagnostic. For a business audience, the credibility of an AUC figure is low, because they have no reference for what a good one looks like. The confirmatory A/B was the proof: a 90/10 Test/Control split on the

top three deciles and 10/90 on the bottom seven, run on the actual NAC base, showing the top-3 ATE was significantly higher and RPC/CAC genuinely optimised. That's an artefact they could evaluate on their own terms.

Step 6 — Gave the feature story, because intuition is a form of evidence for a business audience. The top-decile users had recently visited, done more add-to-carts and banner clicks, more search PPVs, added addresses, and had lower bounce rates. Engagement, not demographics, predicted being a Persuadable. That is a behaviourally sensible story — the person already circling the purchase is the one a coupon tips over — and its function was to make the model's answer feel *right* as well as proven.

Step 7 — Closed the remaining fear rather than leaving it unspoken. The unstated worry with any incentive-led acquisition is that the customers are low quality. I brought the answer unprompted: coupon availers' long-term spend and M1/M2/M6 repeat were not materially different from non-availers. Addressing an objection before it's raised is more persuasive than answering it well afterwards.

RESULT

- **Leadership approved production deployment.**
- **The model ran for five to six months without needing re-explanation** — which is the practical test of whether an explanation actually landed.
- **The four-segment framework became shared vocabulary** the team used in subsequent targeting discussions. The communication architecture outlasted the specific project.
- The same decile framework was then reused for offer-construct experimentation, which required no new conceptual explanation because the frame was already established.

KEY LEARNING

Complex ideas land with non-technical audiences when you establish the right conceptual frame first — after that, every technical detail becomes an explanation of something the audience already understands rather than a new thing to absorb. The more useful lesson is about the goal: I wasn't trying to be understood, I was trying to make the logic auditable, so they could reach the conclusion independently and disagree with me if it was wrong. And the test of a good simplification is durability — the four-segment frame is still the vocabulary, which means it simplified the problem rather than just the presentation.

What Interviewers Will Probe

- **Explain uplift modelling to me as if I'm the business stakeholder.** We give coupons to a big group of people and count how many buy. The problem is that some of them were always going to buy — the coupon just made their purchase cheaper for us. Uplift finds the people whose decision actually changes because of the coupon. It's the difference between "who buys" and "who buys because of us."

- **How do you avoid over-simplifying to the point of being misleading?** I keep the limitation attached to the simplification. When I said the top 30% gives 80% of customers at 50% of cost, I also said explicitly that we lose about 20% of incremental NC and named it as the trade-off being accepted. Simplification becomes misleading when you drop the cost, not when you drop the vocabulary.
- **What did you do when someone still didn't follow?** Went back to the segment frame rather than re-explaining the mechanics. Almost every confusion traced back to conflating "likely to buy" with "influenced by the coupon," so returning to Sure Things versus Persuadables resolved it. Having one anchor to return to is worth more than having three ways to explain the same thing.
- **How did you handle someone who was sceptical of the model specifically?** Agreed with them. A model is a reason to run an experiment, not a reason to move budget — which is exactly why the confirmatory A/B existed. I never asked anyone to trust the model; I asked them to act on an experimental result the model had helped design.
- **Was there anything you chose not to explain at all?** Yes — the S-Learner double-scoring mechanic, and why we chose it over a T-Learner. It was genuinely irrelevant to their decision, and including it would have signalled that understanding the architecture was a prerequisite for approving the recommendation, which would have made the decision feel riskier than it was. I had it ready if asked.
- **How is this Simplify rather than just good presentation?** Because the frame outlived the project. If it were presentation skill, the explanation would have been needed again every time the topic came up. Instead the team adopted Persuadables and Sure Things as working vocabulary, and the next wave of offer-construct work needed no conceptual setup at all. That's complexity removed from the organisation, not just from one meeting.

Why This Story Screams Simplify

- ✔ Reduced a statistically dense method to one frame a non-technical audience could reason with independently
- ✔ Led with the business inefficiency so the technical content answered an existing question
- ✔ Translated every output into metrics the audience already used daily — CAC, RPC, cost, customers
- ✔ Deliberately omitted architecture detail that added complexity without informing the decision
- ✔ Chose the proof point the audience could evaluate on its own terms — an experiment, not a diagnostic
- ✔ The simplification was durable: the four-segment frame became the team's standing vocabulary

---

Q17. Describe a situation where miscommunication caused an issue and how you resolved it.

Coupang Leadership Principle: THINK SYSTEMATICALLY *(secondary: Demand Excellence)*

The principle in plain English: Coupang's Think Systematically asks for scalable processes with prompt feedback mechanisms, and for measures that not only fix defects but prevent them recurring. Applied to a miscommunication question, the interviewer is really asking two things: did you resolve it cleanly, and did you make it structurally impossible next time? A story that ends with "we talked it through and aligned" answers only the first half. The second half — the mechanism — is what earns the principle.

How interviewers may frame this question:

- Describe a situation where miscommunication caused an issue and how you resolved it.
- Tell me about a time two teams were working off different assumptions.
- Give me an example of a costly misunderstanding.
- How do you make sure everyone means the same thing by the same metric?
- Tell me about a time you caught a problem during preparation rather than in production.
- Describe a process you put in place after something went wrong.
- Tell me about a time your definition of a metric differed from someone else's.

Story: BBD NC Reporting — a target definition mismatch caught in the dry run

Why this is the best fit:

This is a small-sounding problem with a large blast radius, which is exactly the kind of miscommunication worth talking about. Two teams both had a correct NC number by their own definition, and neither had any reason to suspect a mismatch — that is the most dangerous class of communication failure, because nothing looks wrong until both numbers are in the same room in front of the CEO. It also demonstrates the Think Systematically pattern cleanly: the immediate resolution was alignment, but the durable output was a definition glossary that became a standard reference for all subsequent sale event reporting, plus the recognition that the dry run itself is a communication mechanism rather than a technical rehearsal.

2-Minute Executive Summary:

"During BBD preparedness I was responsible for enabling hourly NC reporting that leadership would use to track live performance against target. The Demand Planning team had set the BBD NC target, and I built the hourly phasing and tracking dashboard on my understanding of how NC was defined in our reporting systems. Midway through the dry run it emerged that DP had used a slightly different NC definition — based on a different

event window rather than calendar day — than what our reporting pipeline captured. Both numbers were internally correct and they didn't match. If that had gone live, the target mailer and the live tracker would have shown different numbers to the CEO for eight days, which doesn't read as a definitional nuance, it reads as one of them being wrong. I root-caused it immediately by pulling both definitions side by side, isolating the exact parameter causing the divergence, and quantifying the impact. Then rather than picking a definition unilaterally, I convened a quick alignment call with DP, Analytics and Business to agree which definition governed live tracking, updated the pipeline logic, recalculated the hourly phasing, and re-ran the dry run to confirm target and tracker now agreed. Afterwards I documented the exact definition used in live BBD tracking as a glossary, which became a standard reference for subsequent sale events. BBD launched with aligned definitions and zero discrepancy across the event."

Full STAR Version

SITUATION

BBD reporting is consumed live by the CEO, CFO and SVP across eight days. My responsibility included enabling hourly NC reporting against target — which means two artefacts that have to agree: the target, phased hourly, and the live tracker showing actuals against it. The target came from Demand Planning; the tracker ran off our reporting pipeline. I built the hourly phasing and tracking dashboard based on my understanding of how "NC" was defined in our reporting systems.

Midway through the dry run, the numbers didn't reconcile. The cause was a definitional mismatch: DP had set the target using a slightly different NC definition, based on a different event window, than what our reporting pipeline captured. Neither definition was wrong. Both teams had a defensible number. And critically, neither team had any reason to suspect a problem, because each was internally consistent — this only became visible when the two artefacts were put side by side against real data.

The consequence had it gone live: the target mailer and the live tracker would have shown different NC numbers to leadership, for eight days, in the highest-visibility window of the year. In that room a definitional discrepancy doesn't read as nuance — it reads as one of the numbers being wrong, and once leadership doubts one metric they reasonably doubt the rest.

TASK

Identify exactly where the mismatch was, align on the governing definition, update the pipeline, and validate the fix — all before BBD went live, with the event calendar fixed and no room to slip.

ACTION

Step 1 — Root-caused it precisely rather than approximately. I pulled both definitions side by side and isolated the exact parameter causing the divergence — event window definition versus calendar day definition — and then quantified the impact on the numbers. That second part mattered: knowing the direction and size of the gap turned it from "the

numbers don't match" into a specific, bounded problem, which is what made the alignment conversation short.

Step 2 — Did not decide unilaterally, even though I could have. The fastest path was to pick the definition my pipeline already produced and move on. I deliberately didn't, because the target is DP's artefact and the business consumes both — a definition imposed by Analytics would have been technically consistent and organisationally fragile, and the first time someone in the room asked "is that our number or theirs," the whole thing unravels. So I convened a quick alignment call with DP, Analytics and Business to agree the correct definition for live tracking. Fast, but joint.

Step 3 — Fixed and re-validated end to end. Updated the tracking logic, recalculated the hourly phasing from the agreed definition, and re-ran the dry run to confirm the target and the tracker now aligned. Re-running the full dry run rather than spot-checking the change was the right call — a definitional change propagates into phasing, and a partially validated fix in a pre-event window is a false sense of safety.

Step 4 — Built the mechanism that made it non-recurring. After the event I documented the exact NC definition used in live BBD tracking as a glossary, which became a standard reference document for all subsequent sale event reporting. This is the actual deliverable. The alignment call fixed one instance; the glossary fixed the class of problem, because the next person building an event tracker doesn't have to reconstruct the definition or hope it matches DP's.

Step 5 — Understood what actually caught this. The dry run caught it, and that reframed how I think about dry runs. I had scheduled them as a technical rehearsal — do the mailers populate, do the targets upload, does the phasing calculate. Their higher-value function turned out to be communication verification: a dry run is the only mechanism that forces two teams' artefacts into the same view against real data early enough to matter. No amount of documentation review would have surfaced this, because both documents were correct.

RESULT

- **BBD launched with aligned definitions across all teams** and zero discrepancy between target and tracker during the event.
- **Zero leadership escalations across eight days** on data visibility or metric consistency.
- **The definition glossary became a standard reference document** for all subsequent sale event reporting.
- The dry run practice itself was reinforced as a pre-event requirement rather than a nice-to-have, alongside target validation with DP and target-upload dry runs with Engineering.

KEY LEARNING

Definition mismatches are the most common and most expensive form of miscommunication in analytics, precisely because nobody is wrong — two internally

consistent numbers can be built on incompatible assumptions and neither team has a reason to check. The fix is always the same shape: make the definition explicit, surface it early through a mechanism that forces the artefacts together, and document it afterwards so the next person inherits it. The broader realisation was that dry runs are the best communication-alignment tool a team has, not a technical rehearsal — they're the only thing that reliably catches errors where both parties are individually right.

What Interviewers Will Probe

- **Why didn't you just pick a definition and move on? It would have been faster.** Because it wouldn't have held. The target belongs to DP and the business consumes both artefacts, so an Analytics-imposed definition would have collapsed the first time anyone in the control tower asked whose number they were looking at. The alignment call cost under an hour. A contested definition mid-BBD would have cost far more.

- **Whose fault was the mismatch?** Both definitions were legitimate, so the fault sits in the process, not with either team — nobody had established which definition governed live tracking, and there was no glossary to check. If I'm assigning it anywhere, it's to me: I built the tracker on my understanding of the definition rather than confirming it against the source of the target, which is a check I should have run before building rather than during the dry run.

- **How do you prevent this class of problem at scale, beyond one glossary?** Structurally, by defining metrics once and having everything consume that definition — which is exactly what the semantic layer work in the GenAI charter does. Views mapping to facts and explores where every measure and dimension is defined a single time, consumed by dashboards, humans and agents alike. A glossary is documentation and depends on people reading it; a semantic layer removes the possibility of divergence.

- **What if you'd caught it during the event instead of the dry run?** Much worse, and the honest answer is that the recovery would have been partial. You can fix the tracker mid-event, but you can't unshow leadership two inconsistent numbers, and the hourly phasing would have needed recalculating live. This is exactly why the preparation window is where I spend disproportionate effort — the same fix costs an hour before the event and a day of credibility during it.

- **Did you change anything about how you build trackers after this?** Yes — I now reconcile against the source of the target before building rather than after. If DP owns the target, the definition question gets settled with DP at the specification stage. It's a small sequencing change that removes the entire failure mode.

- **How did you keep the alignment call from becoming a debate about whose definition was better?** By bringing the quantified impact into the room. When you can say precisely which parameter diverges and how much difference it makes, the conversation becomes "which one should govern live tracking" rather than "which

one is right" — and the first question has an obvious answer while the second doesn't.

Why This Story Screams Think Systematically

- ✔ Fixed the instance *and* the class — the definition glossary became a standard reference for all subsequent events
- ✔ Recognised the dry run as a prompt feedback mechanism, not a technical rehearsal
- ✔ Quantified the divergence rather than just identifying it, which is what made resolution fast
- ✔ Resolved jointly to make the fix durable, rather than unilaterally to make it quick
- ✔ Re-validated end to end rather than spot-checking a propagating change
- ✔ Points toward the structural answer — a semantic layer where definitions can't diverge at all

---

Q18. Give an example of a time you had to convince leadership to make a decision you believed in.

Coupang Leadership Principle: COMPANY-WIDE PERSPECTIVE *(secondary: Influence without Authority)*

The principle in plain English: Coupang wants leaders who act in the best interest of the entire company rather than their own department — considering upstream and downstream implications, and raising problems regardless of whose area they sit in. The sharpest test of this principle is whether you have ever argued *against your own team's headline metric* because the whole-company answer was different. That is uncomfortable, career-relevant, and exactly what the principle describes. The secondary principle is the mechanism: alignment driven through data and insight rather than through position.

How interviewers may frame this question:

- Give me an example of a time you had to convince leadership to make a decision you believed in.
- Tell me about a time you recommended something against your own team's interest.
- Describe a decision where the right answer for the company differed from the right answer for your function.
- Tell me about a time you changed leadership's mind.
- How do you make a case for a counterintuitive recommendation?
- Give me an example of considering the downstream implications of a decision.
- Tell me about a time you reallocated resources away from something you owned.

Story: NC Budget Allocation — recommending 60% of budget move to existing customers (ON) over new customers (NN)

Why this is the best fit:

I sat in the New Customer charter. NC was the metric my work was measured on. And the framework I built recommended moving the majority of the couponing budget away from new-to-Flipkart acquisition toward existing customers who were new to a BU or Super Category — because that produced more long-term platform value per rupee. That is the Company-wide Perspective test in its most literal form: the recommendation was worse for my charter's headline number and better for the platform. It also required looking upstream and downstream of the immediate metric — at 12-month SPC, retention, opportunity size and acquisition cost together — rather than at the NC count the team was rewarded on.

2-Minute Executive Summary:

"A prior project had already solved *which* users within a cohort should get a coupon — the uplift model identified the top three deciles. The natural next question was a level up: given a finite budget, which cohorts should the money go to at all? Couponing could be spent on two broad cohort types with different returns. NN — new to Flipkart — buys an incremental new customer. ON — old to Flipkart but new to a BU — buys an incremental transaction, which directly drives that customer's long-term retention. There was no framework to make that trade-off on a long-term-value basis. I built one, ranking four cohorts (NN, and ON split by Low/Mid/High TPC) on two equally weighted long-term measures — incremental 12-month SPC and incremental 12-month retention — with the incrementality logic made explicit, because a portion of any cohort converts organically anyway and crediting the full value would over-reward cohorts that needed no help. Then I folded in the two factors a pure value ranking misses: CAC, because high CAC means fewer incremental customers per rupee, and TG or opportunity size, because a cohort's realistic ceiling matters as much as its per-customer value. On an ₹11 Cr budget the simulator said allocate ~60% to ON Low and Mid TPC and the remainder to NN, which delivered ₹30 Cr of incremental GMV versus the baseline of spending everything on NN. That was counterintuitive and it was against my own charter's headline metric, so leadership was sceptical. I made the incrementality logic fully transparent, showed the result held across budget scenarios rather than at ₹11 Cr specifically, addressed the NC AOP concern directly, and offered a controlled test before full adoption. Leadership adopted it, the experimental campaign validated the simulation, and it became the standard allocation tool."

Full STAR Version

SITUATION

Within the NC charter, incentive budget is finite and how it is allocated determines how much long-term value the platform gets back. The uplift model had already answered the within-cohort question — which users to target. The unanswered question was one level up: given a fixed budget, which cohorts deserve the money in the first place?

Two broad cohort types were available, with structurally different returns:

- **NN (New to Flipkart):** spending acquires an incremental new customer to the platform.
- **ON (Old to Flipkart, New to a BU):** spending drives an incremental transaction, which directly impacts that customer's long-term retention. Categories ran targeted couponing to acquire ON customers.

The core question was concrete — if we have ₹11 Cr to spend, how much goes to NN versus ON, and within ON, across Low, Mid and High TPC? There was no framework to make that trade-off on a long-term-value basis, so decisions were being made without a consistent way to compare cohorts. And the default instinct across the business was that new customer acquisition should always take priority — which happened to be the instinct that favoured my own charter's metric.

TASK

Build a framework and simulator prioritising cohorts for a finite couponing budget, ranked by long-term platform goodness while accounting for opportunity size and acquisition cost. Then convince leadership to adopt an allocation that defied their intuition and reallocated the majority of budget away from the metric the team was primarily measured on.

ACTION

Step 1 — Defined long-term goodness on an incremental basis. I ranked the four cohorts on two equally weighted long-term measures: incremental 12-month SPC and incremental 12-month retention. The incrementality logic was the part I made most explicit, because it's where the intuition goes wrong. Take a 30 Mn NAC base: if you coupon them today they give ₹x as next-12-month SPC, but even without couponing a portion converts organically and gives ₹y. The incremental value of acquiring them today is $x - y$, not the full x. The same logic applies to ON cohorts. Using absolute rather than incremental SPC would have over-credited exactly the cohorts that needed the least help — which is the same error the uplift work had already exposed one level down.

Step 2 — Brought in the two factors a value ranking misses. Ranking on long-term goodness alone is misleading, because a cohort with high per-customer value but high CAC yields fewer incremental customers per rupee than a cohort with slightly lower value and much lower CAC that can be acquired at far greater volume. So the framework folded in **CAC** — a critical parameter, since high CAC means fewer incremental customers per rupee and can outweigh a high per-customer long-term value — and **TG / opportunity size**, the number of NN or ON customers acquirable by running a coupon over a fixed window, which captures the real ceiling on each cohort's contribution. For each cohort we estimated TG size, base conversion, approximate incrementality and CAC from prior experiments, giving budget required per cohort = TG size × base conversion × incrementality × CAC.

Step 3 — Built the allocation simulator rather than a recommendation. A scenario simulator taking overall budget as input and allocating across cohorts by rank, weighing TG size, CAC, incrementality and long-term 12-month SPC plus retention together —

computing incremental customers and incremental GMV for each allocation, and running multiple scenarios so leadership could compare strategies against a baseline of spending the entire budget on NN. Building a tool rather than delivering a number was deliberate: it meant leadership could interrogate the logic themselves instead of accepting my conclusion.

Step 4 — Surfaced a structural insight the framework exposed. Using TG size as an explicit factor revealed something nobody had articulated: BU-level ON has limited opportunity, because only a limited base is genuinely old-to-Flipkart-but-new-to-that-BU. The real opportunity sits in **Super Category ON**, where the addressable base is far larger. That's a finding about where to *look* for the next tranche of budget, not just how to split the current one.

Step 5 — Made the incrementality logic transparent rather than asking for trust. When leadership pushed back, I walked through exactly why ON generated more incremental long-term revenue per rupee: lower CAC, higher opportunity size at SC level, and meaningful retention impact persisting beyond 12 months. Every step of that chain was checkable. If they could reproduce the reasoning, they didn't need to trust me.

Step 6 — Showed the result held across scenarios, not just at ₹11 Cr. I ran the simulator under multiple budget assumptions to demonstrate the principle was robust rather than an artefact of one specific number. A conclusion that only holds at one budget level is a coincidence; one that holds across the range is a principle.

Step 7 — Addressed the NC AOP concern head-on. The real objection was not analytical, it was that the reallocation risked the NC number the team was accountable for. I showed the NC shortfall from reallocation was smaller than the incremental revenue gain from ON, and that the framework would still direct budget to NN in months where ON opportunity was limited — so it was a rule, not a one-way shift.

Step 8 — Offered a controlled test before full commitment. I proposed running the 60/40 allocation on one experimental campaign to validate the simulator's predictions against real data before adopting it wholesale. Same pattern as the uplift work: reduce the perceived risk of the decision rather than only arguing its upside.

RESULT

- **₹30 Cr incremental GMV** unlocked versus the baseline of spending the entire ₹11 Cr budget on NN.
- **Optimal split identified as ~60% to ON** (Low and Mid TPC), remainder to NN.
- **Leadership adopted the framework**, and the experimental campaign validated the simulation.
- **It became the standard tool** for budget allocation decisions — a consistent, defensible way to allocate finite couponing budget across cohorts on a long-term-value basis, replacing intuition-led splits.
- **Reframed the trade-off org-wide:** established that opportunity size and CAC matter as much as per-customer value, and that a large low-CAC cohort can beat a high-value low-volume one on total platform goodness.

- **Surfaced the SC-ON opportunity**, redirecting attention from a structurally capped BU-level base to a much larger addressable one.

KEY LEARNING

Convincing leadership to act against intuition requires more than a correct analysis — it requires making the logic transparent enough that they can follow it themselves. If they can reproduce the reasoning, they trust the conclusion; if they're being asked to trust a black box, they won't, and they're right not to. The harder lesson is about whose interest you argue for: this recommendation was worse for my charter's headline metric and better for the platform, and being the person who says that is the only way the analysis is worth anything. The moment an analytics function starts optimising for the metric it's measured on, it stops being useful.

What Interviewers Will Probe

- **You sat in the NC charter and recommended moving budget away from NC. How did that land?** Awkwardly at first, and it was the right thing to do. My value to the business is being the person whose recommendation doesn't bend to my own scorecard. I also made the framework symmetric — it directs budget back to NN whenever ON opportunity is thin — so it was a decision rule rather than a permanent reallocation, which made it much easier to accept.

- **Why is incremental SPC the right measure rather than absolute SPC?** Because a portion of any cohort converts organically without any coupon. If you credit the full 12-month SPC to the coupon, you systematically over-reward the cohorts that needed the least help — the same error as targeting Sure Things, one level up. The incremental figure is $x - y$, where y is what they'd have delivered anyway.

- **How confident are you in the ₹30 Cr number?** Confident in the direction and the ranking; less so in the point estimate. Every input — TG size, base conversion, incrementality, CAC — came from prior experiments and carries error, which is why the deliverable was a simulator with scenario comparison rather than a single figure, and why I proposed a controlled test before full adoption. The robust claim is that ON Low/Mid beats an all-NN allocation across the budget range, not that it's worth exactly ₹30 Cr.

- **What if the experimental campaign had contradicted the simulator?** Then the framework's inputs were wrong and I'd want to know which — most likely CAC or incrementality for a specific cohort, since those were the least directly measured. That's recoverable and informative. What wouldn't have been recoverable is reallocating the full budget on a simulation and finding out at the end of the year.

- **How did you handle the concern that this deprioritised new customer acquisition strategically?** By separating the metric from the strategy. Nobody was arguing NC growth doesn't matter — the argument was about the marginal rupee. And the framework surfaced the SC-ON opportunity, which is a growth story of its

own. It reframed the conversation from "NN versus ON" to "where is the largest pool of incremental long-term value we haven't touched."

- **What's the biggest lesson?** That the highest-leverage question is usually one level above the one you were asked. I was asked how to coupon a cohort better. The question worth answering was which cohorts deserve budget at all — and that one was only visible because the first question had been solved.

Why This Story Screams Company-wide Perspective

- ✔ Recommended against the headline metric of my own charter because the platform answer differed
- ✔ Optimised for total long-term platform value — 12M SPC and retention — not the department's KPI
- ✔ Considered upstream and downstream: incrementality, CAC, opportunity ceiling and retention together
- ✔ Surfaced a structural insight (SC-ON versus BU-ON) that belonged to another team's territory
- ✔ Built a symmetric decision rule rather than a self-serving one-way shift
- ✔ Left the org a reusable tool that makes the whole-company trade-off explicit every planning cycle

---

SECTION 6: VALUES FIT

Q19. Tell me about a time when you contributed to a culture of inclusivity or psychological safety.

Coupang Leadership Principle: HIRE AND DEVELOP THE BEST *(secondary: Demand Excellence)*

The principle in plain English: Coupang's phrasing is about raising the team's performance level with every hire and promotion, recognising exceptional talent, investing resources to develop people's full potential, and empowering them to pursue growth. So a psychological-safety question at Coupang is not primarily a culture question — it's a talent-development question. The interviewer wants evidence that people got better because of you, and specifically that you invested in capability rather than just being pleasant to work with. The strongest answers show safety created *in service of* performance, not as an alternative to it.

How interviewers may frame this question:

- Tell me about a time you contributed to a culture of inclusivity or psychological safety.
- How do you develop the people on your team?
- Tell me about someone you've mentored and what changed for them.

- Describe how you build a team that can operate under pressure.
- Give me an example of empowering someone to do work above their level.
- How do you handle mistakes made by junior team members?
- Tell me about a time you raised the performance level of a team.

Story: BBD Control Tower and team onboarding — building capability under maximum pressure

Why this is the best fit:

BBD is the least psychologically safe environment Flipkart produces — real-time leadership visibility, rapid RCA turnaround, and the constant awareness that the CEO and CFO are watching every number, with mistakes visible to the most senior people in the company. For several team members it was their first BBD. The reason this is a Hire and Develop the Best story rather than a niceness story is that the safety was built specifically to produce performance: the control tower's quality depended on the whole team contributing, not just senior members, so distributing genuine ownership to junior analysts wasn't generosity, it was the operating design. And it worked in both directions — zero escalations across eight days, and analysts who were visibly more confident afterwards.

2-Minute Executive Summary:

"During BBD, junior team members were working in an extremely high-pressure environment — real-time leadership visibility, rapid RCA turnaround, and the knowledge that the CEO and CFO were reading every number. For several of them it was their first BBD, and the environment is not naturally safe: mistakes there feel very public. My problem wasn't cultural, it was operational — the control tower's quality depended on the whole team contributing effectively, and people who are afraid of being wrong produce vague, hedged analysis. So I built the safety deliberately. I onboarded new joiners on tools, systems and the nature of the work *before* BBD pressure hit rather than during it. During dry runs I made the norm explicit: the purpose of a dry run is to find mistakes, so every error caught in one was a good outcome, not a criticism — which meant people surfaced problems rather than hiding them at the point where hiding them was still cheap. I distributed real ownership rather than centralising analysis with senior members, giving junior analysts specific accountable sections of the tracker with support attached. And during the live event, when a junior member made an error in a mailer, I corrected it quickly and privately and then helped them build the checklist to prevent recurrence — no blame, just process. The team delivered eight days with zero leadership escalations, and those analysts were visibly more confident in their work afterwards. The culture of ownership without blame carried into later projects."

Full STAR Version

SITUATION

BBD is eight continuous days of NRT reporting consumed by the CEO, CFO and SVP, with rapid RCA expected whenever an L0 metric deviates from target. I was managing

control-tower visibility for BBD metrics deep-dive across Analytics, coordinating 20+ teams and running a core war-room team, with 24×7 rosters across the period.

For several team members this was their first BBD. The environment is genuinely and structurally unsafe in the psychological sense: the feedback loop is hours not weeks, the audience is the most senior people in the company, and a mistake in an hourly mailer is visible immediately and to everyone. That has a specific and predictable effect on analytical quality — people who are afraid of being wrong produce work designed to be unfalsifiable rather than useful. RCAs stop at the geography instead of the input metric, caveats multiply, and nobody flags an anomaly they aren't certain about.

TASK

Create an environment where team members could contribute confidently, ask questions and raise concerns under high pressure without fear of judgement. The framing that mattered: this was not a wellbeing initiative running alongside the work, it was a precondition of the work. The control tower's quality depended on the whole team contributing effectively, not just the senior members, so if junior analysts operated defensively for eight days the output would degrade regardless of how many hours senior people put in.

ACTION

Step 1 — Front-loaded the capability, not just the reassurance. I onboarded new team members on tools, systems and the kind of work the team did *before* BBD pressure hit. This is the least glamorous intervention and the most effective one, because confidence under pressure is mostly a function of familiarity. Telling someone it's safe to ask questions during a live event is far weaker than making sure they already know the answer to most of them. My manager's feedback noted I was able to mentor the team very effectively during this period.

Step 2 — Made "errors are the point" an explicit norm in dry runs. During dry runs I told the team directly that the purpose was to find and fix mistakes, so every error caught in a dry run was a celebration rather than a criticism. That norm had a concrete function: it moved error discovery from the live window, where it's expensive and public, into the preparation window, where it's cheap and private. It also means people surface uncertainty rather than sitting on it — the dry run that caught the NC definition mismatch between Demand Planning and our reporting pipeline only worked because someone was willing to say the numbers didn't reconcile rather than assuming they'd misunderstood.

Step 3 — Distributed genuine ownership rather than centralising for safety. The defensive instinct in a high-stakes event is to concentrate all analysis with senior members. I deliberately didn't. Junior analysts owned specific sections of the tracker with clear accountability and clear support. That builds confidence in a way encouragement cannot — someone who owns the BU-level deep-dive section and gets it right in front of the CEO has evidence about their own capability that no amount of reassurance provides. It also created capacity: with sections genuinely owned, the war-room core team could handle

unpredictable things like quantifying GMV loss within minutes of a payment gateway failure.

Step 4 — Handled live errors as process problems, in private. When a junior team member made an error in a mailer during live BBD, I corrected it quickly and privately, and then helped them build the checklist to prevent recurrence. Three deliberate choices there. Quickly, because an uncorrected error compounds. Privately, because a public correction during a live event teaches the whole team to be defensive. And ending with a checklist rather than a conversation, because it locates the fix in the process rather than in the person — which is both truer and repeatable.

Step 5 — Held the standard at the same time. Safety without a standard produces comfortable mediocrity, which is not what this was. In the same event I gave direct feedback to a teammate whose RCAs were stopping at the geography instead of reaching the input metric, showed a concrete example of what good looked like, and offered to review his next two before they went out. The offer to review is where the two things meet — the standard didn't move, and the risk of attempting it was removed.

RESULT

- **The team delivered eight days of BBD with zero leadership escalations** on data visibility or L0 deep-dive quality.
- **Junior team members who participated were visibly more confident in their analytical work afterwards** — the capability gain outlasted the event.
- **The culture of ownership without blame carried forward into subsequent projects**, which is the only real evidence that it was a norm rather than a stance held for one week.
- Errors were caught in dry runs rather than live, including a definitional mismatch that would otherwise have produced an eight-day discrepancy between target and tracker in front of leadership.

KEY LEARNING

Psychological safety in high-pressure environments is built through actions taken before the pressure arrives, not through statements made during it. Normalising mistakes in preparation, distributing meaningful ownership, and correcting privately rather than publicly are the practical behaviours that create it. The more important reframe is that safety and standards are not a trade-off — the reason to make it safe to be wrong in a dry run is precisely so it's possible to be excellent in the live event. Fear produces hedged, unfalsifiable analysis, which is the worst possible output when leadership needs a decision in the next hour.

What Interviewers Will Probe

- **Isn't distributing ownership to junior analysts during the biggest event of the year a risk?** It's a managed risk, and the alternative is worse. Centralising everything makes senior people the bottleneck in a window with unpredictable events in it — and it guarantees the same situation next year, because nobody has

grown. I managed it by front-loading onboarding, defining section ownership precisely, and keeping review capacity available rather than by withholding the work.

- **How do you reconcile psychological safety with not tolerating mediocrity?** They're the same project. The purpose of making it safe to be wrong in preparation is to make it possible to be excellent in the event. What I didn't do was lower the standard — in the same BBD I told a teammate directly that landing on a city rather than an input metric wasn't the bar. Safety is about the cost of attempting the standard, not about the standard.

- **What did you do about someone who wasn't improving?** The specific case I had was someone whose RCAs were too high-level and who had also misread a couple of questions. The response was specific feedback, a worked example, and an offer to review the next two before they went out. Quality improved by the next mailer cycle. Had it not, the fallback was reassigning the section — which is a fair outcome mid-event, and not the same as writing someone off.

- **How do you know the confidence gain was real rather than you being generous about it?** Because it showed up in later work — those analysts took on more independent ownership afterwards, and the norm of surfacing problems early rather than hiding them persisted into subsequent projects. Also, one of them told me directly that the specific worked example was what made feedback land, which is the kind of thing people only say when they felt safe enough to receive it.

- **What would you do differently?** Build the checklists before the event rather than after the first error. The prevention checklist we built with the junior analyst mid-BBD was genuinely useful and it existed because something had already gone wrong. Most of what was in it was predictable, and it should have been part of onboarding.

- **How does this scale beyond a team you personally know?** Through artefacts rather than presence. The onboarding, the dry-run norm, the section-ownership structure and the checklists are all transferable — they're how you hold a culture without standing in the room. That's also why the definition glossary and the deep-dive template mattered: a well-specified standard is what lets someone junior be confident without needing me to validate them.

Why This Story Screams Hire and Develop the Best

- ✔ Invested in capability before the pressure arrived, rather than reassuring people during it
- ✔ Empowered junior analysts with genuine, accountable ownership at the highest-stakes moment
- ✔ Raised the team's performance level — analysts were measurably more independent afterwards

- ✔ Corrected privately and built process, locating fixes in systems rather than in people
- ✔ Held the standard simultaneously, with direct feedback and a worked example of the bar
- ✔ The norms outlived the event and carried into later projects

---

Q20. Give an example of when you challenged the status quo in a respectful and constructive way.

Coupang Leadership Principle: DISAGREE AND COMMIT *(secondary: Learn Voraciously)*

The principle in plain English: Coupang treats constructive confrontation as essential to good decisions — leaders challenge openly when they disagree, and explicitly do not compromise for social cohesion. The second half is equally binding: once the decision is made, commit fully, and afterwards proponents and opponents should be indistinguishable. So this question is looking for a real challenge to a real belief, made openly rather than diplomatically avoided, and resolved by evidence rather than by seniority or compromise. A story where you softened the challenge to keep everyone comfortable fails the principle even if the outcome was fine.

How interviewers may frame this question:

- Give me an example of when you challenged the status quo in a respectful and constructive way.
- Tell me about a widely held belief you proved wrong.
- Describe a time you disagreed with your leadership.
- How do you challenge an assumption that everyone else accepts?
- Tell me about a time you were the only person who thought something was wrong.
- Give me an example of a strongly held view you changed — yours or someone else's.
- What do you do when you disagree with a decision that's already been made?

Story: Intelligent Couponing — challenging the "deeper discount = lower RPC" assumption

Why this is the best fit:

This was a belief held firmly across leadership, never formally tested, and actively shaping offer-construct decisions across every NC couponing campaign. That combination — confident, untested, and consequential — is the ideal target for a status-quo challenge. What makes it constructive rather than confrontational is the method: I didn't argue the belief was wrong, I ran the experiment that would settle it, decomposed the metric so the *mechanism* was visible rather than just the outcome, and presented the result as a question rather than a verdict. And the finding contradicted the intuition, which meant the team's shared mental model got better rather than just one decision changing.

2-Minute Executive Summary:

“There was a firmly held belief across leadership that increasing discount depth would always reduce RPC — Revenue Per Customer — because higher burn would outpace the incremental new customers gained. It had never been formally tested; it was intuition plus early-stage observation. And it was consequential, because it was shaping offer construct decisions across every NC couponing campaign. Rather than argue the belief, I ran the test that would settle it. Using the uplift model’s top three deciles as the target group, I designed A/B experiments exposing them to different constructs — 10% off up to ₹50 versus 30% off up to ₹50, and later a ₹9 store variant. The key methodological choice was decomposing rather than comparing headline numbers: I broke RPC into incremental revenue over incremental burn, and CAC into incremental cost over incremental burn, for each construct. That made the mechanism visible instead of just the outcome. And I presented it as a question — the data suggests the relationship might be non-linear, here’s what we found — which invited leadership into the analysis rather than challenging them directly. The result surprised everyone including me: the 30% off construct had *higher* RPC and *lower* CAC, because the stronger offer perception drove a spike in incremental NC large enough to more than compensate for the higher burn. Leadership updated the mental model from ‘deeper discount equals lower RPC’ to ‘RPC depends on the incremental NC spike, which depends on how customers perceive offer value,’ and future construct decisions were made through that decomposed framework rather than through intuition.”

Full STAR Version

SITUATION

Across leadership there was a deeply held belief that increasing discount depth would always reduce RPC, on the reasoning that higher burn would outpace the incremental NC gained. It had the two properties that make an assumption dangerous: it was never formally tested — it rested on intuition and early-stage observations — and it was actively in use, influencing offer construct decisions across all NC couponing campaigns. Nobody was being careless; the logic is superficially sound, and in the absence of a test, a sensible prior becomes a settled fact.

I was in a position to test it because the uplift work had already built the necessary infrastructure. Once the right target group was identified — the top three uplift deciles — the natural next question was what the right *offer* was for that group. Which meant the experiment that would settle the discount-depth question was a natural extension of work already underway, not a separate crusade.

TASK

Challenge a long-held leadership assumption using evidence rather than assertion, and update the organisation’s mental model of how discount depth and RPC actually relate — without making it a confrontation, since the belief was widely and sincerely held.

ACTION

Step 1 — Chose to test rather than to argue. This was the whole approach. Arguing against a confident, untested belief puts you in a debate you can’t win, because both sides

are reasoning from priors. Running the experiment moves the disagreement to a place where it can be resolved. And crucially, I committed to the result in advance in both directions — if the intuition had held, that would have been a useful finding too, and saying so up front is what made it a test rather than an attempt to prove a point.

Step 2 — Designed the experiment on the right population. Using the uplift score deciles, I designed multiple A/B experiments on the top deciles — creating T1/T2/T3 and C1/C2/C3 cells — to test offer constructs: 10% off up to ₹50 versus 30% off up to ₹50, and separately 10% off up to ₹100 and a ₹9 store construct with limited selection to maximise NC conversion. Running this on the top uplift deciles rather than the whole base mattered: it isolated the construct effect on the population where the offer actually changes behaviour, instead of averaging it across Sure Things whose response tells you nothing about offer design.

Step 3 — Decomposed the metrics instead of comparing headlines. The methodologically important choice. Rather than comparing RPC and CAC across constructs at the headline level, I broke each into components: **RPC = incremental revenue / incremental burn** and **CAC = incremental cost / incremental burn**. That made the mechanism visible rather than just the outcome — you can see whether a construct wins because it lifts incremental revenue or because it suppresses burn, which is what makes the result generalisable to the next construct decision rather than just a fact about these two offers.

Step 4 — Presented it as a question, not a verdict. The framing was: the data suggests the relationship might be non-linear — here's what we found. That was deliberate and it was also honest, because a single experiment on one target population genuinely does not establish a universal law. Framing it as an invitation into the analysis rather than a correction of their belief meant leadership engaged with the mechanism rather than defending the prior.

Step 5 — Reported a result that surprised me too. The 30% off construct had **higher RPC and lower CAC** than the 10% off construct. The mechanism: the stronger offer perception drove a spike in incremental NC large enough to more than compensate for the higher burn per customer. The intuition was wrong in this case, and being explicit that it was wrong *in this case* — rather than claiming discount depth always helps — is what kept the finding credible.

Step 6 — Left behind a framework rather than a fact. The durable output was not "use 30% off." It was the decomposed RPC and CAC framework, which meant future construct decisions could be evaluated on the same basis rather than requiring a new experiment or a new intuition each time. That converted a single challenge to the status quo into a permanent change in how the decision was made.

RESULT

- **Leadership updated the mental model** from "deeper discount = lower RPC" to "RPC depends on the incremental NC spike, which depends on customer perception of offer value."

- **Future offer construct decisions were made through the decomposed framework** rather than through intuition.
- The decile-based experimentation structure became the foundation for an ongoing second wave of offer-construct testing, including the ₹9 store construct.
- The finding directly improved the efficiency of the intelligent couponing programme, which was already running at ~₹4 Cr monthly versus ~₹9 Cr, by ensuring the freed budget was deployed through the best-performing construct.

KEY LEARNING

The most respectful way to challenge a long-held assumption is to run the test that settles it, and then present the result with intellectual honesty regardless of which direction it goes. Data-driven challenges are inherently constructive because they improve the team's shared understanding rather than establishing who was right. The subtler lesson is that decomposition is what makes a challenge durable: comparing headline RPC would have produced a fact about two offers, while decomposing it into incremental revenue and burn produced a framework the org could apply to every subsequent decision. And committing to the result in both directions before running it is what separates a test from an argument with extra steps.

What Interviewers Will Probe

- **What if the experiment had confirmed leadership's belief?** Then I'd have reported that, and it would have been genuinely valuable — an untested assumption driving every construct decision would have become a tested one. I said as much before running it, which is what made it a test. If I'd only been willing to report one outcome, I wasn't challenging the status quo, I was campaigning.

- **How do you challenge a leadership belief without damaging the relationship?** By making the disagreement about a question rather than about a judgement. I never said the belief was wrong; I said it was untestable in its current form and worth testing. And the framing at presentation — "the data suggests this might be non-linear" — is accurate, not just diplomatic, because one experiment on one population doesn't establish a law.

- **Isn't one experiment thin evidence to overturn a standing assumption?** Yes, which is why the output was a framework rather than a rule. The claim I stood behind was narrow: on this target population, at these constructs, deeper discount produced higher RPC through an incremental-NC spike. The general claim — that the relationship is non-linear and depends on offer perception — is what the decomposition supports, and it's deliberately weaker than "deeper is better."

- **Why decompose RPC rather than just compare it?** Because the headline comparison tells you which offer won and nothing about why, and "why" is what transfers. Splitting RPC into incremental revenue over incremental burn showed the win came from a volume spike rather than from burn efficiency — so the next construct decision can ask "will this construct produce a perception-driven volume spike?" rather than guessing again.

- **How did you know the incremental NC spike was from offer perception and not something else?** It's the inference I'm least certain about and I'd flag that honestly. The evidence is that the constructs differed in perceived value more than in actual expected value — 30% off up to ₹50 versus 10% off up to ₹50 caps at the same ₹50 — and the volume response tracked the perception rather than the cap. That's consistent with perception driving it, but I'd want a construct designed specifically to separate perceived from realised value to claim it firmly.

- **How does this fit with "commit" once a decision is made?** Both directions. Before the test, the org was committed to the existing construct approach and I worked within it while building the case to test it — I didn't run rogue campaigns. After the result, the framework became the standard and I used it rather than continuing to relitigate discount depth. The point of the challenge was to improve the decision, not to keep the argument open.

Why This Story Screams Disagree and Commit

- ✔ Challenged a firmly held leadership belief openly rather than working around it
- ✔ Refused to compromise for social cohesion — tested a live assumption that was shaping real budget decisions
- ✔ Pre-committed to the result in both directions, which is what made it a test rather than advocacy
- ✔ Reported a finding that surprised the challenger too, and bounded the claim honestly
- ✔ Framed as a question to invite engagement rather than to force a concession
- ✔ Committed to the new framework afterwards rather than keeping the debate alive

---

Q21. Tell me about a time you influenced product direction.

Coupang Leadership Principle: WOW THE CUSTOMER *(secondary: Dive Deep)*

The principle in plain English: Coupang's first principle is that the company exists to transform customers' lives for the better, and the customer is the beginning and the end of every decision. For an analytics candidate this is the principle most likely to be under-served, because analytics work is usually framed in internal metrics. What the interviewer wants is evidence that you traced a business metric back to an actual customer experience and changed the product because of what the customer was going through — not just because a number moved.

How interviewers may frame this question:

- Tell me about a time you influenced product direction.
- Give me an example of when your analysis changed what got built.
- Tell me about a time you advocated for the customer against a business preference.
- Describe how you've used customer behaviour to shape a roadmap.

- Tell me about a time the data told you something about customers that surprised the business.
- How do you make sure your analysis reflects real customer experience?
- Give me an example of when you looked past a metric to what customers were actually experiencing.

Story: Low-TPC Retention Driver Model — reframing a discount problem as a product discovery and trust problem

Why this is the best fit:

The business had diagnosed Low-TPC churn as a pricing and affordability problem and was responding by asking for larger coupon budgets and discount-led features. The model said otherwise: retention was driven by early feature engagement and community contribution, not discount depth. But the reason this is a Wow the Customer story rather than a modelling story is the third layer — I cold-called 500 churned Low-TPC customers, and what came back was a description of a customer experience, not a metric. New users felt overwhelmed by the app interface and lacked trust in non-branded items. That's a customer problem with a product answer, and it's what moved the roadmap from discounts to guided discovery and trust signals.

2-Minute Executive Summary:

"Flipkart's Annual Active Customer growth was starting to saturate organically, which made Low-TPC users — customers with one to three transactions a year, over 40% of the active base — the single most important lever for hitting growth targets. The problem was that around 70% of them churned within six to twelve months. The prevailing assumption across Product and Marketing was that this was a pricing and affordability problem, which led to requests for larger coupon budgets and discount-based features. I built an XGBoost classification model on 50+ behavioural variables captured in a customer's first 90 days, across seven themes — selection, pricing, experience, diversity, engagement, feedback and social awareness. Feature importance and SHAP surfaced three dominant drivers, and pricing depth was not among the leading ones: super-high engagement, quality at low ASP, and community contribution through rating and reviewing purchases. I then ran EDA to confirm the mechanism behind each driver rather than the correlation, and went one level further by sharing 500 churned Low-TPC accounts with the business team for cold-calling. The qualitative insight triangulated exactly with the model and gave it a human shape: new users felt overwhelmed by the app interface and didn't trust non-branded items. That let me present the comparison that changed the conversation — coupon availers churned at almost the same rate as non-availers once the incentive stopped, whereas users who interacted with Flipkart Assured products or left ratings had roughly double the twelve-month retention. The Product team pivoted the roadmap toward guided discovery, onboarding and post-purchase rating prompts, and within three months we saw a 100 bps improvement in Low-TPC monthly repeat."

Full STAR Version

SITUATION

AAC and MAC comprise New customers plus Low TPC (1–3 transactions in 12 months), Mid TPC (4–7) and High TPC (8+). Low TPC is a huge base — over 40% of the total — and retains very poorly, with roughly 70% churning in the long term, meaning no transaction in the next six to twelve months. Mid and High TPC are entrenched users who understand the platform's value proposition; Low TPC are early in their journey and haven't formed it yet.

With aspirational top-down AAC and MAC targets and new-customer growth beginning to saturate, solving Low-TPC retention became the most important available lever. And the prevailing hypothesis was that it was a pricing problem. That belief had real consequences: Product and Marketing were requesting larger coupon budgets and prioritising discount-based features, which meant the roadmap was being allocated against a diagnosis nobody had validated.

TASK

Identify the behavioural drivers of Low-TPC long-term retention — repeat in months M6 to M12 — determine whether product and experience rather than discounts could address the churn, and translate the findings into targeted, measurable actions the Business and Product teams could own.

ACTION

Step 1 — Built the driver model on first-principles behavioural themes, not convenient variables. An XGBoost classification model with the target being whether an M0-acquired Low-TPC customer repeated in months M6–M12. I captured their first-90-days behaviour using 50+ variables across seven deliberately designed themes:

- **Selection:** category affinity, quality of selection (Flipkart Assured, branded, highly-rated products, selection freshness).
- **Pricing:** discount over BAU, offers applied on selection.
- **Experience:** RTO share, RVP share, customer-support contacts, speed metrics, breach metrics.
- **Diversity:** number of Super Categories purchased in the first 90 days.
- **Engagement:** number of visits, number of PPVs, finding methods (search / merch / reco), PPVs by category.
- **Feedback / social awareness:** percentage of products rated or reviewed.

The theme design is the part that made the answer trustworthy. Pricing was included as a full theme rather than omitted, so when it didn't dominate, that was a finding rather than an artefact of what I chose to measure.

Step 2 — Identified the drivers and read the SHAP values. Feature importance and SHAP surfaced three themes as most important: **super-high engagement** (regular visits and interaction with key app features), **quality at low ASP** (buying and browsing more Flipkart Assured, highly-rated products at discounted low prices), and **community contribution** (rating and reviewing purchases). Notably, quality at low ASP is not the same

as discount depth — it is about *what* customers were guided toward at a given price point, which is a product-surfacing problem rather than a burn problem.

Step 3 — Confirmed the mechanism rather than settling for the correlation. EDA on each driver. Churners' engagement dropped disproportionately week over week compared with retainers. At the *same* ASP bucket, retainers bought more branded, Flipkart Assured and 4+-rated products than churners — which is the finding that separates quality from price, because it holds price constant. And retention probability rose with the percentage of units rated, confirming that soliciting ratings drives engagement and retention rather than merely correlating with engaged users.

Step 4 — Went one level deeper to the actual customer. I shared 500 Low-TPC accounts — active browsers who churned — with the business team for cold-calling, to understand their pain points and churn reasons in their own words. This is the step that turned an analysis into a customer story. The qualitative insight triangulated exactly with the model's drivers, and gave them a human shape: new users felt overwhelmed by the app interface, and they lacked trust in non-branded items. Those two sentences are the model's top drivers restated as lived experience — low engagement because the app was hard to navigate, and quality-at-low-ASP because they couldn't tell what was trustworthy.

Step 5 — Presented the comparison that changed the conversation. Rather than arguing against discounts abstractly, I put a side-by-side in front of the Product team: coupon availers churned at almost the same rate as non-availers once the incentive stopped, whereas users who interacted with Flipkart Assured products or left ratings had roughly double the twelve-month retention. That single comparison reframed the problem from "we need more burn" to "we need customers to find products they trust, quickly."

Step 6 — Translated each driver into a specific product intervention and an owned metric. Not recommendations, but action items with targets:

- **Engagement:** improve adoption of essential features (CLP, Search, Flipkart Assured, Ratings, Reviews) via guided video tutorials and a simpler app; incentivise weekly visits through gamification and engagement awareness. Key metric: **WAU of the Low-TPC base**, with an aggressive target taken.
- **Quality at low ASP:** ensure platform-guided discovery of high-quality products across Homepage, Merch, CRM, Search and Reco for the first three transactions. Key metrics: **4+-rated product share of units** and **Flipkart Assured unit share** for Low TPC, aggressive targets taken.
- **Community contribution:** solicit Low TPC to rate previous purchases, including gamified post-purchase prompts. Key metric: **percentage of units rated by Low TPC**, aggressive target taken.

RESULT

- **The Product team pivoted the roadmap** toward guided discovery and onboarding features, on the recognition that habit formation drove LTV better than unsustainable coupon burn.

- **100 bps improvement in Low-TPC monthly repeat (M1R)** within three months of deploying the changes.
- **50+ behavioural variables across seven themes** modelled on first-90-days behaviour, with three retention levers isolated.
- Each lever landed as an **owned Business action item with an aggressive key-metric target** — Low-TPC WAU, 4+-rated and FA unit share, percentage of units rated — turning analysis into an accountable retention plan rather than a document.
- The work directly addressed the saturation of new-customer growth by attacking Low-TPC churn, which was the higher-leverage lever.

KEY LEARNING

Influencing product direction requires translating analytical outputs into intuitive customer behaviours. Combining the XGBoost model with 500 cold calls is what let me reframe a "discount problem" into a "product discovery and trust problem" — and the second framing is one product managers were excited to solve, because it's a problem they can build for. The deeper lesson is about triangulation: the model told me what drove retention, the EDA told me the mechanism, and the cold calls told me what the customer was actually experiencing. Each layer made the action items more specific, and the 100 bps came from that specificity, not from the model alone.

What Interviewers Will Probe

- **How did you get the Product team to accept your findings when they preferred giving coupons?** I showed them the ceiling of their own approach rather than arguing against it. Coupons drove the immediate next transaction but didn't build platform habituation — availers churned at almost the same rate as non-availers once the incentive stopped. Set against users who engaged with Flipkart Assured or left ratings having roughly double the twelve-month retention, the ROI case for UX investment over marketing burn made itself.

- **What if the cold-calling feedback had contradicted the model?** It would have meant a feature in the model was capturing a proxy rather than a root cause, which is a common failure in behavioural models. I'd have paused, used the calls to identify the actual friction point, re-engineered features to isolate it, and re-run before making recommendations. Contradiction between quantitative and qualitative isn't noise — it's usually the model measuring a symptom.

- **How did you separate "quality at low ASP" from just "discounts"?** By holding price constant. The EDA compared retainers and churners *within the same ASP bucket* and found retainers bought more branded, Flipkart Assured and 4+-rated products. Same price, different quality of selection, different retention. That's what makes it a discovery and surfacing problem rather than a pricing one.

- **Isn't ratings-to-retention just a correlation? Engaged users rate things.** That was my first concern too, and it's why I looked at it three ways: SHAP contribution independent of the engagement features, the monotonic rise in retention probability

with percentage of units rated, and the cold-call evidence that trust in product quality was a stated barrier — which makes ratings causally plausible as a trust mechanism rather than just an engagement marker. I'd still want a proper experiment on rating solicitation to claim causality firmly, and the gamified prompt intervention was effectively that test.

- **Why 500 accounts, and why cold-calling rather than a survey?** Because I wanted unprompted language. A survey constrains customers to the categories I already believe in, which is exactly the risk when you're trying to test whether your own framing is wrong. "Overwhelmed by the interface" is not an answer a survey I designed would have offered. 500 was enough to hear patterns rather than anecdotes, with the business team running the calls so they heard it directly rather than reading my summary — which mattered a lot for adoption.
- **How much of the 100 bps can you actually attribute to this work?** Not all of it, honestly — it was a three-month window with other things moving. What I'd defend is the direction and the mechanism: the metrics we targeted (Low-TPC WAU, 4+-rated and FA unit share, percentage rated) moved, and M1R moved with them. The stronger claim is about the roadmap change, which is unambiguous — features that would not otherwise have been built, got built.

Why This Story Screams Wow the Customer

- ✔ Traced an internal growth metric all the way back to what the customer was actually experiencing
- ✔ Went beyond the model to 500 cold calls — the customer's own words, unprompted
- ✔ Advocated for a product answer over the business's preferred discount answer
- ✔ Held price constant to prove the issue was trust and discoverability, not affordability
- ✔ Converted findings into product interventions that made the platform easier and more trustworthy to use
- ✔ Argued for durable customer habit over short-term burn — the customer as both beginning and end of the decision

---

Q22. Tell me about a time you improved customer experience.

Coupang Leadership Principle: WOW THE CUSTOMER *(secondary: Dive Deep)*

The principle in plain English: Same principle as Q21, and worth understanding why it recurs — for Coupang, customer obsession is not a category of work, it's the frame every decision is evaluated in. Here the specific thing the interviewer wants is the chain from an operational input to a customer-felt outcome. Delivery speed, in-stock availability and reliability are Coupang's core product; they will be listening for whether you understand that a supply-chain metric is a customer promise. Answers that stop at the operational number miss the principle even when the work was excellent.

How interviewers may frame this question:

- Tell me about a time you improved customer experience.
- Give me an example of when you connected an operational metric to a customer outcome.
- Tell me about work you've done on delivery speed or availability.
- Describe a time you improved something customers could actually feel.
- Tell me about a time an internal process was failing customers.
- How do you decide which customer problem to solve first?
- Give me an example of a customer-facing metric you moved.

Story: Cluster Fulfillment Improvement — recovering local delivery speed from an 8-week plateau

Why this is the best fit:

Cluster Fulfillment is a supply-chain metric that is really a customer promise: it's the share of orders served from the customer's own local warehouse rather than shipped across the country. At ~40% CF, 60% of demand was cascading to distant warehouses, which means longer promised delivery dates, more transit delays and worse speed perception — for real orders, every day, for eight weeks with no improvement. The work traced that customer-felt outcome back through fulfilment lanes, warehouse tiers, master selection lists and PO execution, and found two operational gaps that were degrading customer experience invisibly. And it moved the metric ~10% in two months, which converts into millions of long-lane national shipments becoming local deliveries.

2-Minute Executive Summary:

"Customer speed perception and delivery reliability depend heavily on fulfilling orders locally rather than shipping them across regions. Over an eight-week period Flipkart's Cluster Fulfillment — the share of units fulfilled from a cluster's own local warehouse — was stuck at about 40% with 0% YoY growth. That meant 60% of customer demand was cascading to distant warehouses: longer delivery promises, higher transit delay risk, degraded experience. The metric had been flat so long it was being treated as a structural limit. I broke the 60% loss into a waterfall across seller types and warehouse tiers, which converted a vague fulfilment problem into four sized, owner-mapped buckets. The largest was structural — MP marketplace volume had grown from 35% to 65% of FBF units while MP Forward CF stayed flat at about 1%, so none of that growth had translated into local fulfilment. The deepest investigation was Diamond Forward, where I found two operational flaws directly degrading customer experience: the inventory master list stocked in local Forward warehouses covered only 80% of regional customer demand, and actual placement adherence was only 60% — warehouse teams weren't stocking what they were supposed to. I designed a recency-weighted rolling 12-week demand algorithm identifying 10,000 priority FSNs to expand local coverage to 95%, and built a Looker dashboard tracking placement adherence and each loss bucket weekly, surfaced in leadership mailers to enforce accountability across four teams. Within two months overall CF improved by

about 10%, converting a large volume of long-lane national shipments into local cluster deliveries and reducing delivery speed breaches."

Full STAR Version

SITUATION

The work sat in Retail Analytics with the Retail Placement team, whose mandate is placing the right selection in the right warehouse — decisions that directly drive customer speed perception. Two structural facts matter. Selection is either **Diamond** (Flipkart-owned inventory) or **MP/Marketplace** (third-party seller inventory, with MP-FBF physically held by Flipkart). And the network is two-tier: **Forward** and **Deep** warehouses, where a warehouse plus its mapped pincodes forms a cluster. If selection isn't available in the Forward, the order cascades to the parent Deep, then to other Deeps — fulfilment is guaranteed, but at the cost of speed.

CF = units fulfilled from the cluster's own warehouse ÷ total units fulfilled in that cluster's pincodes. For eight weeks CF sat at ~40% with 0% improvement YoY and WoW. In customer terms, 60% of demand was travelling long fulfilment lanes — longer promised delivery dates, more exposure to transit delay, and worse speed perception. The metric had been flat long enough that it was being treated as a structural ceiling rather than a solvable problem.

TASK

Systematically identify why local warehouses were failing to serve local demand, size the operational gaps, and execute a strategy to maximise local delivery — improving end-customer speed perception. In practice that meant decomposing the 60% loss by seller type and warehouse tier so each bucket could be sized and attributed, prioritising the opportunities, and translating findings into owner-mapped recommendations with a governance mechanism so the improvement sustained.

ACTION

Step 1 — Built the CF Loss Waterfall to make a vague problem specific. Decomposing the 60% loss by seller type and warehouse type turned one flat number into four prioritised buckets: MP Forward 30%, Diamond Forward 15%, MP Deep 10%, Diamond Deep 5%. Every bucket has a different owner and a different fix, which is why "improve CF" had gone nowhere for eight weeks.

Step 2 — Sized the largest bucket and exposed its structural cause. MP Forward loss was 30% — half the total. The finding: despite **MP FBF unit share growing from 35% to 65%** over the previous year, **MP Forward CF stayed flat at ~1%**. All of that marketplace growth had bypassed local fulfilment entirely, meaning a rapidly growing share of customer orders was structurally guaranteed to travel a long lane. The lever was MP selection placement in Forward warehouses.

Step 3 — Diagnosed MP Deep as a distinct problem. The 10% MP Deep loss traced to four-zone seller onboarding gaps — a separate workstream, not a placement problem, and worth separating so the fix went to the right team.

Step 4 — Found the two operational flaws degrading customer experience invisibly. The Diamond Forward bucket needed the deepest work, and it split into four sub-buckets. Two of them were pure execution failures:

- **Master coverage gap.** The existing master selection — finalised with Category, IPC and Demand Planning inputs — covered only **80% of cluster demand**. So even perfect execution would leave a fifth of local customer demand structurally unservable locally.
- **Placement adherence gap.** Comparing the master list against actual Forward inventory showed only **60% of master selection was physically placed in Forwards**. Warehouse teams weren't stocking what had been agreed. This was invisible until the two datasets were compared, and it's entirely independent of selection quality — the right decisions were being made and not executed.
- **Forecasting / spike loss:** ~20% of residual loss on correctly-placed master selection came from demand spikes exceeding forecast.
- **Operational loss:** a significant remaining share from IWIT and PO issues.

Step 5 — Designed the fix for coverage. A recency-weighted rolling 12-week demand algorithm identifying a **priority 10K FSN list**, recommended to expand local master coverage from 80% to ~95%. The recency weighting is the part that matters for customers: it means the local warehouse stocks what people are buying *now*, not what they were buying two quarters ago, and it self-updates so coverage doesn't decay when attention moves on.

Step 6 — Made every finding owner-mapped. Six recommendations with named owners: MP Forward placement (largest bucket), four-zone seller onboarding, master list expansion to 95%, 100% placement adherence in Forwards, demand forecasting spike capture, and operational governance on IWIT and PO issues. None of these sat with Analytics.

Step 7 — Built governance, because a customer-facing metric needs an operating rhythm. A Looker dashboard tracking CF and each loss bucket weekly — non-master loss, forecasting error loss, placement adherence loss, operational loss — with metrics surfaced in leadership mailers. That made each of four cross-functional teams' specific loss visible weekly, including operational failures like PO delays, which is how the invisible execution gap became something someone was accountable for.

RESULT

- **CF improved ~10% within two months** — from ~40% to ~50% — after eight weeks flat with 0% YoY.
- In customer terms this converted a very large volume of long-lane national shipments into local cluster deliveries, enabling faster promised delivery speeds and significantly reducing delivery speed breaches.

- **Master FSN coverage expanded from 80% to 95%** of cluster demand via the priority 10K FSN methodology.
- **The 60% placement adherence gap was surfaced** — a pure execution problem invisible until the analysis.
- **Standardised master definition:** a recency-weighted rolling 12-week methodology replaced a more subjective process.
- **Sustained governance:** the Looker dashboard and leadership-mailer cadence turned a one-time analysis into an operating rhythm across four teams.

KEY LEARNING

Improving customer experience analytically requires connecting supply-chain inputs to customer-facing outputs. A stagnant CX metric is usually a sign that nobody has decomposed the operational chain down to the exact point where the customer's intent breaks — here it broke in two places nobody was looking, a master list covering only 80% of demand and 60% adherence to the list that existed. Breaking it down is what makes it solvable. The second lesson is that when a metric has been flat long enough, the organisation stops treating it as a problem and starts treating it as a constraint, and challenging that framing is half the work.

What Interviewers Will Probe

- **How did you balance delivery speed improvements against inventory holding cost?** Through the rolling 12-week recency-weighted model. Only high-velocity FSNs with demonstrated regional demand qualified for Forward placement, so we weren't tying up working capital in slow-moving stock to chase a speed metric. That's the specific trade-off the priority-10K methodology exists to manage.

- **What was the hardest part of driving placement adherence?** Warehouse teams were prioritising bulk inwarding over specific FSN placement, because bulk volume was what their own KPIs rewarded. Surfacing the placement adherence gap directly in leadership mailers made an invisible execution gap visible, which forced operations KPIs to align with customer fulfilment speed rather than with throughput.

- **How do you connect a 10% CF improvement to something a customer actually notices?** Through the promise. CF determines whether an order ships from the customer's own cluster warehouse or cascades to a parent Deep and then further. Each cascade step lengthens the promised delivery date and increases exposure to transit delay. A ten-point CF improvement means a substantial share of orders now get a shorter promise and fewer breaches — which is what the customer experiences, even though they never see the metric.

- **Why had nobody found the placement adherence gap before?** Because it required joining two datasets owned by different teams — the master selection list and actual Forward inventory. Each was correct in isolation. Nobody had a reason to compare them, because the assumption was that an agreed master list gets stocked.

That's the general pattern with invisible execution gaps: they live in the space between two teams' correct data.

- **Which bucket did you prioritise and why?** MP Forward got the headline recommendation because it was 30% — half the total loss. But I deliberately pushed the Diamond Forward sub-buckets first for execution, because expanding a master list and enforcing placement adherence are achievable in weeks while MP selection placement is a structural programme. That sequencing is why the ~10% landed in two months rather than two quarters.
- **How did you stop it regressing after the two months?** Two mechanisms. The Looker dashboard with weekly loss-bucket tracking and leadership-mailer visibility, so each team's slice stayed accountable. And the methodology itself — the rolling 12-week recency-weighted priority list self-updates, so master coverage doesn't drift back toward 80% when the project stops being new.

Why This Story Screams Wow the Customer

- ✔ Treated a supply-chain metric as what it actually is — a delivery promise to a specific customer
- ✔ Challenged the org's framing that a long-flat CX metric was a structural ceiling
- ✔ Traced the customer-felt outcome back to the exact operational points where intent broke
- ✔ Found two invisible execution gaps that were degrading experience daily and unmeasured
- ✔ Converted millions of long-lane shipments into local deliveries — faster promises, fewer breaches
- ✔ Built governance so the customer outcome kept improving after the analysis ended

---

Q23. Tell me about a tradeoff you consciously accepted.

Coupang Leadership Principle: MOVE WITH URGENCY *(secondary: Ruthless Prioritization)*

The principle in plain English: Coupang's Move with Urgency is explicit that urgency is a sense of crisis, that inaction is a threat to survival, and that leaders learn by doing rather than delaying decisions in search of a perfect solution. A trade-off question is where this gets tested most directly, because the classic trade-off in any build is speed against completeness. What the interviewer wants is a trade-off you made *knowingly*, with the cost named out loud, and a plan for the part you gave up — not a rationalisation constructed afterwards.

How interviewers may frame this question:

- Tell me about a tradeoff you consciously accepted.

- Describe a time you shipped something you knew was imperfect.
- Tell me about a time you chose speed over completeness.
- How do you decide when something is good enough?
- Give me an example of a decision where you accepted a known cost.
- Tell me about a time you deliberately took on technical debt.
- What's a compromise you'd defend?

Story: Multi-agent RCA System — shipping V1 on GSheet before V2 on BigQuery

Why this is the best fit:

This is the trade-off in its cleanest form: the ultimate requirement was near-real-time actionability at City × Category × Brand level, and the architecture that could deliver it would take months of engineering time — during which the business would have no AI capability at all. I shipped V1 on GSheet knowing it had a two-hour data lag and a 10M-cell ceiling. What makes it a defensible trade-off rather than a shortcut is that both limitations were documented and communicated on day one, V1 was framed as a capability proof rather than the final architecture, and V2 was committed as a fast follow before V1 even launched. The cost was named in advance, and the plan for paying it existed in advance.

2-Minute Executive Summary:

"When I started building Flipkart's first multi-agent RCA system for the Minutes business, the ultimate business requirement was near-real-time actionability at highly granular City × Category × Brand level. But there was no internal playbook for agentic AI workflows, and building the technically right solution — a robust NLQ-to-SQL layer on dedicated BigQuery infrastructure — would have taken months of engineering time, during which the business would have no AI capability at all and would still be burning analyst bandwidth on manual trackers. So I consciously launched V1 using Google Sheets as the backend, on metrics precomputed via an existing hourly pipeline, and explicitly accepted two costs: a roughly two-hour data freshness lag, and a 10M-cell limit that prevented deep granular cuts. The discipline was in not hiding them. I documented both limitations from day one, told the business team V1 was a capability proof rather than the final architecture, and committed to V2 on BigQuery as a fast follow before V1 launched. Then I used V1 as a discovery mechanism: the business started using the agentic flow in daily scrums, the core use case was being served, and as predicted they hit the GSheet ceiling — which gave me validated real-world evidence to justify a dedicated BigQuery processing slot and the engineering bandwidth for the NLQ-to-SQL semantic layer. V1 delivered value weeks before V2 was ready and proved the architecture worked. V2 came in at under an hour freshness with full City × Category × Vertical × Brand granularity, and it was built significantly faster because V1 had already validated the prompt logic and surfaced the exact requirements."

Full STAR Version

SITUATION

Flipkart Minutes chases OPD, and the existing process for understanding per-store performance consumed real analyst bandwidth: hourly achievement mailers triggering rounds of discussion, analysts working multiple GSheet trackers to establish what was happening, then writing deep-dive SQL to reach an actionable. That bandwidth should have gone into acting on the actionables. The target state was a natural-language system returning structured trends with growth against a benchmark and a RAG read, with follow-up RCA on demand.

The ultimate requirement was near-real-time actionability at highly granular City × Category × Brand level. The architecture that could deliver that — a robust NLQ-to-SQL layer over dedicated BigQuery infrastructure with a provisioned processing slot — represented months of engineering effort, in a space where nobody at Flipkart had an established playbook for agentic workflows.

TASK

Balance speed to market against architectural completeness: make a conscious engineering trade-off delivering immediate diagnostic value without blocking or compromising the long-term scalable solution.

ACTION

Step 1 — Chose the V1 scope and named the costs precisely. I decided to launch V1 with Google Sheets as the backend, using metrics precomputed via the existing hourly pipeline. The reasoning was concrete rather than convenient: the team already used GSheet trackers to answer some deep dives, so genuinely useful data was already there in a usable form. And I accepted two explicit costs: a **~2-hour data freshness lag** from the hourly pipeline, and a **10M-cell limit** that made granular City × Cluster × Vertical × Brand views impossible. Both were known before a line of it was built, not discovered afterwards.

Step 2 — Managed expectations up front rather than defending later. I didn't hide the trade-offs. I documented both limitations from day one and told the business team explicitly that V1 was a capability proof, not the final architecture, with V2 on BigQuery committed as a fast follow. This is the part that makes the trade-off defensible rather than a shortcut — the difference between accepting a cost and concealing one is entirely in whether the person who bears it knows about it in advance.

Step 3 — Applied a single, narrow shipping criterion. The test I used was: does it solve the core use case well enough to be used in daily scrums? V1 could pull metric trends and run directional RCA in natural language, so yes. Everything beyond that — granularity, freshness, cardinality — was optimisation. Having one explicit criterion is what stopped scope creep in both directions: it prevented shipping something unusable, and it prevented polishing something that was already sufficient.

Step 4 — Used V1 as a discovery mechanism for V2. We deployed and the business began using the agentic flow — Master, Discovery, Metric Pull and RCA agents — in daily scrums. The core use case was served. Then, exactly as predicted, they hit the GSheet ceiling: they needed granular NRT actionability that the cell limit and the two-hour lag

made impossible. That was a *validated requirement* rather than a failure, and it's the reason the trade-off compounded rather than cost. I used that real-world feedback to justify securing a dedicated BigQuery processing slot and the engineering bandwidth for the NLQ-to-SQL semantic layer.

Step 5 — Made sure the sunk cost was minimal by design. The prompt architecture, the Master router, the Discovery agent, the RCA issue tree, the attribution tools and the synthesis agent were all data-layer agnostic. Only the metric-fetch path was coupled to GSheet. So when V2 came, what had to be rebuilt was one sub-flow's connector, while everything expensive — the issue tree co-designed with the business, every prompt-engineering rule learned through failure — transferred intact.

RESULT

- **V1 delivered value weeks before V2 was ready**, proving the agentic architecture worked in production.
- **V2 achieved under-1-hour data freshness** (from over two hours) and **full City × Category × Vertical × Brand granularity**.
- **V2 was built significantly faster** than a from-scratch V2 would have been, because V1 had already validated the prompt logic and surfaced the exact user requirements.
- The system is in daily production use by both Analytics and Business — the first thing checked at the start of the day.
- The infrastructure request for a dedicated BQ processing slot was approved on the strength of demonstrated usage rather than a hypothesis.

KEY LEARNING

In novel problem spaces, perfect is the enemy of good — shipping a constrained V1 is the fastest way to learn what the right solution actually is. But the discipline that makes it work is transparency: consciously accepting an explicit, well-documented trade-off lets you deliver immediate value while building the conviction needed to fund the full architecture. The version of this that fails is accepting the same trade-off silently, because then the ceiling arrives as a broken promise rather than as a validated requirement. And designing so the sunk cost is confined to a replaceable layer is what turns a trade-off into a staged build.

What Interviewers Will Probe

- **What was the risk of shipping V1 before V2 was ready?** The main risk was anchoring — that users would adopt V1's limitations as permanent, treat the two-hour lag as acceptable, and make V2 a harder sell for engineering resources. I mitigated it by being entirely transparent about the limitations up front and setting a clear V2 transition plan before V1 even launched, so the ceiling was a shared expectation rather than a discovery.

- **How did you know V1 was good enough to ship despite the trade-offs?** One criterion: does it solve the core use case well enough to be used in daily scrums? It could pull metric trends and run directional RCA in natural language, so it did.

Everything else was optimisation, and delaying V1 to reach perfection would have cost weeks of learning and left the business on manual trackers throughout.

- **Isn't this just technical debt with a good story attached?** The distinction is whether the debt was priced and scheduled. Both limitations were quantified before building, communicated to the people affected, and V2 was committed before V1 launched. And the architecture confined the debt to one connector rather than letting it spread through the agent design. Undocumented, unscheduled, structurally entangled debt would be a fair criticism; this wasn't that.
- **What if the business had been satisfied with V1 and V2 never got funded?** That was the real risk and it's why I set the V2 plan before launch. In practice the Minutes use case made it self-correcting — quick commerce is an intraday business, so a two-hour-old answer at a coarse cut genuinely doesn't support intraday action. The ceiling wasn't a preference, it was a functional wall. If the use case had been weekly reporting, I'd have needed a harder commitment mechanism up front.
- **How do you decide which trade-offs are acceptable and which aren't?** I ask whether the cost is reversible and whether it's visible to the person bearing it. A two-hour lag on a prototype is reversible and I made it visible. What I wouldn't have accepted is a trade-off on correctness — which is why all computation went into Python tools rather than being left to the model even in V1. A slow right answer is a trade-off; a fast wrong answer isn't.
- **What's the biggest lesson?** That the value of a constrained V1 isn't just earlier delivery — it's that it converts your assumptions into evidence. I *believed* Minutes needed granular NRT. After V1, I could demonstrate it, which is what unlocked infrastructure I couldn't have got on conviction alone.

Why This Story Screams Move with Urgency

- ✔ Refused to delay a decision in pursuit of a perfect architecture in a space with no playbook
- ✔ Treated inaction as the real cost — weeks of analyst bandwidth burning on manual trackers
- ✔ Learned by doing: V1's usage produced the evidence that specified and funded V2
- ✔ Named both costs explicitly and in advance rather than rationalising them afterwards
- ✔ Applied one clear shipping criterion instead of debating readiness
- ✔ Designed the trade-off so the sunk cost was confined to a single replaceable layer

---

SECTION 7: DEEP DIVE PRINCIPLES (Google/Agoda/Coupang Style)

This section maps your strongest projects to the interview principles used by Google, Agoda, and Coupang. Each entry follows: Why this is the best fit → 2-Minute Executive Summary → Full STAR Version → What interviewers will probe → Why this story screams the principle.

1. DIVE DEEP/ Tell me about a time you influenced product direction."

Coupang Leadership Principle: DIVE DEEP *(secondary: Wow the Customer)*

The principle in plain English: Coupang wants hands-on leadership with a passion for detail — digging down to the smallest details to gain a full understanding, which is what equips you to empower the right people and deliver results. They add pointedly that no task is beneath us. The interviewer is measuring how many layers below the obvious you went, and whether you did the unglamorous work yourself rather than delegating or inferring it.

How interviewers may frame this question:

- Tell me about the deepest piece of analysis you've done.
- Describe a time you found a root cause nobody else could.
- Tell me about a time you influenced product direction.
- Give me an example of when you didn't stop at the first answer.
- How do you validate that a driver is causal and not just correlated?
- Tell me about a time you went and spoke to customers directly.
- What's the most detailed work you've personally done rather than delegated?

Story: Modelling the Drivers of Low-TPC Long-Term Retention

Customer_Analytics_STAR

Why this is the best fit:

Interviewers say: "We dig down to the smallest details to gain a full understanding." This story is a textbook Dive Deep because: most teams stopped at the aggregate churn number. You went seven themes deep — 50+ behavioural variables across selection, pricing, experience, diversity, engagement, and community contribution. You then went deeper still: for each driver, you ran EDA to confirm the mechanism (not just the correlation), and then went one level further by cold-calling 500 churned customers to triangulate the quantitative model with qualitative grounding. Three levels of depth — model, EDA, cold-calling — on the same problem.

2-Minute Executive Summary:

"Low TPC customers — those who have made 1-3 transactions in the last 12 months — represent 40%+ of Flipkart's active customer base but churn at ~70% in the long term. With new-customer growth starting to saturate, reducing Low-TPC churn became one of the most important levers available to hit AAC and MAC targets. I built an XGBoost model on 50+ behavioural variables captured in the first 90 days of a Low-TPC customer's journey

— across seven themes: selection, pricing, experience, diversity, engagement, feedback, and social awareness. Feature importance and SHAP values surfaced three dominant drivers: super-high engagement, quality at low ASP, and community contribution (rating and reviewing purchases). I then ran EDA to confirm the mechanism behind each driver — not just that they correlated with retention, but why. And then I went one level deeper: shared 500 Low-TPC churner accounts with the business team for cold-calling, and the qualitative insights triangulated exactly with the model's drivers. The analysis converted a single churn number into a prioritized, owner-mapped retention plan with aggressive targets — and drove 100 bps improvement in Low-TPC monthly repeat within 3 months."

Full STAR Version

SITUATION

Flipkart's AAC and MAC — the primary customer health metrics — are composed of New Customers plus Low TPC (1-3 transactions/12M), Mid TPC (4-7), and High TPC (8+). Low TPC is a huge base — 40%+ of the total — but retains very poorly: ~70% long-term churn rate (no transaction in the next 6-12 months). Mid and High TPC are entrenched platform users who understand the value they get from Flipkart. Low TPC customers are early in their journey — and most of them never become Mid or High TPC. With top-down AAC/MAC targets growing more aspirational and new-customer acquisition starting to saturate organically, solving Low-TPC retention was the highest-leverage lever available. No one had systematically diagnosed why they churned.

TASK

Identify the behavioural drivers of Low-TPC long-term retention — specifically, what distinguishes a Low-TPC customer who repeats in months M6-M12 from one who churns — and translate those drivers into concrete, measurable action items and Key metrics the Business could chase to reduce churn.

ACTION

Step 1 — Designed the modelling framework from first principles. I defined the target variable as: did an M0-acquired Low-TPC customer repeat in months M6-M12 (1/0)? This captured long-term retention — not just M1 repeat. I then built 50+ behavioural features from the first 90 days of each customer's journey, organized across seven themes:

- **Selection:** category affinity, quality of selection (FA, branded, > highly-rated products, selection freshness)
- **Pricing:** discount over BAU, offers applied on selection
- **Experience:** RTO share, RVP share, customer-support contacts, > speed metrics, breach metrics
- **Diversity:** number of Super Categories purchased in the first 90 > days
- **Engagement:** number of visits, PPVs, finding methods (search / > merch / reco), PPVs by category

- **Feedback / Social awareness:** percentage of products rated or > reviewed

Step 2 — Built and validated the XGBoost model. Ran feature importance and SHAP values to surface the dominant drivers. Three themes emerged: super-high engagement, quality at low ASP, and community contribution (rating and reviewing purchases).

Step 3 — Went deeper: EDA to confirm the mechanism behind each driver. I didn't stop at the model output. For each driver, I ran EDA to understand why it drove retention — not just that it correlated with it:

- **Engagement:** churners' weekly engagement dropped > disproportionately week-over-week in the first 90 days vs. > retainers — confirming that engagement loss was a leading > indicator of churn, not just a correlate.
- **Quality at low ASP:** at the same ASP bucket, retainers bought > more branded, FA, and 4+-rated products than churners — > confirming that it was quality of selection at low price, not > price alone, that drove retention.
- **Community contribution:** retention probability rose monotonically > with % units rated — confirming that soliciting ratings was not > just a feedback mechanism but an engagement and retention lever in > itself.

Step 4 — Went one level deeper still: qualitative triangulation. I shared 500 Low-TPC churner accounts (active browsers who had stopped transacting) with the Business team for cold-calling. The qualitative insights triangulated exactly with the model's quantitative drivers: affordability and quality of selection (customers couldn't find products at the right price-quality combination), variety (platform didn't show them products they hadn't thought to search for), and post-order experience (poor product quality on some categories eroded trust).

Step 5 — Translated drivers into owner-mapped action items and targets. Each driver became a specific business action with an aggressive KPI:

- **Engagement:** improve adoption of essential features (CLP, Search, > FA, Ratings, Reviews) via video tutorials / simpler app UX; > incentivize weekly visits through gamification. Key metric: WAU of > the Low-TPC base.
- **Quality at low ASP:** ensure platform-guided discovery of > high-quality products for the first 3 transactions (90 days) > across Homepage, Merch, CRM, Search, and Reco. Key metrics: > 4+-rated product share of total units and FA unit share for Low > TPC.
- **Community contribution:** proactively solicit Low TPC customers to > rate previous purchases. Key metric: % units rated by Low TPC.

RESULT

100 bps improvement in Low-TPC monthly repeat observed within 3 months of the business teams acting on the identified levers. The analysis converted the most important (and worst-retaining) customer segment into a clear, prioritized set of retention levers —

backed by a model, SHAP values, EDA, and cold-calling validation. Each lever landed as an owned business action item with an aggressive KPI target. Manager feedback: *"Richa has spent significant time in understanding the data, drivers and subsequently has made best effort in driving these insights to action. This was very high quality work."*

What Interviewers Will Probe

Question: Why didn't others find this?

Strong Answer: Because most churn analyses stop at the aggregate number or at simple demographic cuts. The insight required going into behavioural data — 50+ variables across 7 themes — and specifically looking at what customers did in their first 90 days, not what they looked like demographically. The mechanism (engagement loss as a leading indicator, quality at low ASP as the specific combination that drove retention) was not visible at the aggregate level.

Question: Why did you go to cold-calling after building the model? Wasn't the model enough?

Strong Answer: The model told me which variables correlated with retention. The cold-calling told me why those variables mattered — the actual customer experience behind the numbers. For example, the model surfaced "quality at low ASP" as a driver. The cold-calling revealed what that meant in practice: customers couldn't find products they wanted at the price range they could afford. That's a product and selection problem, not a pricing problem. The qualitative layer made the action items much more specific than the model alone could have.

Question: How did you decide on 50+ variables and 7 themes?

Strong Answer: I built the themes from first principles — by asking: "what are all the ways a customer's first 90-day experience on Flipkart could differ between someone who stays and someone who churns?" Selection, pricing, experience, diversity, engagement, feedback — these are the dimensions of the customer's first-impression of the platform. Within each theme I captured every available signal. The constraint was data availability in the first 90 days, not a pre-determined variable count.

Question: What's the biggest lesson?

Strong Answer: Depth in analysis is not about having more variables. It is about going to the next level of "why" every time you find something. The model found what drove retention. EDA found the mechanism. Cold-calling found the customer experience behind the mechanism. Each level made the action items more specific and more actionable. The 100 bps improvement came from that depth — not from the model alone.

Why This Screams Dive Deep:

- ✔ Went three levels deep on the same problem: model → EDA → > cold-calling
- ✔ 50+ variables across 7 themes — designed from first principles, > not convenience

- ✔ EDA confirmed the mechanism behind each driver, not just the > correlation
- ✔ Cold-calling triangulated quantitative findings with qualitative > grounding
- ✔ Each level of depth made the action items more specific — 100 > bps improvement in 3 months

2. THINK SYSTEMATICALLY

Coupang Leadership Principle: THINK SYSTEMATICALLY *(secondary: Dive Deep)*

The principle in plain English: Build scalable processes with prompt feedback mechanisms, and take measures not only to fix defects but to prevent them recurring. The interviewer is checking whether you fixed an instance or fixed the system that produced it — and specifically whether you built the measurement layer *before* the analysis, because that is what separates a repeatable process from a one-off answer.

How interviewers may frame this question:

- Tell me about a process or system you built rather than a problem you solved.
- Describe a time you had to design measurement before you could analyse anything.
- How do you prevent the same defect recurring?
- Give me an example of building something that made future work easier.
- Tell me about a time you saw the whole system rather than one part of it.
- Describe how you'd instrument a process that has no data today.
- What have you built that's still in use?

Story: Pricing Automation — End-to-End Instrumentation Design

Why this is the best fit:

Thinking systematically means seeing the system before analyzing its parts. The Pricing Automation Pilot had two products built and measured in isolation — no shared data layer, no system-level view. Before a single analysis was possible, the system had to be mapped. This story demonstrates that the most important analytical decision was not a model choice or a metric — it was understanding the architecture of the problem first.

2-Minute Executive Summary:

"Flipkart's automated pricing Pilot had two products — the Optimizer (which recommended prices) and OI (which mapped offers to achieve those prices). Each product was built and measured independently. There was no shared data layer capturing the end-to-end story: from price recommendation to offer mapping to realized ASP. Before any analysis was possible, I mapped the full system journey at reaction ID × FSN level and defined the exact metrics needed to stitch the story together. This instrumentation layer became the foundation for everything that followed — including discovering that ASP deviation was the single biggest driver of GMV misses, which was completely invisible before the system-level view existed."

Full STAR Version

SITUATION

Flipkart's 1P catalog had 10M+ listings with 90-95% of pricing done manually. The CEO approved a Pilot of automated pricing at the Super Category level. Two products were built to work in tandem: the Optimizer (recommended FSN × ASP × FSP combinations maximizing GMV within CM constraints) and Offer Intelligence/OI (bridged the FSP → ASP gap by recommending which offers to map or unmap). At project start, each product team had built their own instrumentation independently. There was no shared data layer connecting the two — no way to trace the journey from price recommendation to realized ASP. Without this, any analysis of Pilot performance would be measuring shadows.

TASK

My first mandate was to design and drive end-to-end data instrumentation across both products so the system could be measured as a whole — not as two isolated tools. This was the prerequisite for everything else.

ACTION

Step 1 — Mapped the full system journey. I traced every step: reaction ID generation → FSP/ASP recommendation → BCD projection → offer go-live → actual BCD → realized ASP. At each step I asked: "What can go wrong here? What metric do I need to capture it?" This gave me a complete, non-overlapping set of measurement points.

Step 2 — Defined exact metrics at reaction ID × FSN granularity: FSP, ASP, predicted DRRs, reaction go-live time, offer go-live time, projected mandatory BCD, non-mandatory BCD, actual BCD, and actual GMV and CM. The go-live timestamps were critical — they enabled decomposing ASP deviation by synchronization failure vs. offer mapping error.

Step 3 — Advocated for instrumentation as product. I ran multiple alignment rounds with Product, Engineering, and Program teams — making the case that instrumentation was as critical as the product itself, especially for a first-of-its-kind CEO-level Pilot. This was not universally accepted at first; teams wanted to move quickly to the Pilot. I held the position.

Step 4 — What the instrumentation revealed. Once live, the data showed that only 80% of GMV had both price and offer live in sync — and that ASP deviation was the single biggest driver of GMV and CM misses. Neither of these findings was visible before the system-level view existed.

RESULT

The instrumentation became the foundation for all subsequent Pilot reviews. It enabled the four-layer governance framework, the ASP deviation diagnosis (reducing Pricing deviation from **70% to 30%** of Revenue), and the KVI standardization (reducing CI-match portfolio from **70-80% to ~30-40%**). It also revealed ~**500 Cr of bad burn risk** at BBD scale that would have been completely invisible without the system-level view.

What Interviewers Will Probe

Question: Why did you start with instrumentation rather than analysis?

Strong Answer: Because without the right data, any analysis is measuring the wrong thing. The Pilot was designed to maximize GMV, but the primary failure mode — ASP deviation — wasn't being tracked anywhere. Building the instrumentation wasn't a delay; it was the prerequisite. Starting analysis before the data existed would have produced confident answers to the wrong questions.

Question: How did you convince teams to invest in instrumentation before seeing results?

Strong Answer: I framed it in terms of risk. This was a CEO-level Pilot with significant org change attached. If we couldn't measure what was happening, we couldn't learn from it. The cost of inadequate instrumentation in a high-stakes Pilot was far higher than the cost of building it properly. That argument resonated with leadership even when it didn't immediately resonate with individual teams.

Question: What's the biggest lesson?

Strong Answer: Before analyzing a system, you have to understand it. Mapping the full journey — every step, every handoff, every place where the outcome can diverge from the intention — is not preparation for the analysis. It is the analysis. The instrumentation design told me everything I needed to know about where the system could fail before I had seen a single data point.

Why This Screams Think Systematically:

- ✔ Saw the full system before analyzing any part of it
- ✔ Mapped every step where the outcome could diverge from intention
- ✔ Defined non-overlapping metrics at each step — no gaps, no > overlaps
- ✔ Held the position that instrumentation was a prerequisite, not > overhead
- ✔ The system-level view revealed the root cause (ASP deviation) that > was invisible to everyone else

3. BIAS FOR ACTION

Coupang Leadership Principle: MOVE WITH URGENCY *(this is Coupang's equivalent of Bias for Action — secondary: Ruthless Prioritization)*

The principle in plain English: Coupang doesn't use the phrase "Bias for Action"; their equivalent is Move with Urgency, and it's stronger — urgency is a sense of crisis, inaction is treated as a threat to survival, and leaders learn by doing rather than delaying decisions to seek a perfect solution. If an interviewer asks a Bias for Action question, this is the principle they are scoring against, so emphasise shipping into real use and the evidence that speed generated.

How interviewers may frame this question:

- Tell me about a time you shipped something imperfect on purpose.
- Describe a time speed mattered more than completeness.
- Give me an example of when you acted before having full information.
- Tell me about a time waiting would have been the wrong call.
- How do you decide when something is ready to go live?
- Describe a time you learned by doing rather than by planning.
- Tell me about a decision you made quickly that you'd defend today.

Story: Multi-agent RCA System — shipped V1 on GSheet before V2 on BigQuery

Why this is the best fit:

Bias for Action means shipping something real and learning from it — rather than waiting for the perfect solution. This story demonstrates it because: nobody had a playbook. I had never built an agentic system before. The technically superior solution (BQ-backed V2) would have taken significantly longer. I shipped V1 explicitly, with known limitations documented, so the business could start getting value while V2 was built. V1 also validated the architecture before the harder V2 investment.

2-Minute Executive Summary:

"When I started building Flipkart's first multi-agent RCA system for the Minutes (Quick Commerce) business, I had two options: wait to build the technically correct V2 solution (BigQuery-backed, NRT, granular City × Cluster × Brand actionability) or ship a V1 on GSheet quickly to prove the concept and start delivering value. I chose V1. I was explicit about V1's limitations — 2+ hour data freshness lag, 10M cell constraint. I committed to V2 as a fast follow. V1 got into the hands of Business and Analytics teams weeks earlier. It proved the agentic architecture worked. It surfaced the exact limitations that V2 needed to solve. And it generated the organizational confidence that made V2 a faster build."

Full STAR Version

SITUATION

Flipkart Minutes (Quick Commerce) chases Orders Per Day (OPD) as its key metric, with the ambition to become the second-largest quick-commerce player. An hourly mailer goes out every 40th minute with input and output metrics — and triggers a constant stream of questions about metric underachievement. Analytics and Business time was being consumed figuring out what was happening rather than acting. The opportunity: build a prompt system that identifies root causes behind OPD drops in near-real-time so actionability can be driven immediately. This was Flipkart's first agentic-flow project. No internal playbook existed.

TASK

Build and deploy a working multi-agent RCA system that Business and Analytics could use in daily scrums — as fast as possible — while also planning for the full NRT, granular V2 that the business ultimately needed.

ACTION

Step 1 — Chose the right V1 scope. V1 used GSheet as the backend — metrics precomputed across many dimension cuts, updated hourly via a scheduled pipeline. This meant I could build the full agentic architecture (Master agent, Discovery agent, Metric Pull agent, Orders RCA agent) without solving the harder BQ/NLQ-to-SQL problem first. GSheet had real limitations: 2+ hour freshness lag, 10M cell cap preventing City × Cluster × Vertical × Brand granularity. I documented these explicitly from day one.

Step 2 — Shipped V1 as a proof of concept. The agentic architecture — Master routing agent, Discovery agent, entity extraction, growth/RAG calculation — worked. Business and Analytics teams started using it in daily scrums. The core use case (metric pull and directional RCA) was being served.

Step 3 — V1 surfaced exactly what V2 needed to solve. Because V1 was in use, the business team was able to articulate the specific limitations that mattered: they needed NRT data (not 2-hour lag) and City × Category × Vertical × Brand granularity. These were concrete, validated requirements — not hypothetical ones.

Step 4 — Built V2 with a clear specification. Switched the backend to BigQuery with a dedicated processing slot (reducing freshness from 2+ hours to <1 hour), rebuilt the Metric agent with NLQ→SQL (semantic layer, fuzzy dimension matching, standard aggregation mappings), and extended the RCA agent with contribution analysis and Shapley attribution tools.

RESULT

V1 delivered value weeks before V2 was ready. V2 was built faster because V1 validated the architecture and surfaced the exact requirements. Final system: data freshness **<1 hour**, RCA turnaround from **days to <1 hour**, actionability at **City × Category × Vertical × Brand** granularity. Adopted in daily scrums and intraday diagnostics.

What Interviewers Will Probe

Question: What was the risk of shipping V1 before V2 was ready?

Strong Answer: The main risk was that users would adopt V1's limitations as constraints — treating the 2-hour lag as acceptable and making V2 a harder sell. I mitigated this by being transparent about the limitations upfront and setting a clear V2 commitment before V1 launched. When the business team hit the GSheet ceiling, they expected it. V2 was already in progress.

Question: How did you know V1 was good enough to ship?

Strong Answer: I used one criterion: does it solve the core use case well enough to be used in daily scrums? V1 did — it could pull metric trends and run directional RCAs. Everything else was optimization. The cost of delaying V1 to achieve perfection was weeks of lost learning and delayed value.

Question: What's the biggest lesson?

Strong Answer: In novel problem spaces — especially first-of-a-kind builds — shipping something real is the fastest way to learn what the right solution actually is. Abstract requirements gathering never surfaces the specific constraints that matter. Only usage does. V1 was not just a delivery — it was a discovery mechanism for V2.

Why This Screams Bias For Action:

- ✔ Shipped real value weeks before the perfect solution was ready
- ✔ Documented limitations explicitly — no false confidence
- ✔ Used V1 as a discovery mechanism, not just a delivery
- ✔ V2 was built faster because V1 validated the architecture
- ✔ Did not wait for a playbook — built one while building the > system

4. OWNERSHIP

Coupang Leadership Principle: COMPANY-WIDE PERSPECTIVE *(this is Coupang's equivalent of Ownership — secondary: Deliver Results with Grit)*

The principle in plain English: Coupang doesn't have an "Ownership" principle by that name; Company-wide Perspective is the closest and it's more specific. Leaders think like owners and act in the best interest of the entire company, consider upstream and downstream implications, and raise problems inside or outside their own department — never saying "that's not my job." If asked an Ownership question, emphasise the whole-company interest over the local metric, and problems you picked up outside your remit.

How interviewers may frame this question:

- Tell me about a time you took ownership of something outside your remit.
- Describe a decision where the company-wide answer differed from your team's answer.
- Give me an example of considering the downstream impact of your work.
- Tell me about a time you raised a problem in someone else's area.
- How do you behave when something falls between two teams?
- Tell me about a time you owned an outcome you didn't control.
- Describe the largest scope of impact you've personally been responsible for.

Story: Maximizing BBD Success — Baseline Shift, Customer Shape Projection, Live Control Tower & Intrigue Strategy

Why this is the best fit:

Ownership means taking full, end-to-end accountability — for the strategy, the execution, and the outcome. This story is the strongest ownership example because it spans three interconnected workstreams that I owned completely: (1) decomposing the baseline shift

and projecting the optimal BBD customer shape — the strategic foundation for how BBD was targeted, (2) designing the Intrigue HVA strategy to maximize BBD traffic and customers — the pre-event execution lever, and (3) running the live control tower with zero escalations from the CEO and CFO across 8 continuous days. Each workstream was high-stakes and high-visibility. Dropping any one of them would have undermined the others. I owned all three.

2-Minute Executive Summary:

"BBD is Flipkart's largest event — and the event after which the platform sees the most important long-term growth outcome: baseline shift. I owned the analytics end-to-end across the entire BBD cycle. Pre-BBD: I decomposed the post-festive baseline shift (establishing NC and Low/Med TPC as the dominant drivers), built the GMV-guardrailed customer shape projection, and designed the Intrigue HVA strategy — identifying which cohort-construct combinations drove the most BBD-conversion lift and standing up the daily tracker that governed Intrigue execution. During BBD: I solely managed the live control tower — hourly mailers, NRT dashboard, 20+ team roster, war room RCA support — with zero escalations from leadership including the CEO and CFO across all 8 days. The Intrigue strategy delivered +5% incremental BBD UVs and +0.25 Mn customers. The baseline shift approach drove a 10% shift in Customer DRR. Zero leadership escalations on data visibility or L0 deep-dive."

Full STAR Version

SITUATION

BBD (Big Billion Days) is Flipkart's single biggest event — 8 consecutive days where the CEO, CFO, and SVP sit in a war room tracking every L0 metric in near-real-time. It is also the event after which the platform sees its most important long-term growth outcome: post-festive baseline shift, where customer DRR runs ~1.5x pre-festive DRR for the following 60 days. The three months before BBD involve: understanding what drives baseline shift (to steer BBD strategy), building the Intrigue strategy (to maximize BBD traffic before the event starts), and running the live control tower flawlessly during the event itself. All three workstreams were mine.

TASK

Own the full analytics lifecycle for BBD: (1) decompose baseline shift and build the optimal customer shape projection, (2) convert the open-ended Intrigue problem into a targeted, measurable HVA strategy with a governing tracker, and (3) run the live control tower for 8 days with zero CEO/CFO escalations on data visibility or L0 deep-dive.

ACTION

Workstream 1 — Baseline Shift Decomposition & Customer Shape Projection

Step 1 — Decomposed the post-festive baseline shift. Broke down customer DRR by TPC cohort (Low = 1-3 txns/12M, Med = 4-7, High = 8+) for pre-festive (60 days prior) and post-festive (60 days after). Findings: ~30% led by NCs acquired during festive, ~20% by

incremental NC DRR network effect post-festive, ~30% by Low and Med TPC transacting during BBD — High TPC contribution structurally capped at ~20% because they were already entrenched platform users.

Step 2 — Built the GMV-guardrailed customer shape simulator. Projected organic BBD customer numbers by TPC cohort and NC (using historical BBD incidences by recency × BAU YoY multiplier), then projected the shape needed to hit the baseline shift target — with GMV as a non-negotiable guardrail. Shared the shape that maximized baseline shift while ensuring BBD GMV was still met. Triangulated with Demand Planning using multiple approaches (Units ÷ UPC cross-check, category GMV targets).

Workstream 2 — Intrigue HVA Strategy

Step 3 — Established that Intrigue visits matter. Set up sessions with the Events team to learn constructs and page IDs, then ran EDA: for each TPC cohort controlled on recency, BUPC, SPC — computed the conditional probability of converting in BBD given they visited during Intrigue vs. not. The disproportionate lift confirmed Intrigue visits matter.

Step 4 — Decomposed Intrigue UVs and phase importance. Segmented UVs (brand-new, not-yet-converted, Low/Med/High TPC buyers, 12M+ reactivations). Established that visiting all three Intrigue phases gave maximum BBD-conversion lift — with Phase 2 and Phase 3 driving disproportionate lift. This established engagement (visits/UV, PPV/UV) as the primary metric to chase.

Step 5 — Ranked constructs and HVAs by cohort. Top conversion-lift constructs: Central Intrigue page (links to all deals, BBD specials, affordability constructs), Affordability page (bank offers, EMI, NCEMI, exchange), BBD Special page. Cohort-specific HVAs: High TPC → becoming Plus during Intrigue; Low TPC and new users → transacting during Intrigue.

Step 6 — Set targets and stood up the tracker. Shared HVA × cohort matrix with the Events team, set day-level targets on each (Central Intrigue page UV share, affordability page, game-zone interactions, Plus base). Built a daily tracker monitoring actuals vs. targets for key engagement metrics — used in the daily Intrigue stand-up to drive actionability on lagging constructs (first-fold visibility, Central Intrigue link on order-summary page, coupons/TDs for Low TPC).

Workstream 3 — Live BBD Control Tower

Step 7 — Pre-BBD infrastructure. Hourly target phasing for every L0 metric, dry runs of hourly mailers, cheat sheets for each likely failure mode, Omniture SOP (P0/P1/P2 tagging, segment optimization, limits on simultaneous schedules), traffic spike estimates shared with Adobe for hardware preparedness.

Step 8 — 24×7 roster across 20+ teams. Built explicit cross-functional coverage across Analytics, IT, Omniture, and FDP — named owners for every shift, defined escalation chain, no single point of failure.

Step 9 — War room presence. Sat in the war room for 8 days, delivering real-time RCAs on any L0 metric deviation — payment gateway drops, conversion dips, traffic shortfalls — within minutes using cheat sheets and roster support.

RESULT

Intrigue strategy: +5% incremental BBD UVs. Central Intrigue page interaction rose from 12% to 14.5%, delivering **+0.25 Mn incremental BBD customers**. Low-TPC Intrigue purchase focus and game-zone engagement each contributed **+0.1 Mn customers**.

Baseline shift: 10% shift in Customer DRR post-BBD. Diversity and event-to-event repeat adopted as explicit BBD goals for the first time — delivering **+0.7 Mn incremental diversified customers** over plan and **+500 bps event-to-event repeat** YoY.

Control tower: Zero leadership escalations from CEO/CFO on NRT data visibility or L0 deep-dive across all **8 days of BBD**.

What Interviewers Will Probe

Question: How did you manage three high-stakes workstreams simultaneously?

Strong Answer: The key was sequencing — the baseline shift analysis fed the Intrigue target-setting, which fed the control tower tracking. Each workstream's output was an input to the next. Getting the sequencing right meant downstream work was never blocked. The pre-BBD workstreams had to land before the event; the control tower was owned during the event.

Question: What happened when something broke during BBD?

Strong Answer: I had pre-built response playbooks for the most likely failures. The cheat sheets captured the exact diagnostic steps for each scenario. The roster meant I always had the right person available. The fastest RCAs came from preparation, not from real-time analysis. The war room's job was to execute playbooks, not improvise them.

Question: How did you drive the Intrigue strategy with a team you didn't manage?

Strong Answer: The HVA × cohort matrix gave the Events team something specific and measurable to own. Instead of saying "drive more Intrigue engagement," I gave them: "Central Intrigue page UV share should be X% by Day 5 of Phase 2." Specific targets with a daily tracker made the ownership self-evident. I didn't need to manage — the tracker managed.

Question: What's the biggest lesson?

Strong Answer: True ownership of a high-stakes event means owning the strategy, the pre-event execution, and the live operations — not just one of them. The control tower ran cleanly because the strategy was right. The Intrigue strategy worked because the targets were grounded in data. Each workstream reinforced the others. Partial ownership would have produced partial results.

Why This Screams Ownership:

- ✔ Owned three interconnected high-stakes workstreams end-to-end — > strategy, pre-event execution, live operations
- ✔ Intrigue strategy built from scratch: open-ended problem → > targeted HVA × cohort matrix → daily tracker
- ✔ Baseline shift decomposition changed how BBD was targeted — > NC/Low TPC first, not High TPC first
- ✔ Zero CEO/CFO escalations across 8 days — outcome matched the > accountability
- ✔ All three workstreams delivered: +5% incremental BBD UVs, 10% > baseline shift, zero escalations

5. DATA-DRIVEN DECISION MAKING

Coupang Leadership Principle: INFLUENCE WITHOUT AUTHORITY *(secondary: Demand Excellence — Coupang has no standalone "Data-Driven Decision Making" principle)*

The principle in plain English: Coupang folds data-driven decision making into Influence without Authority: leaders lead by communicating ideas clearly, drive alignment through data and insights, and create an environment where knowledge trumps position. The related bar is Demand Excellence — a passion for excellence and no tolerance for mediocrity, held to a perpetually rising standard. If asked a data-driven-decision question, the scoring is on whether the data changed a decision, not on the sophistication of the method.

How interviewers may frame this question:

- Tell me about a time data changed a decision.
- Describe your most rigorous piece of analysis and what it changed.
- Give me an example of replacing intuition with evidence.
- How do you make sure your analysis is trusted?
- Tell me about a time you raised the analytical standard on your team.
- Describe a decision you'd have got wrong without the data.
- What's the highest-quality analytical work you've done and why was it high quality?

Story: Intelligent Couponing — uplift model that replaced blunt full-base targeting

Why this is the best fit:

Data-driven decision making means letting data change both your conclusions and your methodology — even when the change is uncomfortable. This story is the strongest fit because: the entire team was running couponing the same way for years. The data revealed not just a better answer, but a fundamentally better question. The shift from "who is most likely to buy?" to "who is most likely to buy because of the coupon?" is the kind of insight that only comes from letting data drive the reframing — and then proving it with a controlled experiment before asking anyone to commit.

2-Minute Executive Summary:

"The NC couponing strategy at Flipkart targeted the entire NAC base with a discount coupon, measured Average Treatment Effect, and reported incremental new customers. It was the standard approach — and it was fundamentally inefficient. It was giving coupons to 'Sure Things': users who would have purchased anyway. The coupon was wasted on them. I introduced uplift modeling — an S-Learner XGBoost model that estimates per-user CATE (the individual-level lift in conversion probability due to the coupon) — and identified the top 3 deciles (Persuadables) that drove ~80% of incremental new customers at ~50% of the cost. The model achieved ~80% AUC on normalized cumulative gain. A confirmatory A/B proved the hypothesis before production deployment. The result: monthly couponing spend reduced from 9 Cr to 4 Cr (~55%) while preserving ~80% of incremental new customers at a CAC of 200 vs. 300. The model ran in production for 5-6 months."

Full STAR Version

SITUATION

Within the NC charter, one of the key metrics was NC growth with an AOP target of ~30% YoY growth. The incentive lever — couponing — was being used to bridge the gap between organic acquisition and the AOP target. The existing strategy was blunt: take the entire NAC (non-acquiring customer) base, split Test and Control, give Test a discount coupon, measure ATE (Test conversion − Control conversion), and report incremental NC at some RPC and CAC. The fundamental inefficiency was invisible because no one had questioned the targeting methodology: the offer was going to users who would have purchased anyway (Sure Things), wasting significant spend with no incremental benefit.

TASK

Build an intelligent couponing strategy that identifies users whose uplift in conversion probability is high — and target only those users — to optimize for both RPC and CAC. The question to answer was not "who is most likely to buy?" (propensity) but "who is most likely to buy because of the coupon?" (uplift).

ACTION

Step 1 — Reframed the problem using the four behavioral segments. I introduced the conceptual framework that made the inefficiency of propensity targeting immediately visible: Persuadables (buy because of the coupon — worth targeting), Sure Things (buy anyway — wasted spend), Sleeping Dogs (become less likely to buy with the offer — negative uplift), Lost Causes (never buy regardless — zero uplift). Explicitly designed against propensity targeting — which loads the target group with Sure Things.

Step 2 — Ran a clean randomized A/B as the training data foundation. Split the NAC base into Test and Control; Test received an x% off up to ₹z coupon. Computed ATE and confirmed statistical significance (a near-zero ATE would make uplift modeling pointless). Captured each user with Test/Control flag plus engagement features (ATC clicks, banner clicks, bounce rate, session length, scroll depth, PPVs by BU) and demographic dimensions (gender, city tier, zone, age).

Step 3 — Built the S-Learner uplift model. Used a single XGBoost Classifier taking all features plus the treatment/control flag as a feature, with conversion (1/0) as target. To estimate per-user uplift (CATE), scored the test hold-out twice: once with treatment flag = 1, once with treatment flag = 0. The difference — P(conversion | treated) − P(conversion | not treated) — gave the individual-level uplift score.

Step 4 — Validated model accuracy three ways. (a) Decile analysis: created 10 bins ranked by predicted uplift, computed true uplift per decile — a well-calibrated model shows higher true uplift in top deciles, which ours did. (b) Calibration plot: predicted average uplift vs. true uplift per bin — scatter sat close to the 1:1 line. (c) Normalized cumulative gain chart: AUC ~80%, at industry standard (random model = 50%).

Key finding: top 3 deciles captured ~80% of incremental new customers at ~50% of total cost — CAC down to ~150 (vs. ~350 full-base) at RPC of ~4 (vs. ~2). Feature analysis: top-decile users had recently visited, done more ATC and banner clicks, more search PPVs, added addresses, lower bounce rates — engagement features dominated over demographics.

Step 5 — Ran a confirmatory A/B before production deployment. 90/10 Test/Control for top 3 deciles, 10/90 for bottom 7 — to prove that top-3 ATE was significantly higher and RPC/CAC were optimized. The February experiment confirmed the hypothesis. Model pushed to production with Data Science team.

Step 6 — Extended to offer-construct optimization. Using the decile framework, designed A/B experiments on top deciles across offer constructs (30% off up to ₹50 vs. 10% off up to ₹100 vs. ₹9 store). Disproved the leadership assumption that deeper discount always lowers RPC — by decomposing RPC = incremental revenue/burn and CAC = incremental cost/burn per construct. The 30% off construct had higher RPC and lower CAC, because customer perception of offer value drove a spike in incremental NC large enough to outweigh the higher burn.

Step 7 — De-risked the strategy. Quantified that coupon availers' long-term repeat and spend behavior was not materially different from organic customers — removing the concern that incentive-driven NC would be low quality.

RESULT

Monthly couponing spend reduced from **9 Cr to 4 Cr (~55% reduction)** while preserving **~80% of incremental new customers** at a **CAC of 200 vs. 300**. Model ran in production for **5-6 months**. Uplift-based targeting adopted as the standard methodology for NC couponing. Offer-construct optimization framework disproved a long-held leadership assumption — 30% off drove higher RPC than 10% off in the right target group.

What Interviewers Will Probe

Question: Why uplift modeling instead of propensity modeling — isn't it more complex?

Strong Answer: Propensity models predict who is likely to buy. For organic customer journeys, that's the right question. For incentive targeting, it's the wrong question — because high-probability buyers are Sure Things who buy anyway. The coupon has zero impact on them. Uplift models predict who is more likely to buy because of the offer — which is the exact question for an incentive targeting problem. The additional complexity is worth it because the alternative (propensity) actively loads your target group with wasted spend.

Question: How did you validate that the model was actually measuring uplift and not just propensity?

Strong Answer: Three ways: decile analysis showing higher true uplift in top deciles, calibration plot showing predicted vs. actual uplift close to the 1:1 line, and a confirmatory A/B specifically testing top-3 vs. bottom-7 deciles with reversed treatment/control splits. The confirmatory A/B was the most important validation — it proved the hypothesis with fresh experimental data, not the training data.

Question: What's the biggest lesson?

Strong Answer: The most powerful data-driven insights come from questioning the metric definition, not optimizing for it. The question "who is most likely to buy?" is a reasonable question. But it's the wrong question for incentive targeting. Asking "who is most likely to buy because of this incentive?" is a better question — and only a data-driven approach surfaces the difference. Reframing the question was the key insight. The model was just the tool.

Why This Screams Data-Driven Decision Making:

- ✔ Data revealed a fundamentally better question — not just a > better answer
- ✔ Introduced a new methodology (uplift vs. propensity) grounded in > the data, not convention
- ✔ Three-layer model validation before asking anyone to commit
- ✔ Confirmatory A/B proved the hypothesis with fresh experimental > data
- ✔ 55% spend reduction, 80% NC preserved — outcome validated the > methodology

6. COLLABORATION AT SCALE

Coupang Leadership Principle: INFLUENCE WITHOUT AUTHORITY *(secondary: Company-wide Perspective — Coupang has no standalone "Collaboration at Scale" principle)*

The principle in plain English: Large-scale collaboration at Coupang is scored under Influence without Authority — driving alignment through data and insight, in an environment where knowledge beats position — and Company-wide Perspective, which asks you to consider upstream and downstream implications and never treat a problem as

someone else's. If asked a collaboration question, the differentiator is what you did when incentives genuinely conflicted, not how many teams were involved.

How interviewers may frame this question:

- Tell me about the most complex cross-functional project you've run.
- Describe a time you aligned teams with conflicting incentives.
- How do you drive a decision when no single person owns it?
- Give me an example of being the neutral party between biased stakeholders.
- Tell me about a time you had to work with many teams under time pressure.
- How do you handle a stakeholder who's optimising for their own metric?
- Describe a situation where you had responsibility but no authority.

Story: Pricing Automation Pilot — 30+ stakeholders with competing incentives

Why this is the best fit:

Collaboration at scale means working effectively with large, diverse groups — especially when interests conflict. This story is the strongest fit because: 30+ stakeholders, fundamentally conflicting incentives, CEO-level visibility, and no formal authority. The collaboration challenge was not getting people to cooperate — it was getting people with genuinely opposing interests to trust a shared measurement framework.

2-Minute Executive Summary:

"The Pricing Automation Pilot involved 30+ stakeholders across Product, Engineering, BizFin, Data Science, Category, and Operations — each carrying significant bias on whether automated pricing would succeed. Category teams had owned manual pricing for years and were resistant to ceding control. Data Science and Product teams were biased in favor. BizFin cared about P&L. Engineering cared about tech stack. Every party had conflicting incentives. With a CEO-level Pilot at stake, the project needed one neutral body that all parties could trust. My role was to be that body — building measurement frameworks that addressed each team's specific concern, making the framework transparent before sharing with leadership, and using data to settle disagreements rather than authority. The collaboration worked not because I convinced anyone to agree — but because the framework was defensible enough that disagreement could be resolved by evidence rather than politics."

Full STAR Version

SITUATION

Flipkart's first automated pricing Pilot was directly sponsored by the CEO and CFO. Every team came in with its own stake and bias: Category teams had owned manual pricing forever and viewed automation as a direct threat to their control; Data Science and Product were naturally biased toward the product; BizFin was concerned about P&L impact; Engineering owned the tech stack. There was no neutral party. My leadership took a bet on

me to be that neutral body — the role demanded not just building the right frameworks, but the conviction to hold the objective ground as each party pushed its own narrative.

TASK

Lead the analytics workstream across 30+ stakeholders — without formal authority over any of them — and build measurement frameworks that all parties could trust, even with conflicting incentives. Make the collaboration work through the quality of the framework, not through authority.

ACTION

Step 1 — Understood each party's specific fear, not just their stated objection. Category teams were not resisting measurement — they were afraid that the framework would be designed to make automated pricing look good regardless of reality. I addressed this directly: the governance framework included a target sanity check layer that would flag inflated targets (which categories had incentive to set, since a failed Pilot would discredit automation). Making the framework protect their interests too was what earned their engagement.

Step 2 — Made the framework transparent before sharing with leadership. I shared the measurement framework with each team separately before it went to leadership — giving them the chance to raise concerns before the framework became the official standard. This made teams feel included rather than policed.

Step 3 — Used data to settle disagreements, not authority. When Data Science teams pushed their own favorable interpretation of early Pilot results, I didn't escalate or override — I proposed additional analytical checks that both parties agreed to in advance. The data then settled the disagreement.

Step 4 — Built the KVI definition collaboratively. The finding that 70-80% of the portfolio was on CI match was contentious — categories felt their KVI judgment was being overridden. I built the 3×3 KVI grid (Search PPV × Price Elasticity) collaboratively with categories, so the final definition had their input. The standard they agreed to reduced the CI-match portfolio to ~30-40% — but they owned the methodology.

RESULT

The Pilot ran successfully under CEO/CFO scrutiny. Pricing deviation reduced from **70% to 30%** of Revenue. CI-match portfolio reduced from **70-80% to ~30-40%**. Bad burn risk of **~500 Cr** quantified and mitigated. The governance framework became the standard evaluation template for every subsequent Pilot — adopted by the same teams that initially resisted it.

What Interviewers Will Probe

Question: How do you build trust with teams that are resistant to you?

Strong Answer: By demonstrating that the framework protects their interests too — not just the outcome you want to prove. When Category teams saw that the measurement

framework would also surface cases where automated pricing underperformed, they understood it wasn't designed to make one side win. A framework that can deliver bad news is a framework people trust.

Question: What do you do when a team refuses to cooperate entirely?

Strong Answer: I would escalate — not to force cooperation, but to surface the dependency to leadership. The Pilot's success required all teams to participate. Making that dependency explicit to leadership creates the right incentive for engagement without making the conflict personal.

Question: What's the biggest lesson?

Strong Answer: Collaboration at scale with conflicting interests is not about alignment — it's about building a framework that all parties can trust because it's defensible, not because it's convenient. The most durable collaborations come from shared methodology, not from shared goals.

Why This Screams Collaboration At Scale:

- ✔ 30+ stakeholders with fundamentally conflicting incentives
- ✔ No formal authority — influence through framework quality alone
- ✔ Addressed each team's specific fear, not just their stated > objection
- ✔ Made the framework transparent to all parties before sharing with > leadership
- ✔ The same teams that initially resisted ended up adopting the > framework as the standard

7. DISAGREE AND COMMIT

Coupang Leadership Principle: DISAGREE AND COMMIT *(secondary: Deliver Results with Grit)*

The principle in plain English: Constructive confrontation is essential to good decisions — leaders challenge openly when they disagree and do not compromise for social cohesion. And once a decision is made they commit fully, so that afterwards proponents and opponents are indistinguishable. Both halves are scored: the willingness to challenge, and the discipline to commit once the call is made.

How interviewers may frame this question:

- Tell me about a time you disagreed with leadership.
- Describe a time you challenged a widely held belief.
- Give me an example of when you were the only dissenting voice.
- How do you behave once a decision goes against you?
- Tell me about a time you had to hold a position through repeated pushback.
- What does “commit” mean to you in practice?

- Describe a time your finding was unwelcome.

Story: BBD Customer Shape Projection — challenging the High-TPC-first playbook

Why this is the best fit:

Disagree and Commit means: raise the objection clearly, with evidence, through the right channels — and then fully support the decision once it's made. This story is the best fit because: everyone in the room had strong incentives to stay with the High-TPC playbook. I disagreed with evidence. I went through multiple iterations to make the case. And when leadership adopted the recommendation, I didn't hold it over the teams that had resisted — I helped them execute it.

2-Minute Executive Summary:

"For years, Flipkart's BBD planning defaulted to prioritizing High-TPC customers — they drive the highest GMV per customer, and BBD GMV was the metric everyone was measured on. When I was asked to decompose the post-festive baseline shift, the data told a different story: ~80% of the baseline shift was driven by NCs and Low/Med TPC customers, not High TPC. High TPC contribution was structurally capped because they were already entrenched platform users. This finding went directly against the dominant playbook. I held the position across multiple iterations of review — backed by both the baseline shift decomposition and the long-term event impact analysis — and built a GMV-guardrailed simulator to show that the shift in customer shape did not require trading off BBD GMV. Leadership eventually adopted it. The approach delivered a 10% baseline shift in Customer DRR and became the core of Flipkart's event strategy for the following year."

Full STAR Version

SITUATION

Post-festive baseline shift — where platform Customer DRR runs ~1.5x pre-festive DRR after BBD — was one of Flipkart's most important long-term growth outcomes. For years, the dominant approach was: maximize BBD GMV by prioritizing High-TPC customers. High-TPC drives the highest revenue per customer during the event. BBD GMV is the primary metric. The entire resource allocation machinery was built around this. Every business team, category, and marketing team had incentives aligned to this playbook.

TASK

Decompose the actual drivers of post-festive baseline shift — and build the BBD customer shape projection that maximizes it while keeping BBD GMV as a guardrail. Present the finding honestly, even if it contradicts established orthodoxy.

ACTION

Step 1 — Built the baseline shift decomposition. Broke down customer DRR by TPC cohort (Low = 1-3 txns/12M, Med = 4-7, High = 8+) for pre-festive and post-festive windows. The finding: ~30% of baseline shift was led by NCs acquired during festive, ~20% by the incremental NC DRR network effect post-festive, ~30% by Low and Med TPC who

transacted during BBD. High TPC: ~20%. Their contribution was structurally capped — BBD did not change their trajectory meaningfully because their platform value was already established.

Step 2 — Disagreed clearly and with evidence. I presented the finding in multiple review iterations. Initial response was skepticism — shifting focus away from High TPC felt like a risk to BBD GMV. I held the position. Each iteration I refined the GMV guardrail logic and connected the finding more explicitly to the long-term event impact analysis (which had separately established ~30% TPC lift and ~4% churn improvement for Low/Med TPC event transactors). My manager's later feedback captured it: "problem definition, approach and presentation went through several iterations to finally reach some conclusion."

Step 3 — Built the GMV-guardrailed simulator. The critical step that removed the perceived trade-off. I projected multiple BBD customer shapes (NC-heavy, Low/Med TPC-heavy, High-TPC-heavy) with BBD GMV as a hard constraint. This showed leadership that the recommendation did not cost them their GMV number — it was a shape optimization within a GMV floor. Once the trade-off was shown not to exist, the resistance dissolved.

Step 4 — Committed fully once adopted. Once leadership adopted the NC/Low TPC framework, I didn't hold the disagreement over the teams that had resisted. I helped them set the new BBD targets — for diversity, event-to-event repeat, Low TPC acquisition — and worked with the events team to design the targeting strategy around these goals.

RESULT

Leadership adopted the GMV-guardrailed NC and Low/Med TPC customer shape as the BBD targeting framework. Diversity and event-to-event repeat taken as **explicit BBD goals for the first time**. BBD delivered **10% baseline shift in Customer DRR**, **+0.7 Mn incremental diversified customers** over plan, **+500 bps event-to-event repeat YoY**. Manager feedback: *"An analysis that led to forming the core of event strategy this year — to focus on NCs and Low TPCs throughout the year. One of a kind analysis."*

What Interviewers Will Probe

Question: How did you disagree?

Strong Answer: I challenged the assumption using the baseline shift decomposition — which was evidence, not opinion. The goal wasn't to prove the existing playbook wrong. It was to surface what was actually driving the metric everyone cared about. I presented the finding as a question: "here is what the data shows about baseline shift drivers — does this change how we should think about BBD customer shape?"

Question: What if leadership had ultimately rejected the recommendation?

Strong Answer: I would have documented the finding and proposed a controlled BBD experiment — one cohort with the NC/Low TPC shape and one with the traditional High TPC shape — to generate empirical evidence for the following year. Sometimes the right answer needs one more cycle of evidence before it lands. Disagree and Commit doesn't mean giving up — it means accepting the decision while continuing to surface evidence.

Question: What does "commit" mean in practice?

Strong Answer: It means that after the decision is made, there is no longer "my side" and "their side." There is only the team's decision. I helped the events team set the new BBD targets and design the strategy. I didn't qualify my support or hedge my commitment. The disagreement was in the room. The commitment was in the execution.

Why This Screams Disagree And Commit:

- ✔ Challenged established orthodoxy with evidence, not opinion
- ✔ Went through multiple iterations — held the position without > escalating
- ✔ Built the GMV guardrail to remove the perceived trade-off
- ✔ Once adopted, committed fully — no hedging, no "I told you so"
- ✔ The recommendation became the core of Flipkart's event strategy > for the following year

8. LEARN VORACIOUSLY

Coupang Leadership Principle: LEARN VORACIOUSLY *(secondary: Move with Urgency)*

The principle in plain English: Be hungry for the best ideas and seek them from all sources, embrace growing pains, and — explicitly — ego is the enemy: avoid rationalising mistakes and be vocally self-critical. The distinguishing feature Coupang looks for is that the learning was *sought* because the business needed it, rather than being incidental. Candour about what you got wrong on the way is part of the answer, not a risk to it.

How interviewers may frame this question:

- Tell me about something significant you learned recently and why.
- Describe a time you worked well outside your expertise.
- How do you get productive in an unfamiliar domain quickly?
- Give me an example of a mistake you made while learning something new.
- Tell me about a time you were the least experienced person in the room.
- What's an assumption of yours that turned out to be wrong?
- Tell me about learning something because the business needed it, not because you wanted to.

Story: Multi-agent RCA System — learned agentic AI, prompt engineering, LLM cost architecture from scratch

Why this is the best fit:

Learn Voraciously means: sought out learning because the business needed it, not for personal development. This story is the strongest fit because: I had zero background in agentic AI, LLM prompt engineering, or multi-model cost architecture. No one on the team had done this before. No internal playbook existed. I learned three entirely new technical

domains simultaneously — while building a production system — because the business needed it. Every failure became a learning data point. The learning was not abstract — it was immediately applied.

2-Minute Executive Summary:

"When I was asked to build Flipkart's first multi-agent RCA system for the Minutes business, I had a strong analytics and ML background but zero experience with agentic frameworks, LLM prompt engineering, or multi-model cost architecture. Google ADK had limited documentation. No internal playbook existed. I learned all three domains simultaneously while building — treating every failure as a design rule. I learned that plain-English instruction prompts caused unpredictable agent behavior because LLMs do next-token prediction and rely on structure. I learned that hybrid-model architecture (Flash/Pro/Sonnet by complexity) was the only way to make cost work at production scale. I learned that SQL semantic layers — not individual agent instructions — were the path to enterprise-scale agentic analytics. Within weeks I went from zero to a production system deployed and used in daily scrums. The learning velocity was only possible because every failure was immediately incorporated as a rule, not written off as an obstacle."

Full STAR Version

SITUATION

Flipkart Minutes needed a prompt system to identify root causes behind OPD drops in near-real-time. The solution was an agentic AI framework — a team of agents interacting to accomplish the task. This was Flipkart's first agentic-flow project. I had a strong background in analytics, statistical modeling, and ML — but agentic AI, LLM prompt engineering, cost architecture for multi-model systems, and semantic layer design were entirely new domains. Google ADK had minimal documentation. No one on the team had experience with it. The expectation was to build a production system.

TASK

Learn agentic AI architecture, Google ADK, LLM prompt engineering, multi-model cost optimization, and SQL semantic layer design well enough to build a working production system — while building it.

ACTION

Step 1 — Started with the business problem, not the technology. I didn't try to understand all of agentic AI before building. I started with the simplest component (Master routing agent), learned what was needed to build it correctly, and expanded outward. Each component built on the understanding of the previous one.

Step 2 — LLM cost architecture. Learned the 1000 tokens ≈ 750 words rule, the input vs. output token pricing differential, and how per-request cost varies ~10x across model tiers (Flash = 1x, Pro = 4x, Sonnet = 11x). Built the full cost estimate and hybrid-model rationale from first principles — this was required to secure LLM key approval before building could start.

Step 3 — Prompt engineering through failure. The first versions of instruction prompts were written in plain English. The agents produced unpredictable behavior — routing errors, output format violations, context handoff failures. I diagnosed why: LLMs do next-token prediction and rely on structure to parse complex instructions. Plain-English paragraphs are ambiguous about structure. I rebuilt every prompt using Markdown headers (roles, tools, positive triggers, negative triggers, output format, few-shot examples) — and behavior immediately stabilized. Every failure became a design rule.

Step 4 — Semantic layer design (V2). For the scalable agentic framework, I learned LookML — Looker's modeling language — and built Views, Explores, and standard aggregation mappings (e.g., Orders = COUNT(DISTINCT order_external_id), Revenue = SUM(net_amount)). I learned that the semantic layer was not just convenient — it was the architectural prerequisite for enterprise-scale agentic analytics where multiple agents need to read from a single definition of each metric.

RESULT

A working multi-agent system deployed and used in production for daily scrums and intraday diagnostics. RCA turnaround from **days to <1 hour**. Data freshness from **>2 hours to <1 hour**. The semantic layer + context store architecture became the foundation for Flipkart's enterprise agentic analytics strategy. Manager feedback: *"Built from scratch in an environment where nobody knew the right way to build."*

What Interviewers Will Probe

Question: How did you learn so quickly?

Strong Answer: I focused on first principles rather than terminology. The question I kept asking was: "what problem is this concept solving?" — not "what does this concept mean?" When I understood why Markdown structure improved LLM instruction following (because LLMs rely on structure to parse ambiguity), the rule became obvious and generalizable. Abstract terminology came after understanding.

Question: What was the hardest concept to learn?

Strong Answer: Understanding the relationship between prompt structure and agent behavior. The insight — that LLMs do next-token prediction and that plain-English paragraphs are structurally ambiguous to them — was counterintuitive. Before understanding this, I kept tweaking the content of prompts. After understanding it, I changed the structure. That was the unlock.

Question: Why does this demonstrate Learn Voraciously?

Strong Answer: Because I didn't learn these things for personal development — I learned them because the business needed a production system and nobody else was going to build it. The goal was never to become an expert in agentic AI. The goal was to solve the OPD root-cause problem for the Minutes business. Learning was the mechanism, not the objective. Coupang's principle says "sought out learning" — that's exactly what this was.

Why This Screams Learn Voraciously:

- ✔ Zero background in three technical domains — learned all three > simultaneously
- ✔ No internal playbook — built one while building the system
- ✔ Every failure immediately incorporated as a design rule, not an > obstacle
- ✔ Learning was driven by business need, not personal development
- ✔ Went from zero to production-ready in weeks

9. RUTHLESS PRIORITIZATION

Coupang Leadership Principle: RUTHLESS PRIORITIZATION *(secondary: Hate Waste)*

The principle in plain English: To focus on what you must win, you give up what you want to do — laser focus takes courage and confidence, and fast iteration and high-quality execution both begin with it. The scoring is almost entirely on what you *gave up* and whether giving it up cost you something. A story where you delivered everything is not a Ruthless Prioritization answer.

How interviewers may frame this question:

- Tell me about a time you had to say no to valuable work.
- What's the hardest prioritisation call you've made?
- How do you decide what not to do?
- Describe a time you deliberately deprioritised something important.
- Tell me about a period when everything was urgent.
- Give me an example of focusing narrowly when you were being pulled wide.
- What did you stop doing, and what did it cost?

Story: NC Analytics Charter — uplift model over funnel deep-dives and offer construct work

Why this is the best fit:

Ruthless Prioritization means choosing between good projects — not good vs. bad — and intentionally delaying valuable work because only one initiative can materially change the outcome. This story fits because: I was running multiple workstreams simultaneously within the NC analytics charter (funnel deep-dives, offer construct A/Bs, BBD preparedness, NC tracking, uplift model). All of them had value. Only one — the uplift model — had the potential to structurally change how the org deployed incentive budget. I consciously sequenced the others around it.

2-Minute Executive Summary:

"Within the NC analytics charter, I was simultaneously owning: daily NC tracking and funnel deep-dives (diagnosing NAC→NC drops), offer construct A/B experiments, BBD NC preparedness, and the uplift model build. All of them had real value. But I recognized that the uplift model was the only workstream with the potential to structurally change how the

org deployed incentive budget — turning a blunt 9 Cr/month spend into a precise 4 Cr/month spend while preserving 80% of incremental NC. I prioritized the uplift model as the first-class workstream and sequenced the others around it — running funnel deep-dives and offer A/Bs in parallel only when they didn't block the model build. The result: the uplift model shipped, ran in production for 5-6 months, and created structural cost savings. The other workstreams still got done — just not at the expense of the highest-leverage one."

Full STAR Version

SITUATION

The NC analytics charter had broad scope: tracking and governing NC metrics daily, diagnosing sudden NAC→NC funnel drops (visits → PPV → CABN → checkout → payment → orders), running offer construct A/B experiments to find the optimal coupon structure, preparing BBD NC reporting infrastructure, and building the uplift model for intelligent couponing. Each workstream had a real stakeholder need and real urgency. The existing couponing strategy was also burning ~9 Cr/month with no targeting efficiency. Leadership was watching NC AOP progress closely.

TASK

Deliver across the full NC charter while ensuring the highest-leverage workstream — the uplift model — actually shipped rather than being perpetually deprioritized in favour of more immediate asks. The challenge was not a lack of work. The challenge was resisting the pull of urgent-but-lower-leverage tasks in favour of the important-and-highest-leverage one.

ACTION

Step 1 — Evaluated each workstream on a single criterion: "If this ships, does it structurally change the business?" The uplift model was the only one that passed. Funnel deep-dives diagnosed drops but didn't prevent them structurally. Offer A/Bs optimized the margin; the uplift model optimized the targeting. BBD preparedness was important but not structurally differentiated.

Step 2 — Made the uplift model the first-class workstream. All other workstreams were sequenced around it: I ran the clean randomized A/B (required for the uplift model training data) first, which also gave the team baseline NC metrics — meaning the tracking workstream got value as a byproduct of the model setup.

Step 3 — Deliberately delayed lower-leverage exploratory work. Several offer construct experiments and funnel deep-dives were queued rather than acted on immediately. I communicated this tradeoff explicitly to stakeholders — "these will be picked up after the model ships" — rather than quietly deprioritizing them.

Step 4 — Built the model to be reusable and low-maintenance once deployed. Working with the Data Science team, the model was pushed to production to score the NAC base monthly

— so once it shipped, it required minimal ongoing analytical effort, freeing bandwidth for the other workstreams.

Step 5 — Ran offer construct A/Bs on the model's decile output once it was in production. By sequencing the offer experiments after the uplift model, the A/Bs ran on the already-identified top-3-decile Persuadables — making the offer construct experiments more efficient and precise than if they had run on the full base.

RESULT

Uplift model shipped and ran in production for **5-6 months**. Monthly couponing spend reduced from **9 Cr to 4 Cr (~55%)** while preserving **~80% of incremental NC** at **CAC of 200 vs. 300**. Offer construct A/Bs ran on the optimized target group and disproved the "deeper discount = lower RPC" assumption. BBD NC preparedness was delivered on time. Funnel deep-dives resumed post-model deployment. All workstreams were eventually completed — in the right order.

What Interviewers Will Probe

Question: Why was this ruthless rather than normal prioritization?

Strong Answer: Because I wasn't choosing between a good project and a bad one. I was choosing between several good projects — all of which had real stakeholder demand. The ruthless part was intentionally delaying valuable work because only one workstream could structurally change how the org deployed incentive budget. That required actively saying "not yet" to legitimate asks, not just "no" to low-value ones.

Question: What projects did you explicitly delay?

Strong Answer: Several funnel deep-dives (NAC→NC drop diagnoses) and offer construct experiments. They were useful — and were eventually completed — but none of them had the same structural leverage as the uplift model. Delaying them by 6-8 weeks had minimal cost; delaying the uplift model by 6-8 weeks would have cost another 9 Cr/month at the inefficient CAC.

Question: What's the biggest lesson?

Strong Answer: Prioritization is not about what you do first — it's about what you protect. The uplift model needed protected time to be built correctly. Every interruption for a funnel deep-dive or offer A/B was time not spent on the model. Ruthless prioritization means protecting the highest-leverage workstream from the tyranny of the urgent.

Why This Screams Ruthless Prioritization:

- ✔ Chose between several good projects — not good vs. bad
- ✔ Sequenced workstreams so the highest-leverage one shipped first
- ✔ Explicitly communicated delays to stakeholders rather than quietly > deprioritizing

- ✔ Built the model to be low-maintenance post-deployment — > protecting future bandwidth
- ✔ All workstreams eventually completed — in the right order

10. DELIVER RESULTS WITH GRIT

Coupang Leadership Principle: DELIVER RESULTS WITH GRIT *(secondary: Aim High and Find a Way)*

The principle in plain English: Coupang is explicit that grit is not effort — they do not confuse long hours or effort with results. Leaders deliver impact in a timely fashion and with high quality, finish what they start, work around dependencies, and never let excuses stop them. So the scoring is on whether you *finished*, and whether the recovery involved changing something concrete rather than persisting harder.

How interviewers may frame this question:

- Tell me about the hardest thing you've pushed to completion.
- Describe a time you faced sustained resistance and still delivered.
- Give me an example of working around a blocking dependency.
- Tell me about something you finished that would have been easy to abandon.
- How do you keep momentum when a project stalls repeatedly?
- Describe a time you held a position for months.
- Tell me about a result you're proud of that took real endurance.

Story: Pricing Automation Analytics — held the analytical stance across 30+ stakeholders for months

Why this is the best fit:

Grit is not about working hard. It is about continuing to push when the path is blocked, the resistance is real, and it would be easier to reduce scope or stop. This story fits because: the Pricing Automation Pilot had 30+ stakeholders with fundamentally opposing incentives. Category teams actively resisted. Data Science teams pushed favorable narratives. At multiple points it would have been easier to soften the measurement framework or declare the Pilot successful based on partial evidence. I didn't. The project ran for months under CEO-level scrutiny, and I held the neutral analytical stance throughout — not because it was comfortable, but because it was right.

2-Minute Executive Summary:

"The Pricing Automation Pilot was Flipkart's first attempt at automated pricing at scale — CEO and CFO directly involved, 30+ stakeholders with conflicting interests, and an outsourced product whose ROI was being tested for the first time. My role was to be the neutral governance body — building the measurement framework and holding the objective ground as each party pushed its own narrative. This required months of sustained effort: driving instrumentation across two product teams who hadn't collaborated before, building a governance framework that every party tried to challenge, diagnosing ASP

deviation as the primary GMV driver when Data Science teams were pointing elsewhere, and designing the feedback loop that ultimately reduced Pricing deviation from 70% to 30% of Revenue. At multiple points, it was easier to declare the Pilot successful and move on. I didn't. The grit was in staying with the problem — and the true picture — until the system actually improved."

Full STAR Version

SITUATION

Flipkart had made a substantial investment in an outsourced automated pricing product. The CEO approved a Pilot with the mandate to test whether automated pricing was better than or at par with manual pricing. Every party had a stake and a bias: Category teams had owned manual pricing for years and were resistant; Data Science and Product were biased in favor of the product; BizFin was focused on P&L impact. The Pilot needed one neutral party with both the analytical rigor to surface the true picture and the conviction to hold it even as other parties pushed their narratives.

TASK

Lead the analytics workstream across months of Pilots — building the instrumentation, the governance framework, and the diagnostic deep-dives — and hold the neutral analytical stance even when it was uncomfortable. The deliverable was not a single analysis but a sustained operating system: measurement, governance, diagnosis, and product improvement across every Pilot the org ran.

ACTION

Step 1 — Drove instrumentation when teams wanted to skip it. The initial response from both product teams was: "let's start the Pilot and figure out measurement later." I held the position that instrumentation was the prerequisite, not an overhead — and ran multiple alignment rounds with Product, Engineering, and Program teams to get it done before the first Pilot launched. This was not a popular stance in the short term.

Step 2 — Built the governance framework under sustained challenge. Category teams challenged the target sanity check layer (they had incentive to set high bars). Data Science teams challenged the ASP adherence layer (it showed their product was underperforming). I maintained each layer — not because I enjoyed the conflict, but because removing any layer would have made the framework indefensible.

Step 3 — Diagnosed ASP deviation when the prevailing narrative pointed elsewhere. When early Pilots showed mixed GMV performance, the dominant explanation from Data Science teams was "category mix" and "demand variability." The instrumentation I had built showed something different: 70% of GMV had ASP deviation of more than ±2% from model-recommended ASP. I held this diagnosis against significant pushback — running multiple analytical checks that both parties agreed to in advance so the data could settle the disagreement.

Step 4 — Designed and advocated for the feedback loop. Once the root cause (OI's BCD projection error using coarse historical heuristics) was established, I designed the granular adoption model and live feedback mechanism — and pushed for it to be built into the standard Pilot cadence. This required another round of alignment with a Data Science team that was initially resistant to the implication that their model needed improvement.

Step 5 — Held the KVI definition work through months of Category resistance. The finding that 70-80% of the portfolio was on CI match — severely limiting the Optimizer's scope — was deeply uncomfortable for Category teams. I built the 3×3 KVI grid collaboratively, gave Category teams input into the methodology, and eventually got the CI-match portfolio reduced to ~30-40%. This took months of sustained engagement, not a single meeting.

RESULT

Pricing deviation reduced from **70% to 30%** of Revenue. CI-match portfolio reduced from **70-80% to ~30-40%**. Bad burn risk of **~500 Cr** quantified and mitigated at BBD scale. The governance framework became the standard evaluation template for every subsequent Pilot. The same teams that initially resisted ended up relying on the framework to understand their own pricing performance. Leadership feedback: *"The role demanded not just building the right measurement frameworks and deep dives, but also the conviction to hold the stance, not give up easily, and stay firm on the true picture even as other parties pushed their own narratives."*

What Interviewers Will Probe

Question: Why was this grit rather than just a difficult project?

Strong Answer: Because at multiple points the path of least resistance was available — and I chose not to take it. I could have softened the governance framework to reduce Category resistance. I could have accepted the Data Science team's "demand variability" explanation. I could have declared early Pilots successful based on partial evidence. Grit was choosing the harder path — maintaining the full framework, holding the diagnosis, pushing for the feedback loop — because the easier path would have produced an incomplete picture.

Question: How did you sustain motivation over months of resistance?

Strong Answer: By staying focused on the outcome rather than the resistance. The question I kept returning to was: "If the ASP deviation problem isn't fixed, does the Pilot actually work at scale?" The answer was no. That made the resistance feel like a problem to solve, not a reason to stop.

Question: What's the biggest lesson?

Strong Answer: Grit in analytics is not about working harder. It's about maintaining the rigorous standard when every party around you has an incentive for you to lower it. The moment you soften the framework to reduce friction, you lose the credibility that makes the framework valuable. The neutral analytical stance is only worth anything if it's maintained under pressure.

Why This Screams Deliver Results With Grit:

- ✔ Drove instrumentation when teams wanted to skip it — not a > popular stance
- ✔ Maintained governance framework layers under sustained challenge > from multiple parties
- ✔ Held the ASP deviation diagnosis against significant pushback — > let data settle it
- ✔ Pushed for the feedback loop through months of Category and Data > Science resistance
- ✔ Pricing deviation from 70% to 30%, 500 Cr bad burn mitigated — > results matched the grit

11. SIMPLIFY

Coupang Leadership Principle: SIMPLIFY *(secondary: Hate Waste)*

The principle in plain English: Complexity is the enemy of scale, speed and customer experience, and leaders are fanatical about simplifying what they touch. The emphasis is on simplifying the *thing*, not the explanation — so removing steps, consolidating definitions, or eliminating repeated manual work scores higher than communicating a complex thing well. The related bar is Hate Waste: do more with less, and save costs rather than cut them.

How interviewers may frame this question:

- Tell me about a time you simplified something complicated.
- Describe repeated manual work you eliminated.
- How have you reduced complexity for other people?
- Give me an example of consolidating something fragmented.
- Tell me about a time you made a process faster by removing steps rather than adding resources.
- Describe something you automated and what it freed up.
- What's the most complexity you've taken out of a system?

Story: Multi-agent RCA System for Flipkart Minutes — simplifying how the org diagnoses OPD drops

Why this is the best fit:

Simplify doesn't mean "explain something simply." It means removing complexity from a system — so that something that required skilled manual effort can now happen automatically, faster, and at greater scale. This story fits because: before the multi-agent RCA system existed, diagnosing an OPD drop required analysts to manually pull metrics, run SQL queries, traverse an issue tree, and write an RCA — taking hours or days. The system compressed this into minutes, made it available to non-analysts, and delivered it at City × Category × Vertical × Brand granularity that no human could practically achieve in real time. The complexity wasn't explained away. It was engineered away.

2-Minute Executive Summary:

"Flipkart Minutes sends an hourly achievement mailer every 40th minute with input and output metrics. Every mailer triggered a stream of questions about why metrics were underperforming — and Analytics and Business time was consumed figuring out what was happening rather than acting on it. In quick commerce, the window to act is minutes, not hours. I built a multi-agent RCA system (Google ADK, Claude) that takes a natural-language query and returns either a metric pull or a full root-cause analysis — executive summary plus detailed issue-tree breakdown — at City × Category × Vertical × Brand granularity, in under an hour. What previously required a skilled analyst, SQL access, an issue tree, and hours of work now takes a prompt. The system is used in daily scrums and intraday diagnostics. The complexity of the diagnosis process was not explained away — it was built away."

Full STAR Version

SITUATION

Flipkart Minutes (Quick Commerce) chases Orders Per Day (OPD) as its key metric. An hourly achievement mailer triggered constant questions: "Why did orders drop in Bangalore?" "What's driving the OOS in Electronics?" "Is the low AOV this hour a mix issue or a pricing issue?" Every question required an analyst to: pull the relevant metrics from BQ, traverse the issue tree (orders → city → category → input metrics: pricing, OOS, speed, SLA breach, surge visits), run contribution analysis or Shapley attribution, and write an executive summary. In Quick Commerce, actionability is real-time — a drop identified 4 hours later is a drop you couldn't respond to. The manual RCA process was the bottleneck.

TASK

Build a system that simplifies the RCA process — not by making it easier to explain, but by removing the manual analytical work entirely — so that anyone (analyst or business user) could get a full root-cause diagnosis in natural language within minutes.

ACTION

Step 1 — Identified what complexity to remove. The existing process had five layers of manual work: (1) knowing which metrics to pull, (2) writing the SQL, (3) running the query and formatting the output, (4) traversing the issue tree and deciding which branches to follow, (5) synthesizing the findings into an executive summary. The system needed to remove all five.

Step 2 — Designed the multi-agent architecture to mirror the human RCA workflow. Master agent routes the query. Metric Pull agent does NLQ → SQL → growth vs. baseline with RAG coding. RCA agent traverses the full issue tree — at each node it calls the NLQ→SQL agent, compares output to threshold, and decides whether to traverse to a child or sibling node. Synthesis agent reads the full tree traversal from the context variable and produces an executive summary + detailed breakdown.

Step 3 — Built the semantic layer to make SQL generation reliable at scale. Standard aggregation mappings (Orders = COUNT(DISTINCT order_external_id), Revenue = SUM(net_amount)), schema discovery, dimension-filter resolution via fuzzy matching, shared query patterns. This was the simplification layer underneath the agents — without it, the system would have been unreliable.

Step 4 — Built attribution tools to handle analytical complexity the agents couldn't solve with SQL alone. Contribution analysis tool: surfaces cities contributing ≥10% to the order drop for further traversal (filtering 80+ cities to the relevant subset). Shapley attribution tool: isolates category mix change vs. actual AOV drop per category — a calculation that would take an analyst 30+ minutes to do manually.

Step 5 — Moved from GSheet (V1) to BigQuery (V2) to achieve NRT granularity. V1 proved the concept. V2 made it operationally useful — freshness from 2+ hours to <1 hour, and City × Category × Vertical × Brand granularity that no GSheet-based system could achieve within the 10M cell limit.

RESULT

RCA turnaround from **hours/days to <1 hour**. Data freshness from **>2 hours to <1 hour**. Actionability at **City × Category × Vertical × Brand** granularity — previously impractical manually. System adopted for **daily scrums and intraday diagnostics** by Business and Analytics teams. The system is used in daily scrums to drive actions on pricing, serviceability, and selection — the inputs that drive OPD. The complexity of the manual RCA process was not simplified. It was eliminated.

What Interviewers Will Probe

Question: What was the hardest complexity to remove?

Strong Answer: The issue-tree traversal logic. The agent had to make decisions at each node — does the data warrant going deeper into this branch, or should I move to a sibling? That decision-making logic had to be encoded in the agent's prompt and validated through many failed iterations where the agent either over-traversed (going deep on branches with no signal) or under-traversed (stopping too early). Getting the traversal logic right was the hardest simplification.

Question: How did you ensure the simplified output was still accurate?

Strong Answer: Two ways: the semantic layer ensured SQL generation was consistent and correct (same aggregation logic every time), and the attribution tools ensured mathematical operations were done in Python (not by the LLM, which can't do reliable arithmetic). The simplification was in the interface — natural language in, structured RCA out. The analytical rigor underneath was preserved.

Question: What's the biggest lesson?

Strong Answer: Simplification at scale requires standardization first. Before the system could simplify the RCA process, the RCA process had to be standardized — the issue tree had to be defined, the aggregation mappings had to be documented, the synthesis template

had to be fixed. You can't automate chaos. Simplification requires turning a complex process into a repeatable one before you can eliminate the manual steps.

Why This Screams Simplify:

- ✔ Removed five layers of manual analytical work — not explained > them away
- ✔ Natural language in, full root-cause analysis out — available to > non-analysts
- ✔ City × Category × Vertical × Brand granularity that was previously > impractical manually
- ✔ Semantic layer ensured the simplification didn't sacrifice > analytical rigour
- ✔ Hours/days to <1 hour — the complexity of diagnosis was > engineered away

12. MOVE WITH URGENCY

Coupang Leadership Principle: MOVE WITH URGENCY *(secondary: Wow the Customer)*

The principle in plain English: Urgency is a sense of crisis; inaction is a threat to survival; learn by doing and don’t delay decisions to seek a perfect solution. In the strongest version of this answer, the urgency is not self-imposed — something was actively going wrong while you decided, and the speed of the response is what limited the damage.

How interviewers may frame this question:

- Tell me about a time you had to act immediately.
- Describe a time speed of response materially changed the outcome.
- Give me an example of spotting something wrong and acting before you had the full picture.
- Tell me about a time delay would have been costly.
- How do you respond when you find an anomaly you can’t fully explain yet?
- Describe the fastest you’ve ever turned an insight into an action.
- Tell me about a time you prevented a loss.

Story: Free Shipping Fraud Detection — saved 2 Mn burn within hours

Why this is the best fit:

Move With Urgency means: don't wait for perfect information or perfect systems — act with what you have, learn, and update. This story is the strongest fit because: the fraud was active. Every hour of delay meant more burn. I had no pre-built framework for seller fraud detection. I had to diagnose, confirm, and quantify the impact within hours — using only the data I had available — and get the finding to the business team fast enough for them to block the payments before the burn compounded. Speed was not just a preference. It was the entire value.

2-Minute Executive Summary:

"While monitoring Free Shipping program metrics during a sale day, I spotted a suspicious spike in BGM (Bags, Garments & More) UPC — units per customer — that was significantly above the FS DRR average and had caused a ~2 Mn spike in program burn. I had no pre-built fraud detection framework. I used whatever data was available — order-level UPC distributions, seller-level transaction patterns, product listing prices, shipping fee structures — and within 3 hours identified ~10 sellers exploiting the Free Shipping construct: selling products listed at ₹2 with a ₹316 shipping fee, achieving UPC of 30 on single accounts. I quantified the burn, identified the sellers, and delivered the finding to the business team in time for them to block the payments. 2 Mn of burn was saved. This happened twice on different occasions — the same urgency, the same turnaround."

Full STAR Version

SITUATION

Flipkart's Free Shipping (FS) program offers customers free shipping on qualifying orders — a significant incentive that directly impacts the program's cost structure. During sale days when FS was active, I was responsible for monitoring incremental NC and CAC metrics for the FS construct. One such monitoring session flagged an anomaly: BGM UPC had spiked from the FS DRR average of 1.16 to 1.3 — a sharp deviation that had translated into a ~2 Mn spike in program burn within a single day.

TASK

Diagnose the root cause of the UPC spike and burn anomaly — without a pre-built fraud detection framework — fast enough for the business team to take action before more burn accumulated. The constraint was time: the burn was active.

ACTION

Step 1 — Accepted the constraint and started with what was available. I had no fraud detection model, no pre-defined seller abuse rules, no historical benchmark for "normal" UPC distribution under FS. I started with the simplest question: "What does UPC = 30 look like at the order level?" A UPC of 30 means a single account ordered 30 items in a single FS day — statistically extreme.

Step 2 — Moved through the data layer by layer. Order-level: confirmed the spike was concentrated in a small number of accounts with very high UPC (20-30). Seller-level: traced those orders to ~10 sellers. Product-listing level: found the pattern — products listed at ₹2 with a ₹316 shipping fee. The Free Shipping construct was covering the ₹316 fee, making each "purchase" effectively free for the buyer and highly profitable for the seller per unit shipped. The fraud mechanism was clear within the first 2 hours.

Step 3 — Quantified the burn before escalating. I calculated: number of fraudulent orders × (shipping fee absorbed by FS per order) = burn per seller, summed across the ~10 sellers. This gave the business team a specific number (2 Mn) and a specific set of seller IDs to act on — not a vague "there might be fraud."

Step 4 — Delivered the finding with full context in 3 hours. The business team received: the fraudulent seller IDs, the fraud mechanism (₹2 products + ₹316 shipping fee exploiting FS), the burn quantification (2 Mn), and the recommendation (block their payments). They acted on it immediately.

Step 5 — This happened twice. On a second occasion during a different FS sale, the same pattern emerged with a different set of sellers. The second detection was faster because the first had established the diagnostic pattern — but still required the same urgency and the same willingness to act on directionally correct analysis before having a perfect framework.

RESULT

2 Mn burn saved on each of two occasions — within 3 hours of the spike being flagged. The finding also triggered a broader conversation between the M3 business team and the TNS (Trust & Safety) team about closing the multi-unit fraud loophole in the FS construct — translating an urgent, point-in-time diagnosis into a structural fix.

What Interviewers Will Probe

Question: How did you know the finding was reliable enough to escalate in 3 hours?

Strong Answer: I didn't have certainty — but I had directional confidence and a clear mechanism. The pattern was unambiguous: products listed at ₹2 with ₹316 shipping fees, UPC of 30, concentrated in ~10 sellers. I escalated with explicit caveats: "here is the mechanism, here are the sellers, here is the estimated burn — I recommend blocking payments while further verification runs." Waiting for perfect certainty would have cost the additional burn accumulated during the investigation.

Question: What if you'd been wrong?

Strong Answer: The cost of a false positive — temporarily blocking a seller for a few hours while investigation continued — was much lower than the cost of a false negative — 2 Mn of continued burn. When the asymmetry between false positive and false negative costs is that large, acting on directional confidence is the right call. I made that asymmetry explicit in the escalation.

Question: What's the biggest lesson?

Strong Answer: Urgency is a choice about what level of certainty justifies action. In most analytical work, you wait for statistical significance. In active-loss situations — fraud, system failures, real-time events — the standard for action is "directional confidence plus clear mechanism," not "statistical certainty." The skill is knowing which situation you're in.

Why This Screams Move With Urgency:

- ✔ No pre-built framework — started with what was available and > moved layer by layer
- ✔ Full diagnosis, burn quantification, and seller identification > within 3 hours

- ✔ Escalated on directional confidence, not perfect certainty — > because delay had a real cost
- ✔ 2 Mn burn saved on each of two occasions — speed was the entire > value
- ✔ Point-in-time urgency translated into structural fix — TNS team > closed the loophole

Why Google?

What excites me about Google is the opportunity to work on problems that operate at a massive scale while sitting at the intersection of analytics, product, and AI-driven systems. A lot of the work I've done recently has been around building scalable analytical systems and improving how teams derive insights using automation and AI workflows — and Google is uniquely positioned in terms of data scale, engineering maturity, and AI innovation. The kind of problems I enjoy most are highly ambiguous, cross-functional, and infrastructure-heavy, especially where analytics directly shapes product and strategic direction. That's exactly what this role offers.

Why Coupang?

~100 words, about 45 seconds. Learn the shape, not the wording.

"Coupang competes on selection, price and speed — and those happen to be the three problems I've actually worked on. Local fulfilment and delivery speed, automated pricing at 10M+ listing scale, and customer cohort economics. At Flipkart those were three separate charters; here they're one thesis.

The second thing is that Coupang treats automation and AI as a lever on both customer experience and cost to serve. That's exactly the bet I'm making — I lead agentic analytics for Flipkart Minutes and own the semantic layer behind it.

And honestly, the leadership principles read like how I already work. Hate Waste in particular."

If asked: why leave Flipkart *for Coupang* specifically?

"Not because Flipkart ran out of problems. Two reasons. One is stage — I want to work on the same problems in a business where speed and selection *are* the product, not one input among many. The other is that Coupang is running the same playbook at three points on the curve at once: a mature Product Commerce business, Eats past its investment cycle, Taiwan mid-build. I've only ever worked businesses at one stage. Seeing the same model at three stages and working out what generalises is a different kind of education."

If asked: what would you focus on in your first 90 days?

"Listen first, then instrument. My lesson from Pricing Automation is that with multiple teams and competing narratives, you can't attribute anything until you can trace it end to end — the most useful thing I did there was build the data layer nobody owned, before running any analysis. So I'd spend the first weeks on which decisions get made weekly,

what evidence they rest on, and where the measurement genuinely can't answer the question. Then fix the biggest of those gaps rather than rushing out a first analysis. Slower to show output, and it's what makes every later answer defensible."

If asked: what concerns or questions do you have?

"Two. First, how much of the analytics team's time goes into recurring reporting versus decisions — if it's reporting-heavy, the highest-value thing I could do is probably semantic infrastructure and automation before any individual analysis, and I'd want to know if there's appetite for that. Second, the split between near-real-time diagnostic work and deeper structural analysis, and how the second is protected from the first. In my experience the urgent crowds out the structural unless someone defends it deliberately."

Delivery notes

- **Three beats only: problem set, AI as an operating lever, culture.** If they want depth on any of them, they'll ask. Don't pre-empt.
- **"Three separate charters; here they're one thesis" is the whole answer.** Say it slowly.
- Lead with the selection / price / speed mapping. It's the strongest thing you have, because it isn't enthusiasm — it's three projects that map one-to-one onto Coupang's stated strategy.
- **Name one principle, not five.** Listing them reads as preparation. Naming Hate Waste and then producing the ₹9 Cr to ₹4 Cr story on the follow-up reads as fit.
- Don't mention relocation, growth trajectory or the Korean market unless asked. Those are research, not reasons.
- Do not raise the 2025 data incident, the regulatory fine, or the margin compression unprompted. If an interviewer raises them, the useful register is analytical rather than reassuring — cohort recovery, WOW membership rebuild, and the distinction between a temporary dislocation and a structural change are all things you have genuinely modelled at Flipkart in the form of baseline shift and reactivation work.
- Have a view ready on Coupang's quick-commerce-adjacent businesses. You currently lead analytics for Flipkart Minutes, which is the closest possible analogue to Eats and Rocket Now, and that is a more specific credential than "e-commerce experience."

Why are you leaving Flipkart?

I've had an incredibly rewarding 10-year journey at Flipkart across customer growth, event analytics, pricing automation, and most recently GenAI and agentic analytics. What I'm looking for now is an opportunity to work on even larger-scale platform problems with broader global complexity. I've realized that the problems I enjoy most sit at the intersection of analytics, AI, and product strategy — and Google is a natural next step for exactly that.

30-Second "Why Agoda?" Answer

I'm excited about Agoda because it's a true data-driven marketplace at global scale, where analytics directly drives decisions around supply, pricing, and partner performance. This role stands out because it's not just reporting insights — it's about shaping marketplace outcomes through experimentation, modeling, and strategic decision-making. Agoda's pace, ownership culture, and focus on testing and iterating quickly strongly match how I like to work. Strong analytics translating directly into measurable business impact is exactly what I'm looking for.

Tell me about your biggest strength.

One of my biggest strengths is **structured problem solving under ambiguity**. Many of the projects I've worked on did not have predefined frameworks — whether it was diagnosing why Cluster Fulfillment had been flat for 8 weeks, building an uplift model from scratch to replace an inefficient couponing strategy, or designing the first multi-agent RCA system at Flipkart with no internal playbook. I enjoy breaking ambiguous problems into measurable components, identifying hidden patterns, and translating findings into practical business decisions.

Tell me about your weakness.

Earlier in my career, I had a tendency to over-optimize analyses before socializing insights with stakeholders. I used to believe the analysis had to be nearly perfect before starting discussions. Over time, especially while working on cross-functional projects like the Pricing Automation Pilot, I realized that early stakeholder alignment is often more valuable than analytical perfection — it prevents downstream misalignment and speeds up decision-making. Now I try to socialize directional thinking earlier while still maintaining rigor in final execution.

Describe your leadership style.

My leadership style is **data-driven and collaborative**. I prefer aligning stakeholders through clarity, structured reasoning, and measurable tradeoffs rather than authority. Most of the projects I've led — the Pricing Automation Pilot (30+ stakeholders), the BBD control tower (20+ teams), the agentic RCA system — involved influencing teams I did not directly manage. I've learned that sustainable leadership comes from making problems understandable, acknowledging competing incentives, and building trust through transparency. I also strongly believe in solving systemic bottlenecks rather than repeatedly solving individual issues manually — which is why I lean toward building frameworks, simulators, and governance mechanisms that outlast the specific project.

13. AIM HIGH AND FIND A WAY

Coupang Leadership Principle: AIM HIGH AND FIND A WAY *(secondary: Hate Waste)*

The principle in plain English: Coupang says goals are usually set to what seems possible within existing conditions — and leaders should have the courage to aim for jaw-dropping results, work backwards, and then drive whatever change is necessary to deliver the “wow.” Two things get scored. Was the ambition *yours*, or handed to you? And when the existing conditions made it impossible, did you change the conditions rather than trim the goal?

How interviewers may frame this question:

- Tell me about the most ambitious goal you've set for yourself.
- Describe a time you aimed at something that looked unachievable with what you had.
- Give me an example of working backwards from an outcome rather than forwards from your constraints.
- Tell me about a time the data you needed didn't exist.
- How do you set a target when there's no benchmark?
- Describe something you built that other teams later adopted.
- Tell me about a time you refused to accept the existing way of doing something.

Story: Intelligent Couponing — setting an 80%-of-outcome-at-50%-of-cost target, then manufacturing the data to hit it

Why this is the best fit:

Nobody asked me to halve the couponing budget. The AOP target was ~30% YoY NC growth, and the existing method was delivering against it — inefficiently, but successfully. The ambition here was entirely self-set, and I set it as a specific number before I had any idea whether it was achievable: *what is the ~30% of the target cohort that brings ~80% of incremental NC at ~50% of overall cost?* That is the jaw-dropping formulation, and I chose it deliberately over "improve targeting efficiency," because a vague goal produces a marginal answer.

The existing conditions then made it flatly impossible. Uplift cannot be observed — for any individual customer you see either what they did with a coupon or what they did without one, never both — so the dataset needed to answer my question did not exist anywhere in Flipkart. The org also had no experience with the methodology. So the work backwards ran: to hit the target I need per-user uplift, which needs CATE estimation, which needs a model, which needs a treated-and-untreated sample with a rich feature set captured *while* the treatment runs — which means running a randomised experiment purely to manufacture training data before any modelling could begin. Changing the conditions was the project.

2-Minute Executive Summary:

"Within the NC charter we were leaning on incentive-led acquisition to hit a ~30% YoY NC growth AOP while the product-led roadmap landed. The existing couponing method worked — target the whole NAC base, measure ATE, report incremental NC — so nobody was asking for it to change. I set a target of my own instead, and I made it deliberately aggressive and specific: find the ~30% of the cohort that brings ~80% of incremental NC at ~50% of the cost. Then I worked backwards, and the first thing I found was that the goal was unachievable with anything the org had, because the data required to answer it doesn't exist by nature — you can never observe both the treated and untreated outcome for the same customer. So rather than trimming the ambition, I changed the input conditions: I ran a clean randomised A/B specifically to manufacture the training data, capturing every AID with a Test/Control flag alongside a rich engagement and demographic feature set while

the experiment ran, and gated the whole project on ATE being statistically significant, because a near-zero ATE would have made the target meaningless. Then the S-Learner XGBoost model scoring each user twice to get per-user uplift, validated three ways — decile analysis on true uplift, calibration against the 1:1 line, and normalised cumulative gain at the ~80% AUC industry benchmark. The result came in slightly ahead of the target I'd set: monthly spend fell from ₹9 Cr to ₹4 Cr, about 55% rather than 50%, with ~80% of incremental NC preserved exactly as aimed and CAC down from ~₹300 to ~₹200. The part I'm proudest of is what happened afterwards — the uplift framework was picked up by other teams across Analytics as the standard approach to incentive-based targeting, so the ambition outgrew the project it was set for."

Full STAR Version

SITUATION

The NC charter chased roughly 30% YoY growth, with two levers: product-led acquisition, which is more sustainable but needed 4–5 months to land, and incentive-led campaigns. We had made a deliberate call to lean on incentives in the interim.

The important context for this principle is that **nothing was failing.** The existing method — take the whole NAC base, split Test and Control, run a TD coupon, measure ATE, report incremental NC at some RPC and CAC — was delivering. Leadership had always evaluated couponing this way and the working assumption was that more targeting meant more reach meant more NC. There was no crisis, no escalation, and no mandate to rebuild anything. Which is precisely the condition Coupang describes: the goal had been set to what seemed possible within existing conditions, and everyone was comfortable there.

TASK

I set the target myself, and I chose the aggressive formulation over the safe one. The safe version was "improve coupon targeting efficiency," which would have produced a modest, defensible gain. The version I committed to was a specific, uncomfortable number: **find the ~30% of the target cohort delivering ~80% of incremental NC at ~50% of overall cost.** Half the budget, four-fifths of the outcome. I had no evidence at the point of setting it that such a cohort existed — if the offer changed everyone's behaviour uniformly, it wouldn't have.

ACTION

Step 1 — Reframed the question, because the target was unreachable under the existing framing. The existing method asks how many customers a coupon produced. Under that framing the target is not just hard, it is unanswerable — a campaign that produces customers looks successful whether or not half the spend was wasted, so there is no quantity in the existing measurement framework that could ever reveal the ~50% saving I was aiming at. The reframe was from *who is most likely to buy* to *who is most likely to buy because of the coupon*. That is the move that made an impossible target merely difficult. The four-segment model — Persuadables, Sure Things, Sleeping Dogs, Lost Causes — made it concrete, and I explicitly ruled out the intuitive alternative of a propensity

model, because targeting high probability-to-convert users loads the group with Sure Things and cannot in principle deliver the target.

Step 2 — Worked backwards from the target to the input I didn't have. The chain: to allocate budget to the top 30% I need per-user uplift → per-user uplift means CATE, the difference between a customer's conversion probability treated and untreated → that requires a model trained on a population where both treated and untreated outcomes are represented across all customer types → which requires a randomised experiment with a rich feature set captured during treatment. At the bottom of that chain was a hard fact: **the data did not exist, and could not be found.** Uplift is fundamentally unobservable at the individual level. No amount of digging through historical campaign data would have produced it.

Step 3 — Changed the conditions rather than the goal. This is the "find a way" step. Instead of scaling the ambition down to what historical data could support, I ran an experiment for the sole purpose of manufacturing the missing dataset: the NAC base split Test and Control, Test receiving an x% off up to ₹z coupon. While it ran we captured each AID with its Test/Control flag plus engagement signals — add-to-cart clicks, banner clicks, bounce rate, average session length, average scroll length, PPVs per visit, PPVs by BU — and dimensions including gender, city tier, zone and age, then appended the converted/not-converted flag on conclusion. That meant asking the business to run a campaign whose immediate purpose was training data rather than customers, which is a real ask and only justifiable against a target worth the cost.

Step 4 — Gated the ambition honestly. On experiment conclusion I computed ATE and confirmed statistical significance before proceeding. This was a genuine kill-switch: a near-zero ATE would have meant the offer changed nobody's behaviour materially, in which case no Persuadable cohort exists and my target was arithmetically unreachable. Aiming high is not the same as refusing to check whether the aim is possible, and I would rather have abandoned it at that gate than spent months chasing a number the population could not support.

Step 5 — Built and validated to a standard that would survive being unfamiliar. An S-Learner XGBoost classifier over all features plus the treated indicator, scored twice per user — once with the flag at 1, once at 0 — with the difference giving per-user uplift. Because uplift models cannot be validated like classifiers, three group-level checks: decile analysis computing *true* uplift (actual Test minus Control conversion) per bin and confirming it rose in the top deciles; a calibration scatter of predicted against true uplift sitting close to the 1:1 line; and a normalised cumulative gain chart, where ~50% AUC is random and ~80% is the industry benchmark for a strong uplift model — which ours reached.

Step 6 — Proved it at production scale before claiming the target. Scored the full NAC base, created deciles, and ran a confirmatory A/B — 90/10 Test/Control on the top three deciles and 10/90 on the bottom seven — to demonstrate the top-3 ATE was significantly higher and that RPC and CAC were genuinely optimised. The February experiment

confirmed it, and the model went into production with the Data Science team, scoring the NAC base monthly for five to six months.

Step 7 — Pushed the ambition past the original scope. Having hit the target, the obvious next question was whether the same decile structure could answer the *offer* question, not just the *audience* question. It could: A/B experiments on the top deciles with T1/T2/T3 and C1/C2/C3 cells testing 30% off up to ₹50 against 10% off up to ₹100 against a ₹9 store construct, with RPC decomposed into incremental revenue over incremental burn and CAC into incremental cost over incremental burn. That disproved a standing leadership assumption that deeper discounts always reduce RPC.

RESULT

- **Beat the target I'd set.** Monthly spend **₹9 Cr → ₹4 Cr** — a ~55% reduction against the ~50% aim — with **~80% of incremental NC preserved**, exactly as targeted, and **CAC ~₹300 → ~₹200**. Roughly 0.2 Mn incremental NC monthly against a ~0.3 Mn target, at about half the spend.
- **Ran in production for five to six months**, scoring the NAC base monthly.
- **The framework was adopted across Analytics teams as the standard approach to incentive-based targeting** — other charters picked up uplift-based targeting rather than propensity or full-base targeting for their own incentive problems. The ambition ended up bigger than the project it was set for.
- **De-risked incentives as a lever platform-wide** by establishing that coupon availers' long-term repeat and spend were not materially different from organic customers.
- **The decile structure became reusable infrastructure**, funding a second wave of offer-construct experimentation at no additional analytical cost.

KEY LEARNING

Setting the goal as a specific, uncomfortable number changed what work was even possible. If I had aimed at "improve targeting efficiency," I would never have run an experiment purely to generate training data — the effort would not have been justifiable against a vague goal, and I would have settled for the best answer historical data could support. The aggressive target is what made the expensive prerequisite worth paying for. The second lesson is that "find a way" often means confronting the fact that the required input does not exist rather than is hard to get — and the response to unobservable data is to build the conditions that produce it, not to substitute a weaker proxy.

What Interviewers Will Probe

- **Why set the target at 80% of outcome for 50% of cost rather than something you knew was achievable?** Because a target I knew was achievable would have produced a marginal improvement. I picked the number to be uncomfortable enough that the existing approach could not reach it under any amount of tuning — which forced the reframe. If I had aimed at 10% efficiency, propensity scoring would have got me there and I would have stopped.

- **What would you have done if the ATE gate had failed?** Stopped, and said so. A near-zero ATE means the offer does not materially move behaviour in that population, so no Persuadable cohort exists to find — the target would have been arithmetically impossible, not just hard. I would have redirected at the offer construct itself, because the finding would be that the incentive was too weak to change decisions, which is its own useful answer.

- **You asked the business to run a campaign whose purpose was training data. How did you justify that?** Against the size of the prize. Half of a ₹9 Cr monthly budget is a very large recurring return for one experiment's cost, and the experiment was not wasted spend — it was a live coupon campaign that acquired real customers, just with instrumentation attached and a control group held back. The incremental cost of the ambition was the control group's foregone conversions, not the whole campaign.

- **How much of the ~55% saving is durable versus a one-off?** Durable, because the mechanism is structural rather than a one-time optimisation — Sure Things exist in every NAC base every month, so the model kept finding them across five to six monthly scoring cycles. What I would want to monitor over a longer horizon is drift: the features predicting Persuadability are engagement signals, so if the app experience changes materially the model needs retraining rather than just rescoring.

- **What does "the framework was adopted by other teams" actually mean in practice?** That uplift-based targeting replaced full-base or propensity-based targeting as the default starting point for incentive problems in other charters — the four-segment framing, the requirement for a randomised experiment as the foundation, and decile-based validation. That is the outcome I would point at as jaw-dropping rather than the ₹5 Cr, because a saving happens once and a standard compounds.

- **Where did you stop short of the ambition?** Sleeping Dogs. I concentrated the whole project on isolating Persuadables and treated negative uplift as a small tail, when with a monthly-scored production model there was a real opportunity to quantify whether any cohort was actively suppressed by the offer. Aiming high on the Persuadable half and not on the other half was an incomplete version of my own target.

Why This Story Screams Aim High and Find a Way

- ✔ The ambition was entirely self-set — nothing was failing and nobody asked for a change
- ✔ Set a specific, uncomfortable number (80% of outcome at 50% of cost) rather than a directional goal
- ✔ Worked backwards from the target to discover the required input was fundamentally unobservable
- ✔ Changed the conditions rather than trimming the goal — ran an experiment purely to manufacture the missing data

- ✔ Gated the ambition honestly with a kill-switch rather than committing blindly
- ✔ Beat the target (~55% versus ~50%) and then pushed past its original scope into offer construct optimisation
- ✔ The "wow" outlived the project: adopted across Analytics as the standard for incentive-based targeting

14. DEMAND EXCELLENCE

Coupang Leadership Principle: DEMAND EXCELLENCE *(secondary: Influence without Authority)*

The principle in plain English: Coupang says leaders have a passion for excellence, do not tolerate mediocrity, and hold themselves and others to a standard they raise perpetually. The uncomfortable word is *tolerate* — this principle is not about doing good work, it is about refusing to accept work that would have passed. The interviewer is listening for a moment where "adequate" was on the table, you rejected it, and rejecting it cost you something: time, friction, or someone's goodwill. And the strongest version raises the bar on your own output too, not just on other people's.

How interviewers may frame this question:

- Tell me about a time you refused to accept work that was good enough.
- Describe a standard you held when it would have been easier to let it go.
- Give me an example of raising the bar on yourself, not just on others.
- Tell me about a time you insisted on something unpopular because the quality mattered.
- How do you handle it when a team says their part is complete and you disagree?
- Tell me about a time you added a measure that made your own work look worse.
- What's the highest standard you've held a project to, and what did it cost?

Story: Pricing Automation — refusing to accept incomplete instrumentation, and volunteering the guardrail that made the product look bad

Why this is the best fit:

Twice on this project the easy path was available and I refused it. The first was instrumentation. Both engineering teams had built instrumentation for their own product, both products worked end to end in isolation, and by every local standard the work was complete — there was no defect to point at, no failing test, no unhappy stakeholder. I said it was inadequate anyway, because there was no shared data layer capturing the system-level story from recommendation to realised ASP, and I spent multiple alignment rounds across two product orgs arguing for engineering effort nobody had budgeted, before I could show a single result. The second was the price fluctuation guardrail: I introduced a customer-facing metric into the framework that had a high chance of making the automated pricing product look worse than manual pricing, and then argued for its importance to leadership. Nobody asked for either. Both are the same behaviour — declining to accept a standard the org would have signed off on.

2-Minute Executive Summary:

"On the automated pricing Pilot, the point I would highlight for excellence is that twice I refused work that the organisation considered finished. First, instrumentation. Each product's engineering team had built its own instrumentation, and each product genuinely worked end to end — by any local standard the job was done. But there was no shared data layer, so you could not trace a single reaction from price recommendation through offer mapping to the ASP a customer actually paid. I flagged that as incomplete rather than accepting it, and I did the harder version: I mapped the entire end-to-end journey across both products myself and specified exactly what was needed — reaction ID by FSN granularity, capturing FSP, ASP, predicted DRRs, reaction go-live time, offer go-live time, projected mandatory and non-mandatory BCD, actual BCD, actual GMV and CM. Then I built the measurement framework in advance, before the data existed, so I could show teams precisely which questions would be unanswerable without it rather than asking them to trust me. That took multiple alignment rounds with Product, Engineering, Program and leadership across both product orgs, arguing for sprint capacity nobody had planned, with no results to show yet. The second was the price fluctuation guardrail. Nobody asked for a customer-facing metric in an automated pricing framework, and I introduced one — measuring frequency, magnitude and direction of price changes at FSN level during events, rolled up on unit weight — knowing it was the metric most likely to make the product look bad. It did: it showed the volume of Optimizer reactions was causing significantly higher price volatility than manual pricing. I argued for its importance to leadership rather than quietly dropping it, and it became the input that drove the Optimizer team to introduce reaction throttling. That guardrail was the first customer-facing metric anyone had put into the framework."

Full STAR Version

SITUATION

This was Flipkart's first attempt at automated pricing at scale — 10M+ 1P listings, 90–95% of pricing done manually by categories on intuition, with the CEO and CFO reviewing the Pilot directly because it was the moment of truth on a substantial outsourced product investment. Two products worked in tandem: the **Optimizer**, recommending the FSN × ASP × FSP combination that maximises GMV within CM constraints, and **Offer Intelligence (OI)**, bridging the FSP → ASP gap through BCD projection to decide which offers to map or unmap.

The quality environment was the problem. Category teams were resistant, product and data science teams were invested in the product succeeding, and there was no playbook. In that setting the organisational pull is always toward declaring things complete: every party had a reason to want the Pilot evaluated and moved on, and the definition of "good enough" was being set by whoever was most eager to see a result.

TASK

I owned instrumentation, the governance and measurement framework, and the ASP deviation deep dive. But the task I would point at here is narrower and harder to justify on

a timeline: deciding what standard the measurement of this Pilot had to meet, and then holding it against an organisation that would have accepted considerably less.

ACTION

Step 1 — Called instrumentation incomplete when nothing was visibly broken. The initial instrumentation had been built independently by each product's engineering team, and each product worked end to end in isolation. There was no defect. What there was, was an absence: no shared data layer capturing the system-level story from price recommendation → offer mapping → actual ASP realised. That distinction matters, because "this works but is insufficient" is a much harder argument to make than "this is broken." My position was that for a first-of-its-kind Pilot under CEO scrutiny, a product you cannot trace is indistinguishable from a product that failed — and that measuring two tools separately tells you nothing about the system the customer actually experiences.

Step 2 — Did the work myself rather than raising a concern. The low-effort version of holding a standard is to flag the gap and hand it back. Instead I mapped the full end-to-end product journey of both Optimizer and OI — reaction ID generation → FSP/ASP recommendation → BCD projection → offer go-live → actual BCD → realised ASP — and then specified the exact metrics and granularity required to stitch it together: at **reaction ID × FSN level**, capturing FSP, ASP, predicted DRRs, reaction go-live time, offer go-live time, projected mandatory BCD, non-mandatory BCD, actual BCD, and actual GMV and CM. A specific spec is a fundamentally different ask from "we need better data," and it is what made the standard defensible rather than an opinion.

Step 3 — Built the measurement framework in advance of the data, to make the gap concrete. This was the step that broke the deadlock. Rather than asking teams to invest in instrumentation on the promise of future value, I designed the four-layer governance framework first — target sanity check, GMV/CM achievement with portfolio decomposition, ASP adherence metrics, and the customer-facing guardrail — so I could point at specific questions and show which ones were structurally unanswerable without the instrumentation. The four-bucket ASP deviation decomposition by go-live sync is the clearest example: you cannot separate "price live but offer not" from "both live" without reaction start/end times and offer go-live timestamps, and without that separation you cannot tell a modelling error from a pipeline error. Building the framework early converted an abstract quality argument into a list of things leadership would ask and we would not be able to answer.

Step 4 — Held it through multiple alignment rounds with nothing to show. I ran repeated alignment sessions with Product, Engineering, Program and leadership teams across both product orgs, advocating that instrumentation is as critical as the product itself. This was the expensive part. I was asking two engineering orgs for sprint capacity on something that was not a feature on either roadmap, in the pre-result phase of a project where I had no track record yet, against the natural counter-argument that both products already worked. There was no moment where this became easy; it took repetition and specificity.

Step 5 — Then held the standard on my own side. Once instrumentation was live I partnered with data analysts to build the pipeline and Pilot dashboard, and evangelised it until it was embedded in the standard Pilot review cadence. An instrumented system nobody reviews is the same as an uninstrumented one, so the standard included adoption, not just existence.

Step 6 — Introduced a guardrail metric that was likely to embarrass the product. Nobody asked for a customer-facing metric in an automated pricing framework — the objective function was GMV with CM as a guardrail, both internal. I added **price fluctuation** as a guardrail anyway: frequency, magnitude and direction of price changes during events at FSN level, rolled up on unit weight. I knew when I designed it that it had a good chance of showing the product in a worse light than manual pricing, because a model reacting frequently is going to move prices more than a human updating them occasionally. It did exactly that: the volume of Optimizer reactions was causing significantly higher price volatility than manual pricing, raising a genuine price-perception risk on the platform.

Step 7 — Argued for the uncomfortable metric rather than letting it drop. The path of least resistance was to report it once and let it sit as a footnote — it was outside the stated objective, it complicated a positive narrative, and no stakeholder was asking for it. I argued its importance to leadership instead: that a pricing system optimising internal GMV while degrading customer price perception is not a successful system, whatever the GMV number says. It became the input that led the Optimizer team to introduce reaction throttling, and it was the first customer-facing metric anyone had put into the automated pricing framework.

Step 8 — Raised the bar on target quality too, including where it was awkward. The target sanity check layer benchmarked category-set GMV and CM targets against similar-scale historical events with comparable cost inputs, and flagged inflated ones — which meant challenging numbers a resistant stakeholder group had set, and escalating to BizFin and Category leadership to align on realistic targets. Accepting the submitted targets would have been frictionless and would have made the entire Pilot verdict meaningless.

RESULT

- **The first cross-product data layer** connecting Optimizer and OI — enabling measurement of the system as a whole rather than two isolated products, and the foundation for every future Pilot review.
- **ASP deviation reduced from ~70% to ~30%** of GMV beyond ±2%, which was only diagnosable because of the instrumentation and the four-bucket go-live decomposition it enabled.
- **Go-live synchronisation defect surfaced** — only 80% of GMV had price and offer live in sync — and fixed by Engineering. Invisible without the timestamps.
- **Price fluctuation adopted as a guardrail**, shifting the product team's thinking beyond internal GMV and CM to platform price perception, and driving reaction throttling in the Optimizer.
- **CI-match portfolio rationalised from 70–80% to ~30–40%** through a standardised, data-backed KVI definition built on Search PPVs and price elasticity.

- **The governance framework became the standard evaluation template for every Pilot**, giving leadership a consistent lens across Super Categories and letting the expand-or-pause call be made on evidence.
- **~₹500 Cr of bad burn** quantified and mitigated at BBD scale via the Spend Management measurement framework.

KEY LEARNING

The hardest standard to hold is the one where nothing is visibly broken. Both engineering teams had done competent work and both products functioned; the gap was an absence rather than a defect, and absences are easy for an organisation to sign off on. What made it possible to hold the line was doing the work myself — a precise spec and a framework built in advance — so the argument stopped being "this isn't good enough" and became "here are the questions you will ask that we won't be able to answer." The second lesson is that demanding excellence has to include your own side of the table. Volunteering the metric most likely to make the initiative look bad, before anyone asked for it, is what made every other standard I held credible rather than convenient.

What Interviewers Will Probe

- **Both products worked. Wasn't insisting on more instrumentation over-engineering under time pressure?** It cost weeks and I would defend it. The Pilot's entire purpose was a verdict on whether automated pricing beats manual, and without traceability from recommendation to realised ASP, every attribution becomes an opinion in a room full of biased parties. We would have shipped a Pilot with a GMV number and no ability to explain it — which is a worse outcome than shipping late.

- **How did you convince two engineering orgs to fund work that wasn't on either roadmap?** By removing the vagueness. I brought a spec at reaction ID × FSN level with named fields rather than a request for better data, and I built the measurement framework first so I could show which specific questions were unanswerable without it. Most resistance to instrumentation asks is really resistance to open-ended scope.

- **Why introduce a metric you expected to make the product look bad?** Because it was true and it mattered to customers. A pricing system that hits its GMV target while making prices visibly volatile on the platform is not succeeding, it is succeeding on an incomplete scorecard. There was also a credibility argument: I was the neutral measurement body, and a framework that only contained metrics flattering to the product would have deserved the scepticism Category already had.

- **How did leadership react to the price volatility finding?** It landed because it came with a mechanism and a fix rather than as a criticism — the volatility traced to reaction volume, which pointed directly at throttling as a lever. Presenting an uncomfortable metric alongside its remedy is the difference between raising the bar and just raising an objection.

- **Where did you decide something *was* good enough?** The BCD adoption model. The right answer was a proper predictive model for offer adoption; what I recommended and shipped was a finer static granularity — Event × Vertical × Price Tier × Bank Tier × Payment Instrument with a fallback chain — plus a live mid-event feedback loop. That was the correct call for the Pilot timeline, and I was explicit that the underlying gap remained rather than presenting it as solved.
- **Isn't there a risk this reads as perfectionism?** It would be, if the standard were uniform. It wasn't — I held an unusually high bar on instrumentation and on the customer-facing guardrail, and consciously accepted a heuristic on offer adoption projection. The bar goes where the decision quality depends on it, and I would rather be able to name where I compromised than claim I never did.

Why This Story Screams Demand Excellence

- ✔ Refused work the organisation would have signed off on — no defect existed, only an insufficiency
- ✔ Held the standard through multiple alignment rounds, pre-results, with no track record to lean on
- ✔ Did the hard version: mapped the journey and wrote the spec rather than handing the gap back
- ✔ Built the framework in advance so the quality argument became a concrete list of unanswerable questions
- ✔ Raised the bar on itself: volunteered the guardrail most likely to make the initiative look worse
- ✔ Challenged inflated targets rather than accepting a frictionless but meaningless Pilot verdict
- ✔ The standard became permanent — the framework is the evaluation template for every subsequent Pilot

15. HATE WASTE

Coupang Leadership Principle: HATE WASTE *(secondary: Think Systematically)*

The principle in plain English: Three separate ideas, and interviewers probe all three. **Do more with less** — same or better output from fewer resources, whether that's money, analyst hours or compute. **Save costs rather than simply cut them** — this is the clause that does the real work. Cutting means the spend falls and the output falls with it; saving means you isolate the portion of the spend that was producing nothing and remove only that, so the outcome survives. **Don't skimp on a penny today if it costs a dollar tomorrow** — a deliberate warning against the false economies the first two clauses could otherwise encourage, which means sometimes the Hate Waste answer is to spend more. The one-line test: if the spend went down, did the outcome stay up?

How interviewers may frame this question:

- Tell me about a time you did more with less.

- Give me an example of eliminating significant waste.
- Describe a time you cut cost without cutting capability.
- Tell me about waste that was invisible in the existing metrics.
- Have you ever spent more upfront to avoid a bigger cost later?
- How do you decide whether a cost is genuinely waste or just expensive?
- Tell me about the largest saving you've personally delivered.

Story: Intelligent Couponing — half the budget, 80% of the outcome, and a two-week experiment paid for upfront

Why this is the best fit:

This story hits all three clauses of the principle, which is rare. On *do more with less*: monthly couponing spend fell from ₹9 Cr to ₹4 Cr. On *save rather than cut*: ~80% of incremental NC was preserved and CAC improved from ~₹300 to ~₹200 — the outcome survived, which is the entire distinction Coupang is drawing. And on *don't skimp a penny today*: rather than mining historical campaign data for free, I ran a two-week randomised A/B specifically to build a properly representative training dataset, accepting the control group's foregone conversions and two weeks of calendar as the price of getting the model right. Reusing historical data would have been the cheap option and would have produced a model that couldn't be trusted with production budget — a penny saved for a dollar lost.

The waste itself was also of the most valuable kind: invisible inside the measurement framework being used to evaluate it. The old campaigns genuinely produced customers, so nothing in the reporting could ever reveal that half the spend was buying outcomes that would have happened anyway.

2-Minute Executive Summary:

"Within the NC charter, roughly ₹9 Cr a month of couponing spend was going out under a blunt method — target the whole NAC base, split Test and Control, measure ATE, report incremental NC. The important thing about that waste is that it was invisible: the campaigns genuinely produced customers, so no metric in the existing framework could reveal that a large share of the budget was paying people who were always going to buy. I isolated it by shifting the question from who is likely to buy to who buys *because of* the coupon, using the four behavioural segments — Persuadables, Sure Things, Sleeping Dogs, Lost Causes. The Sure Things were the waste. Then I paid the upfront cost deliberately: uplift is unobservable at the individual level, so I ran a two-week randomised A/B specifically to manufacture a training dataset with proper representation of every customer type across both treated and untreated cohorts, capturing engagement and demographic features while the experiment ran. That cost two weeks and the control group's foregone conversions, and I chose it over reusing historical data because a model trained on unrepresentative data would have been cheap and untrustworthy. An S-Learner XGBoost model scored each user twice to get per-user uplift, validated by decile analysis on true uplift, calibration against the 1:1 line, and normalised cumulative gain at the ~80% AUC benchmark. Monthly spend went from ₹9 Cr to ₹4 Cr — about 55% — while preserving ~80% of incremental NC, with CAC falling from ~₹300 to ~₹200. It ran in production for

five to six months, the decile structure was reused at no extra cost for offer-construct testing, and the framework was adopted across Analytics teams as the standard for incentive-based cohort targeting."

Full STAR Version

SITUATION

The NC charter was chasing roughly 30% YoY growth and leaning on incentive-led acquisition while the product-led roadmap landed 4–5 months out. Monthly couponing spend was around ₹9 Cr, targeting ~0.3 Mn incremental NC at a CAC of ~₹300.

The method was blunt: take the entire NAC base, split Test and Control, run a TD coupon on Test, measure ATE as Test minus Control conversion, report incremental NC at some RPC and CAC. Conceptually, any target base splits into four segments by response to an offer — Persuadables who buy *because* of it, Sure Things who were always going to buy, Sleeping Dogs who become less likely because of it, and Lost Causes who were never going to buy. Full-base targeting pays all four.

The critical feature of this waste is that it was structurally invisible. The campaigns worked. They produced real incremental NC and reported it accurately. Nothing in the measurement framework could distinguish a rupee that changed someone's decision from a rupee that subsidised a purchase already coming — because both show up as a customer. An org can run that method indefinitely and never see the leak, which is exactly why it had been running for years.

TASK

Isolate and remove the wasted portion of the spend without losing the incremental NC the business depended on to hit its AOP. The framing I set was explicitly a do-more-with-less target: what is the ~30% of the target cohort that brings ~80% of incremental NC at ~50% of overall cost?

ACTION

Step 1 — Made the waste visible by changing what was measured. No optimisation of the existing approach could have found this, because the existing metric was incapable of seeing it. The reframe from *who is most likely to buy* to *who is most likely to buy because of the coupon* is what turned an invisible leak into a measurable quantity. I also explicitly ruled out the intuitive alternative — a propensity model — because targeting high probability-to-convert users actively concentrates Sure Things in the target group. That would have increased the waste while looking like a sophisticated improvement.

Step 2 — Paid the upfront cost rather than taking the cheap route. Uplift cannot be observed for any individual: you see either their treated outcome or their untreated one, never both. So the required training data did not exist and could not be recovered from historical campaigns. The cheap option was to approximate it from past campaign data. I rejected that and ran a **two-week randomised A/B specifically to manufacture the dataset**, with the NAC base split Test and Control and Test receiving an x% off up to ₹z

coupon. The design requirement I held to was **proper sample representation of every customer type across both cohorts**, because an uplift model trained on a skewed sample produces confident scores for the segments it has seen and noise for the rest — and it is the noise segments where production budget gets wasted.

That two weeks and the control group's foregone conversions were a real, accepted cost. Against a recurring ₹5 Cr monthly saving, spending it was obvious arithmetic; the point is that the cheap alternative was available and would have produced a model nobody should have trusted with the budget. This is the penny-today-dollar-tomorrow clause in practice.

Step 3 — Captured the features while the experiment ran, because they cannot be recovered afterwards. Each AID was captured with its Test/Control flag plus engagement signals — add-to-cart clicks, banner clicks, bounce rate, average session length, average scroll length, PPVs per visit, PPVs by BU — and dimensions including gender, city tier, zone and age, with the converted/not-converted flag appended on conclusion. Behavioural state during the treatment window is not reconstructable later, so a second experiment would have been the only remedy for missing it. Getting the instrumentation right the first time avoided paying for the experiment twice.

Step 4 — Gated on ATE before spending modelling effort. On conclusion I computed ATE and confirmed statistical significance. A near-zero ATE would have meant the offer moved nobody materially, in which case no Persuadable cohort exists and every subsequent hour would have been wasted. Cheap gates that prevent expensive dead ends are themselves waste elimination.

Step 5 — Built one model rather than two. An S-Learner: a single XGBoost classifier over all features plus the treated indicator, with per-user uplift obtained by scoring each user twice — flag at 1, flag at 0 — and taking the difference. One model instead of two separately trained ones, and an architecture explicable to a business audience in two sentences, which saved a great deal of downstream explanation cost during adoption.

Step 6 — Validated three ways so the saving would survive scrutiny. Decile analysis computing *true* uplift (actual Test minus Control conversion) per bin and confirming it rose in the top deciles; a calibration scatter of predicted against true uplift sitting close to the 1:1 line; and a normalised cumulative gain chart, where ~50% AUC is random and ~80% is the industry benchmark for a strong uplift model — which ours reached. The finding: **the top three deciles captured ~80% of incremental customers at ~50% of total cost**, with CAC at ~₹150 versus ~₹350 for full-base targeting and RPC ~4 versus ~2.

Step 7 — Confirmed the outcome survived before touching production budget. Scored the full NAC base, created deciles, and ran a confirmatory A/B — 90/10 Test/Control on the top three deciles, 10/90 on the bottom seven — to prove the top-3 ATE was significantly higher and RPC and CAC genuinely optimised. Only then did the model go to production with the Data Science team, scoring the NAC base monthly. This is the step that distinguishes saving from cutting: I established the output would hold before reducing the input.

Step 8 — Reused the asset instead of building a second one. Once the deciles existed, the offer question could be answered with the same infrastructure: A/B experiments on the top deciles with T1/T2/T3 and C1/C2/C3 cells testing 30% off up to ₹50 against 10% off up to ₹100 against a ₹9 store construct, with RPC decomposed into incremental revenue over incremental burn and CAC into incremental cost over incremental burn. That disproved the standing assumption that deeper discounts always reduce RPC — the 30% construct had higher RPC and lower CAC, because offer perception drove an incremental NC spike more than compensating for the higher burn. A second wave of learning at essentially no additional analytical cost.

Step 9 — Removed the objection that would have re-inflated the spend. I quantified coupon availers' long-term goodness — spend and M1/M2/M6 repeat — against non-availers and showed no material difference from organic customers. Without that, the persistent fear that incentive-acquired customers are low quality would have kept pulling budget back toward broad targeting.

RESULT

- **Monthly spend ₹9 Cr → ₹4 Cr (~55% reduction)** with **~80% of incremental NC preserved** — roughly 0.2 Mn incremental NC monthly against a ~0.3 Mn target, at about half the spend.
- **CAC ~₹300 → ~₹200** per incremental NC. **The outcome survived the saving**, which is the whole test.
- **Ran in production for five to six months**, scoring the NAC base monthly, so the saving recurred rather than being a one-off.
- **The framework was adopted across Analytics teams as the standard for incentive-based cohort targeting** — the waste elimination generalised beyond the charter it was built in, which compounds far beyond ₹5 Cr a month.
- **The decile structure became reusable infrastructure**, funding offer-construct experimentation at no incremental analytical cost and correcting a mis-calibrated assumption about discount depth.
- **De-risked incentives as a lever** by establishing availers behave like organic customers on long-term repeat and spend.

KEY LEARNING

The most valuable waste to find is the kind that is invisible inside the metrics used to evaluate the spend — the campaigns worked, produced customers, and reported honestly, and none of that could have revealed that half the money was buying outcomes already coming. Finding it required changing the question, not improving the measurement. The second lesson is about the false-economy clause: the cheap path was to approximate the training data from historical campaigns, and it would have produced a model that looked fine and quietly wasted budget on the segments it had never properly seen. Spending two weeks and a control group's conversions upfront to get a genuinely representative sample was the frugal decision, not the expensive one — being cheap about the input is how you end up wasting the output.

What Interviewers Will Probe

- **You lost 20% of incremental NC. Isn't that a cut, not a saving?** It's the trade I'd defend, and I'd rather state it plainly than hide it. We gave up ~20% of incremental NC for ~50% of the budget — roughly ₹5 Cr a month freed. Whether that's right depends on the marginal return of the freed budget, which is exactly why the NC budget allocation framework became the natural follow-on: it showed the same money reallocated across cohorts generated ₹30 Cr of incremental GMV. The 20% was not lost, it was redeployed.

- **Why spend two weeks and a control group on an experiment instead of using historical data?** Because the required quantity is unobservable and historical data cannot supply it. Past campaigns give you who converted under treatment, not what those same people would have done untreated, and any approximation would have skewed toward whatever segments happened to be over-represented. A model that scores confidently on segments it has seen and noisily on the rest wastes production budget silently — which is much more expensive than two weeks.

- **How did you ensure the sample genuinely represented every customer type?** Through clean randomisation across the full NAC base rather than a convenience sample, which is what gives you both cohorts populated across all four behavioural segments, and by capturing a wide feature set during the treatment window so the model had the engagement and demographic variation to distinguish them. I then checked representation at decile level in validation — if a segment were missing, its bin's true uplift would be unstable, and it wasn't.

- **What's the risk the saving erodes over time?** Drift. The features predicting Persuadability are engagement signals — recency, ATC, banner clicks, search PPVs, bounce rate — so a material change to the app experience changes the mapping. Rescoring monthly handles population change; it does not handle relationship change. Retraining cadence is what protects the saving, and I'd want that on a schedule rather than triggered by someone noticing.

- **Where did you leave waste on the table?** Sleeping Dogs. I treated negative uplift as a small tail and focused entirely on isolating Persuadables. With a monthly-scored production model there was a real opportunity to quantify whether any cohort was actively suppressed by the offer — that's burn that is worse than wasted, because it buys a negative outcome, and it sits on top of the ₹5 Cr.

- **Is a ₹5 Cr monthly saving really the headline, or is it something else?** It's the adoption. A saving in one charter happens once and recurs within that charter; a framework other Analytics teams adopt as their default for incentive targeting removes the same class of waste everywhere it's applied. I'd lead with the ₹5 Cr because it's concrete, and I'd point at the adoption as the thing that actually compounded.

Why This Story Screams Hate Waste

- ✔ Found waste that was structurally invisible inside the metrics used to evaluate the spend
- ✔ Did more with less — ₹9 Cr to ₹4 Cr monthly, CAC ~₹300 to ~₹200
- ✔ Saved rather than cut: ~80% of the outcome survived, and it was proven to survive *before* the budget was reduced
- ✔ Refused the false economy — paid two weeks and a control group's conversions for a properly representative dataset rather than approximating from historical data free
- ✔ Cheap gates to prevent expensive dead ends: ATE significance checked before any modelling effort
- ✔ Reused the asset — deciles funded a second wave of offer-construct learning at no extra cost
- ✔ The elimination generalised: adopted across Analytics as the standard for incentive-based cohort targeting

16. HIRE AND DEVELOP THE BEST

Coupang Leadership Principle: HIRE AND DEVELOP THE BEST *(secondary: Demand Excellence)*

The principle in plain English: Coupang expects leaders to raise the team's performance level with every hire and promotion, to hire for the long-term need rather than the immediate gap, to recognise exceptional talent and invest real resources in developing its full potential, and to empower people to pursue growth through internal rotation. Two things get scored that candidates often miss. First, did you pick on *potential* rather than on current credentials — because hiring for the long-term need means backing someone who cannot yet do the job. Second, did the investment cost you something, and did you support the person growing even when it took them out of your reach? A generic "I mentor my team" answer fails this principle; a named person with a before-and-after does not.

How interviewers may frame this question:

- Tell me about someone you developed and what changed for them.
- How do you decide who to invest your time in?
- Describe a time you gave someone work that was above their level.
- Tell me about a time you backed potential over experience.
- How do you handle a team member who isn't contributing?
- Give me an example of supporting someone's growth even when it cost you.
- Tell me about a talent decision you made that others might have made differently.

Story: The HL Semantic Layer — backing a one-year BA over a more experienced analyst, and watching him grow out of the project

Why this is the best fit:

The talent decision here was made under a real constraint and it went against the obvious call. I needed someone from the Hyperlocal team for the agentic project — someone who

understood HL facts, schema and queries — and I started with 100% of a two-year analyst's bandwidth and 50% of a one-year BA's. Within two weeks the pattern was clear: the more experienced analyst's presence was uncertain and she did only what was asked, while the BA on half the bandwidth was getting more done. So I reallocated toward the person with less experience and more curiosity. He went on to build the semantic layer for one stream, then for all 4–5 streams in HL — close to 100 metrics and dimensions for the entire Hyperlocal analytics team. He came in doing ad-hoc SQL support; he came out understanding end-to-end agentic orchestration and why the semantic layer is its backbone, and after the project he moved into a new team under the same manager owning multiple AI-driven RCA automations across the HL charter. None of the coaching was part of my mandate.

2-Minute Executive Summary:

"For the Hyperlocal agentic project I needed someone from the HL team, because the work depended on real understanding of HL facts, schema and queries. I was given 100% of a two-year analyst's bandwidth and 50% of a one-year BA's — he was the only person in the team with capacity to spare. Before starting I ran a session with him on his understanding of the facts and schema, and what stood out was not his credentials but his curiosity: he wanted to know how agents were built, what a semantic layer even was, why it mattered. His SQL was also genuinely good, which you could see from the ad-hoc query work he was doing for his team. Within the first two weeks the more experienced analyst's presence became uncertain and she was doing only what was explicitly asked, while the BA on half the bandwidth was consistently getting more done. I tried to re-engage her, raised it with my manager, gave it a week to change, and then reallocated in the project's interest — deliberately betting on 50% of someone who showed up over 100% of someone who didn't. Then I invested in him properly, which was not part of my mandate. I scoped him narrow first — the semantic layer for a single stream — and ran a daily review cadence so he could move fast without getting stuck. He scaled from one stream to all 4–5 streams in HL, close to 100 metrics and dimensions defined once for the whole Hyperlocal analytics team. He also picked up agent prompt writing in bits and started bringing his own point of view into agent design discussions rather than waiting to be assigned — and the one place he went materially wrong was there: his early agent instructions were loose enough to cause hallucination. I sat with him for a single debugging session on it, and rather than just fixing the prompt I walked him through why strict instructions matter, which turned a defect into the thing that gave him his sharpest understanding of the whole system. His manager was in those reviews because the layer was being built for HL, so the contribution was visible to the person who could act on it. We closed the project in JFM, and afterwards he moved into a new team under the same manager with a couple of long-term projects — a scope change rather than a level change, now handling multiple RCA automations through AI in the HL charter. He arrived doing ad-hoc SQL and left owning AI automation."

Full STAR Version

SITUATION

I had been given the project of building the agentic flow for Hyperlocal. The work was not something I could do alone in any sensible way, and specifically not the data layer: it needed someone from within the HL team who genuinely understood HL facts, schema and query patterns, because a semantic layer built by an outsider guessing at definitions is worse than no semantic layer at all.

Resourcing was constrained. Most of the HL team was occupied. I was given **100% of one analyst's bandwidth — a person with two years in the HL team** — and **50% of a one-year BA's**, because he was the only other person with any capacity to share. On paper the allocation was clear: the experienced analyst was the primary resource and the BA was support.

TASK

Deliver the semantic layer and the agentic flow for HL. But the task that turned out to matter was a talent judgement I had not been asked to make: recognising within the first weeks that the allocation on paper was the opposite of the allocation that would work, and then deciding how much of my own time to invest in the person with less experience.

ACTION

Step 1 — Assessed him for potential rather than for credentials. Before the project started I ran a session with him on his understanding of the HL facts and schema. Two things came out of it. His **SQL quality was genuinely good** — visible from the ad-hoc query work he was doing to answer questions for his team, which is unglamorous work but a fair test of whether someone can navigate a schema correctly. And his **curiosity was unmistakable**: he asked how agents get built, what a semantic layer actually is, how the two connect. He could not do the job at that point. What he had was the disposition to learn it, and for a project with no established playbook that was the more useful signal.

Step 2 — Handled the non-contribution properly before reallocating. Within the first two weeks of kickoff, the more experienced analyst's presence became uncertain and her pattern was to do only what was explicitly asked — no initiative, nothing beyond the brief. Meanwhile the BA on 50% bandwidth was consistently getting more work done than she was on 100%. I did not simply write her off. I tried to re-engage her, then raised it with my manager, then **waited a week to see whether it changed**, and only then made the call to move on in the project's best interest. That sequence matters to me: reallocating in week one would have been a snap judgement, and letting it run for a month would have cost the project. A week of explicit visibility to my manager was the right amount of process for the decision.

Step 3 — Made the deliberate bet: 50% of someone who shows up beats 100% of someone who doesn't. That is the talent judgement in one line. Experience, tenure and formal bandwidth allocation all pointed one way; observed contribution and curiosity pointed the other. I backed the second.

Step 4 — Invested time that wasn't in my mandate. Coaching him was not part of my brief and nobody asked me to do it. Two honest reasons I did. It was the fastest route to the

project's outcome, given who I actually had. And it was genuinely enjoyable — he was curious enough that teaching him was rewarding rather than a tax, and I wanted him to end up in a position to win long-term projects rather than staying on ad-hoc support.

Step 5 — Scoped narrow first so the stretch wasn't a setup to fail. He started with the semantic layer for **one stream** — not the whole HL charter. That is a real piece of work with a visible outcome, contained enough that a mistake was recoverable and specific enough that he had to make actual design decisions about measures, dimensions and business rules rather than following a pattern. Giving a one-year BA the definitional layer for an entire charter on day one would have been reckless; giving him one stream was a stretch he could survive.

Step 6 — Ran daily reviews, to move fast rather than to check up. The cadence was daily and the framing was speed, not supervision. In a domain neither of us had a playbook for, a daily loop meant a wrong design decision cost a day rather than a fortnight, and it let him make calls independently between reviews instead of waiting for permission. That distinction — reviewing to unblock rather than to approve — is what kept the pace up while he was still learning.

Step 7 — Widened the scope as fast as he earned it. From one stream he moved to building the semantic layer across **all 4–5 HL streams, covering close to 100 metrics** — every metric and dimension definition for the entire Hyperlocal analytics team, defined once and consumed by dashboards, humans and agents alike. That is the foundational layer for the whole charter's analytics, built substantially by someone a year into their career.

Step 8 — Coached through the one thing he got materially wrong. When he moved from the semantic layer into writing agent instructions, his early prompts were loose — plain-English, under-specified — and they caused the agent to hallucinate. This is exactly the mistake I had made myself earlier in the project, so it was recognisable immediately. The easy fix was to rewrite the prompt for him and move on. Instead I sat with him for a **single debugging session** where we worked through where the instruction was ambiguous and what the model did with that ambiguity — and I used it to teach the underlying principle rather than the specific fix: that agent instructions need explicit structure, that negative triggers matter as much as positive ones, and that a confident wrong answer is the characteristic failure mode of a loose prompt. That one session materially changed his understanding of prompt engineering and of why the whole architecture is built the way it is. A defect became the highest-leverage teaching moment in the project.

Step 9 — Let him into the work beyond his assigned scope. He also picked up **agent prompt writing in bits and pieces**, and more importantly he started **actively engaging in agent design discussions — bringing his own point of view and his own learnings** rather than waiting to be assigned. I encouraged that instead of keeping him in his lane, because the thing he needed for a bigger role next was exposure to the end-to-end architecture, not more depth on LookML. He came out of the project understanding agentic orchestration end to end and why the semantic layer is its backbone.

Step 10 — Made the contribution visible to the person who could act on it. I did not have to justify him to anyone — nobody challenged whether a one-year BA should be building the HL definitional layer. But visibility was not accidental. **His manager was part of the project reviews**, because the semantic layer was being built for the HL team, so his contribution was seen directly by the person who owned his growth. And in my own weekly reviews with my manager I showed his contribution explicitly rather than presenting the work as the project's or mine. Making someone's work legible to the people who make decisions about them is a large part of developing them.

RESULT

- **He built the semantic layer for the entire Hyperlocal analytics team** — from a single stream to all 4–5 streams, close to 100 metrics and dimensions defined once as the single source of truth for the charter.
- **The project was delivered in JFM**, with the semantic layer as the foundation for the HL agentic flow.
- **He moved into a new team under the same manager after the project**, with a couple of long-term projects — a **scope change rather than a level change**, and now **handles multiple RCA automations through AI across the HL charter**. The scope point matters: the growth showed up as ownership of a materially harder problem space, not as a title.
- **The trajectory in one line: he came in doing ad-hoc SQL support and came out owning AI-driven RCA automation.** Same person, roughly one project cycle.
- **The team's ceiling moved, not just his.** HL now has someone embedded who understands both the semantic layer and agentic orchestration, so that capability no longer depends on the central GenAI pod being available.
- The reallocation decision was validated: the outcome was delivered on 50% of one person's bandwidth plus my own investment, rather than on the fuller resourcing that was originally allocated.

KEY LEARNING

The best signal I had was not on anyone's CV. Tenure, formal bandwidth allocation and prior technical exposure all pointed toward the more experienced analyst, and the thing that actually predicted output was curiosity plus showing up — visible within two weeks if you are paying attention. Backing that meant investing time I had not been given for it, and scoping the work so a one-year BA could stretch without failing: one stream first, daily reviews to unblock rather than to approve, then widening as fast as he earned it. The part I would emphasise is that developing him was also the fastest path to the project outcome — these were not competing objectives. And the measure of whether it worked is not what he delivered on my project; it is that he grew out of it into a role owning AI automation for a whole charter.

What Interviewers Will Probe

- **You reallocated away from a more experienced person within a few weeks. Was that fair to her?** I think the process was fair even though the outcome was

hard. I tried to re-engage her directly, raised it with my manager rather than handling it silently, and gave it a week to change before acting. What I would not defend is spending a month on it — the project had a JFM deadline and no established playbook, and carrying uncertain availability through that would have failed everyone including her. If I am critical of myself, I did not push hard enough to understand *why* her availability was uncertain. There may have been a competing commitment I could have helped resolve.

- **Why back a one-year BA over a two-year analyst on the charter's definitional layer? That's a real risk.** Because the risk was managed by scoping, not by seniority. He started on one stream, with daily reviews, where a wrong design decision cost a day. And the signal I had was directly relevant: good SQL means he could navigate the schema correctly, and curiosity means he would work out the parts neither of us knew. For a project with no playbook, willingness to learn beat two years of experience in a different kind of work.

- **How much of the semantic layer was actually his versus yours?** The design direction and the review were mine early on — I was setting what a good measure and dimension definition looked like. The build was his, and by the time it scaled from one stream to all 4–5 streams and roughly 100 metrics, the decisions were increasingly his too. That progression is the thing I would point at as evidence of development rather than delegation.

- **Did you slow the project down by teaching him rather than building it yourself?** Early on, yes — daily reviews are a real time cost and the first stream took longer than if I had done it. It paid back well inside the project, because once he was independent he covered all remaining streams while I worked on the agentic flow. I could not have done both. And the alternative resourcing was not "me alone, faster" — it was me alone, which would not have delivered.

- **How did he end up in the new role — did you push for him?** His manager saw it directly, which was a better outcome than my advocating for him. Because the semantic layer was being built for HL, his manager was part of the project reviews and watched him not just deliver but bring his own point of view into agent design discussions. My part was making sure the work was legible to that audience, and showing his contribution explicitly in my own weekly reviews rather than absorbing it into the project's results.

- **Did he get anything materially wrong, and what did you do?** Yes — when he started writing agent instructions his prompts were loose and the agent hallucinated. It was the same mistake I had made earlier in the project, so I recognised it fast. I could have rewritten the prompt myself in ten minutes; instead we spent a session debugging it together, working through where the instruction was ambiguous and what the model did with that ambiguity. I taught the principle rather than the fix — structure, negative triggers, and why a confident wrong answer is what a loose prompt produces. It ended up being the single most useful

hour of his development on the project, because it connected the semantic layer he had built to why the agents needed it to be that precise.

- **What did his moving on cost you?** It cost the central pod the person with the deepest working knowledge of the HL semantic layer, and I would still back it — the whole point of the investment was for him to outgrow the project. It also worked out structurally: because the layer is a governed single definition rather than tribal knowledge, and because he stayed within HL, the capability stayed close to where it is used. If he had been the only person who understood an undocumented system, that would be a criticism of how I built it, not of his move.
- **Would you do anything differently?** Two things. I would have brought a second person from HL in earlier so the knowledge was not concentrated in one person. And I would have started the conversation about where he goes next before the project ended, rather than letting it resolve after JFM — the growth happened, but it happened by his manager noticing rather than by a plan.

Why This Story Screams Hire and Develop the Best

- ✔ Hired for the long-term need, not the immediate gap — backed someone who could not yet do the job
- ✔ Recognised exceptional talent on signal (curiosity, SQL quality, output per unit bandwidth) rather than on tenure
- ✔ Made the hard talent call: 50% of someone who showed up over 100% of someone who didn't, with proper process first
- ✔ Invested real resources — daily reviews and coaching outside my mandate, including turning his one real failure (hallucinating agents from loose prompts) into a teaching session rather than a correction
- ✔ Scoped the stretch so it was survivable: one stream, then all 4–5 streams, then ~100 metrics for the charter
- ✔ Deliberately widened his exposure past his assigned scope into agent design, because that was what the next role needed
- ✔ Made his contribution visible to the manager who owned his growth, rather than absorbing it
- ✔ Raised the team's ceiling: HL now owns agentic and semantic capability, and he runs AI-driven RCA automation across the charter